What is System Verilog

- **SystemVerilog** is a combined hardware description language and hardware verification language
- SystemVerilog is an extensive set of enhancements to IEEE 1364 Verilog-2001 standards
- It has features inherited from Verilog HDL,VHDL,C,C++

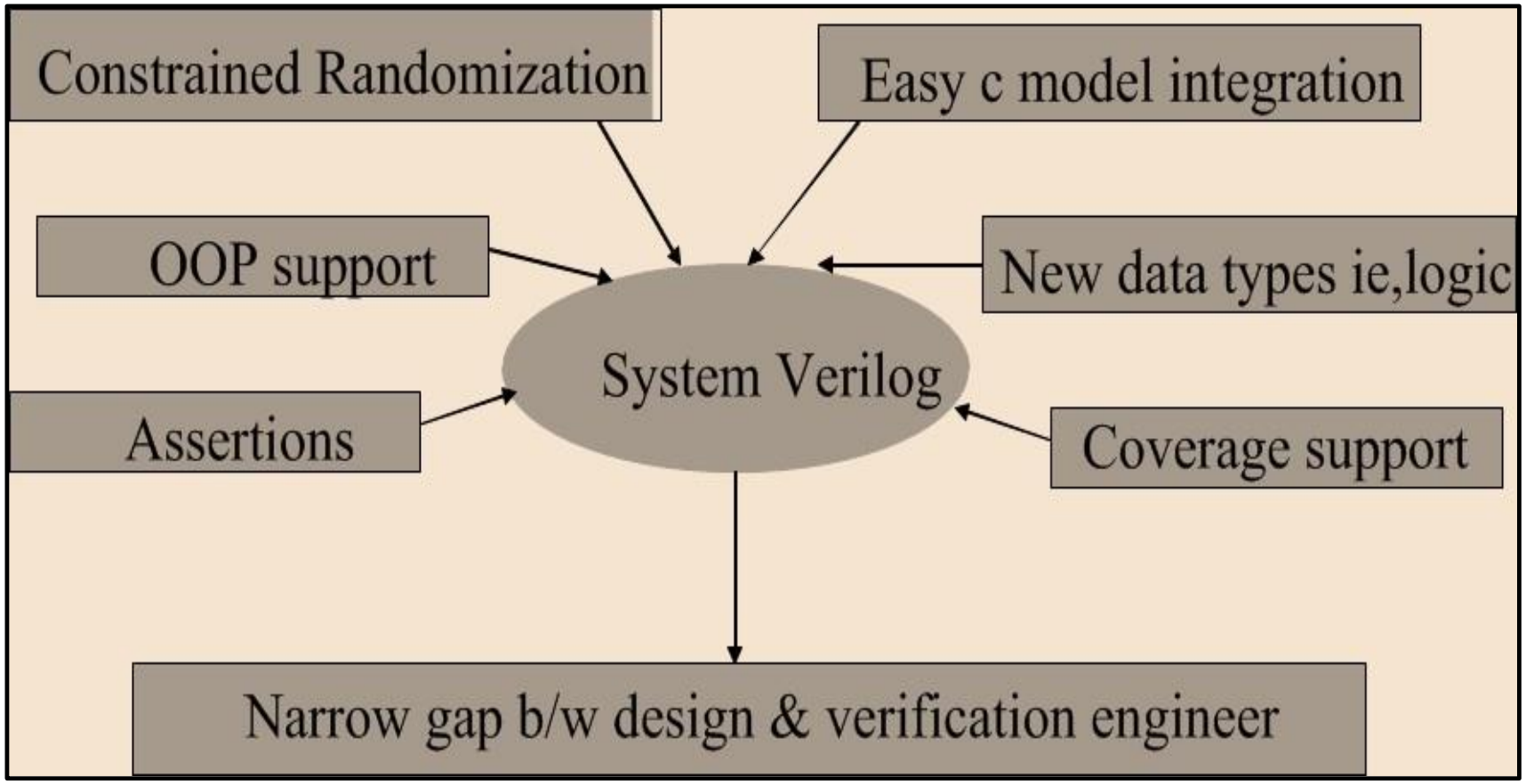
History and evolution of SV

- Verilog (IEEE standard 1364)
  - Began in 1983 as a proprietary language
  - Opened to the public in 1992
  - Became an IEEE standard in 1995 (updated in 2001 and 2005)
  - Between 1983 and 2005 design sizes increased dramatically!
- System Verilog (IEEE standard 1800)
  - Originally intended to be the 2005 update to Verilog
  - Contains hundreds of enhancements and extensions to Verilog
  - Published in 2005 as a separate document
  - Officially superseded Verilog in 2009
  - Updated with more features in 2012 (IEE 1800 2012 standard)

System Verilog – User View

- Has 5 major parts
  - SVD – System Verilog for Design
    - Features supporting Design
  - SVTB – System Verilog for Test benches
    - Test bench specific Features
  - SVA – System Verilog Assertions
    - Features for temporal and concurrent assertions
  - SVDPI – SV Direct Programming Interface
    - For better C/C++ integration
  - SVAPI – SV Application Programming Interface
    - For better Coverage/Assertion integration

Why System Verilog



Verilog Comparison

Verilog

- Used for Design Entry
- Module Level Verification

System Verilog

- Module Level Design
- Constrained Random Verification
- Assertions/Coverage
- One single language for entire design flow

Verilog Comparison – Data types

Verilog

- Strict about usage of wire & reg data type
- Variable types are 4 state – 0,1,X,Z

System Verilog

- Logic data type can be used so no need to worry about reg & wire
- 2 state data type added – 0, 1 state
- 2 state variable can be used in test benches,where X,Z are not required
- 2 state variable in RTL model may enable simulators to be more efficient

Verilog Comparison

Verilog

- Used for Design Entry
- Module Level Verification

System Verilog

- Module Level Design
- Constrained Random Verification
- Assertions/Coverage
- One single language for entire design flow

Verification Capabilities

Verilog

- File I/o
- Random number generation
- Fork/join
- Initial block
- Task & functions
- PLI

System Verilog

- All Verilog features
- Constrained random number generation
- Classes and OOP features
- Fork/join_any,fork/join_none
- Final block
- Task & function enhancements
- DPI

Direct Programming Interface

- DPI is an interface between System Verilog and C that allows inter-language function calls
- Simple to used as compared to PLI's
- Values can be passed directly
- Import Functions
  - SV calls C functions
- Export Functions
  - C calls SV functions

Why not VHDL ?

- VHDL Lacks
  - Constrained Random Generation
  - Functional Coverage
  - Assertions
- Specman E/Vera
  - Used with VHDL and Verilog for Constrained Random generation and Functional Coverage
- PSL (Property Specification Language)
  - Used for Assertions
- Learning 1 language (System Verilog) is better than learning 2 languages (Specman E/Vera and PSL)

Data Types

2 valued data type

bit: *1 bit (packed array)*
byte: *8 bit (character)*
shortint: *16 bit*
int: *32 bit*

4 valued data type

logic: *1 bit (packed array)*

integer: *32 bit*
time: *64 bit unsigned*

```
bit [14:0] bus; // unsigned
bit signed [11:0] i,q;

shortint unsigned addrs;
logic [15:0] addrs;
```

```
string myName = "John Smith";
byte c = "A"; // assign to c "A"
bit [1:4][7:0] s = "hello" ; // assigns to s "ello"
```

Integer Data Type

shortint	2-state (1, 0), 16 bit signed
int	2-state, 32 bit signed
longint	2-state, 64 bit signed
byte	2-state, 8 bit signed
bit	2-state, user-defined vector size
logic	4-state (1,0,x,z) user-def
reg	4-state user-defined size
integer	4-state, 32 bit signed
time	4-state, 64 bit unsigned

Signed/Unsigned

- byte, shortint, int, integer and longint defaults to signed
  - Use unsigned to represent unsigned integer value

    Example: **int unsigned ui;**
- bit, reg and logic defaults to unsigned
- To create vectors, use the following syntax:

  logic [1:0] L; // Creates 2 bit logic vector

To use these types as unsigned, user has to explicitly declare it as unsigned.

int unsigned ui;
int signed si
byte unsigned ubyte;

User can cast using signed and unsigned casting.

if (**signed**'(ubyte)< 150) // ubyte is unsigned

Void

- The void data type represents nonexistent data.
- This type can be specified as the return type of functions to indicate no return value.

```
void = function_call();
```

LITERALS

Integer And Logic Literals

In verilog , to assign a value to all the bits of vector, user has to specify them explicitly.

reg[31:0] a = 32'hffffffff;

Systemverilog Adds the ability to specify unsized literal single bit values with a preceding (').'0, '1, 'X, 'x, 'Z, 'z // sets all bits to this value.

reg[31:0] a = '1;

'x is equivalent to Verilog-2001 'bx
'z is equivalent to Verilog-2001 'bz
'1 is equivalent to making an assignment of all 1's
'0 is equivalent to making an assignment of 0

LITERALS

Time Literals

Time is written in integer or fixed-point format, followed without a space by a time unit (fs ps ns us ms s step).

EXAMPLE
0.1ns
40ps

Array Literals

Array literals are syntactically similar to C initializers, but with the replicate operator ({{}}) allowed.

EXAMPLE
int n[1:2][1:3] = '{'{0,1,2},'{3{4}}};

int n[1:2][1:6] = '{2{'{3{4, 5}}}}; // same as '{'{4,5,4,5,4,5},'{4,5,4,5,4,5}

STRINGS

- In Verilog, string literals are packed arrays of a width that is a multiple of 8 bits which hold ASCII values.
- In Verilog, if a string is larger than the destination string variable, the string is truncated to the left, and the leftmost characters will be lost.
- SystemVerilog adds new keyword "string" which is used to declare string data types unlike verilog.
- String data types can be of arbitrary length and no truncation occurs.

```
string myName = "TEST BENCH";
```

String Methods

- 1. str.len() returns the length of the string, i.e., the number of characters in the string.
- 2. str.putc(i, c) replaces the ith character in str with the given integral value.
- 3. str.getc(i) returns the ASCII code of the ith character in str.
- 4. str.toupper() returns a string with characters in str converted to uppercase.
- 5. str.tolower() returns a string with characters in str converted to lowercase.
- 6. str.compare(s) compares str and s, and return value. This comparison is case sensitive.
- 7. str.icompare(s) compares str and s, and return value .This comparison is case insensitive.
- 8. str.substr(i, j) returns a new string that is a substring formed by index i through j of str.
- 9. str.atoi() returns the integer corresponding to the ASCII decimal representation in str.
- 10. str.atoreal() returns the real number corresponding to the ASCII decimal representation in str.
- 11. str.itoa(i) stores the ASCII decimal representation of i into str (inverse of atoi).
- 12. str.hextoa(i) stores the ASCII hexadecimal representation of i into str (inverse of atohex).
- 13. str.bintoa(i) stores the ASCII binary representation of i into str (inverse of atobin).
- 14. str.realtoa(r) stores the ASCII real representation of r into str (inverse of atoreal)

```
module tb;
   string str = "Hello World!";

   initial begin
       string tmp;

    // Print length of string "str"
      $display ("str.len() = %0d", str.len());

      // Assign to tmp variable and put char "d" at index 3
      tmp = str;
      tmp.putc (3,"d");
      $display ("str.putc(3, d) = %s", tmp);

      // Get the character at index 2
      $display ("str.getc(2) = %s (%0d)", str.getc(2), str.getc(2));

      // Convert all characters to lower case
      $display ("str.tolower() = %s", str.tolower());

      // Comparison
      tmp = "Hello World!";
      $display ("[tmp,str are same] str.compare(tmp) = %0d", str.compare(tmp));
      tmp = "How are you ?";
      $display ("[tmp,str are diff] str.compare(tmp) = %0d", str.compare(tmp));

      // Ignore case comparison
      tmp = "hello world!";
      $display ("[tmp is in lowercase] str.compare(tmp) = %0d", str.compare(tmp));
      tmp = "Hello World!";
      $display ("[tmp,str are same] str.compare(tmp) = %0d", str.compare(tmp));

      // Extract new string from i to j
      $display ("str.substr(4,8) = %s", str.substr (4,8));

   end
endmodule
```

```
str.len() = 12
str.putc(3, d) = Heldo World!
str.getc(2) = l (108)
str.tolower() = hello world!
[tmp,str are same] str.compare(tmp) = 0
[tmp,str are diff] str.compare(tmp) = -1
[tmp is in lowercase] str.compare(tmp) = -1
[tmp,str are same] str.compare(tmp) = 0
str.substr(4,8) = o Wor
```

String Methods

```
module str;
string A;
string B;
initial
begin
A = "TEST ";
B = "Bench";
$display(" %d ",A.len() );
$display(" %s ",A.getc(5) );
$display(" %s ",A.tolower);
$display(" %s ",B.toupper);
$display(" %d ",B.compare(A) );
$display(" %d ",A.compare("test") );
$display(" %s ",A.substr(2,3) ); A = "111";
$display(" %d ",A.atoi() );
end
endmodule
```

RESULTS :

5

test
BENCH
-18
-32
ST
111

String Operators

SystemVerilog provides a set of operators that can be used to manipulate combinations of string variables and string literals.

Equality

Syntax : Str1 == Str2

Inequality.

Syntax: Str1 != Str2

Comparison.

Syntax:
Str1 < Str2
Str1 <= Str2
Str1 > Str2
Str1 >= Str2

Concatenation.

Syntax: {Str1,Str2,...,Strn}

Replication.

Syntax : {multiplier{Str}}

Indexing.

Syntax: Str[index]

String Operators

```
program main;
initial
begin
string str1,str2,str3;
str1 = "TEST BENCH";
str2 = "TEST BENCH";
str3 = "test bench";
if(str1 == str2)
$display(" Str1 and str2 are equal");
else
$display(" Str1 and str2 are not equal");
if(str1 == str3)
$display(" Str1 and str3 are equal");
else
$display(" Str1 and str3 are not equal");

end
endprogram
```

RESULT

Str1 and str2 are equal
Str1 and str3 are not equal

String Operators

```
program main;
initial
begin
string str1,str2,str3;
str1 = "TEST BENCH";
str2 = "TEST BENCH";
str3 = "test bench";
if(str1 != str2)
$display(" Str1 and str2 are not equal");
else
$display(" Str1 and str2 are equal");
if(str1 != str3)
$display(" Str1 and str3 are not equal");
else
$display(" Str1 and str3 are equal");

end
endprogram
```

RESULT

Str1 and str2 are equal
Str1 and str3 are not equal

String Operators

```
program main;
initial
begin
string Str1,Str2,Str3;
Str1 = "c";
Str2 = "d";
Str3 = "e";

if(Str1 < Str2)
$display(" Str1 < Str2 ");
if(Str1 <= Str2)
$display(" Str1 <= Str2 ");
if(Str3 > Str2)
$display(" Str3 > Str2");
if(Str3 >= Str2)
$display(" Str3 >= Str2");
end
endprogram
```

RESULT

Str1 < Str2
Str1 <= Str2
Str3 > Str2
Str3 >= Str2

String Operators

```
program main;
initial
begin
string Str1,Str2,Str3,Str4,Str5;
Str1 = "WWW.";
Str2 = “VITAP";
Str3 = "";
Str4 = “.AC";
Str5 = ".IN";
$display(" %s ",{Str1,Str2,Str3,Str4,Str5});
end
endprogram
```

RESULT

WWW.VITAP.AC.IN

String Operators

```
program main;
initial
begin
string Str1,Str2;
Str1 = "W";
Str2 = ".VITAP.AC.IN";
$display(" %s ",{{3{Str1}},Str2});
end
endprogram
```

RESULT

WWW.VITAP.AC.IN

String Operators

```
program main;
initial
begin
string Str1;
Str1 = "WWW.TESTBENCH.IN";
for(int i =0 ;i < 16 ; i++)
$write("%s ",Str1[i]);
end
endprogram
```

RESULT

W W W . T E S T B E N C H . I N

USERDEFINED DATATYPES

Systemverilog allows the user to define datatypes.
There are different ways to define user defined datatypes.

1. Class.
2. Enumarations.
3. Struct.
4. Union.
5. Typedef.

ENUMARATIONS

- Need for variables that have a limited set of possible values that can be usally referred to by name.
- There's a specific facility, called an enumeration in SystemVerilog .
- Enumerated data types assign a symbolic name to each legal value taken by the data type.

enum {IDLE,READY,BUZY} states;

ENUMARATIONS

- SystemVerilog includes a set of specialized methods to enable iterating over the values of enumerated.

➢ The first() method returns the value of the first member of the enumeration.

The last() method returns the value of the last member of the enumeration.

The next() method returns the Nth next enumeration value (default is the next one) starting from the current value of the given variable.

The prev() method returns the Nth previous enumeration value (default is the previous one) starting from the current value of the given variable.

The name() method returns the string representation of the given enumeration value. If the given value is not a member of the enumeration, the name() method returns the empty string.

ENUMARATIONS

```
module enum_method;
typedef enum {red,blue,green} colour;
colour c;
initial
begin
c = c.first();
$display(" %s ",c.name);
c = c.next();
$display(" %s ",c.name);
c = c.last();
$display(" %s ",c.name);
c = c.prev();
$display(" %s ",c.name);
end
endmodule
```

RESULTS :

red
blue
green
blue

ENUMARATIONS

```
module enum_method;
typedef enum {red,blue,green} colour;
colour c,d;
int i;
initial
begin
$display("%s",c.name());
d = c;
$display("%s",d.name());
d = colour'(c + 1); // use casting
$display("%s",d.name());
i = d; // automatic casting
$display("%0d",i);
c = colour'(i);
$display("%s",c.name());
end
endmodule
```

RESULT

red
red
blue
1
blue

TIP: If you want to use X or Z as enum values, then define it using 4-state data type explicitly.

enum integer {IDLE, XX='x, S1='b01, S2='b10} state, next;

STRUCTURES AND UNIOUNS

Structure:

- The disadvantage of arrays is that all the elements stored in then are to be of the same data type.
- If we need to use a collection of different data types, it is not possible using an array. When we require using a collection of different data items of different data types we can use a structure.
- Structure is a method of packing data of different types.

```
struct {
int a;
byte b;
bit [7:0] c;
} my_data_struct;
```

The structured variables can be accessed using the variable name "my_data_struct".

```
my_data_struct.a = 123;
$display(" a value is %d ",my_data_struct.a);
```

STRUCTURES AND UNIOUNS

Assignments To Struct Members:

- A structure literal must have a type, which may be either explicitly indicated with a prefix or implicitly indicated by an assignment-like context.

```
my_data_struct = `{1234,8'b10,8'h20};
```

Structure literals can also use member name and value, or data type and default value.

```
my_data_struct = `{a:1234,default:8'h20};
```

```
module tb;
    // Create a structure called "st_fruit"
  // which to store the fruit's name, count and expiry date in days.
    // Note: this structure declaration can also be placed outside the module
  struct {
      string fruit;
      int    count;
      byte    expiry;
  } st_fruit;

  initial begin
    // st_fruit is a structure variable, so let's initialize it
    st_fruit = '{"apple", 4, 15};

    // Display the structure variable
    $display ("st_fruit = %p", st_fruit);

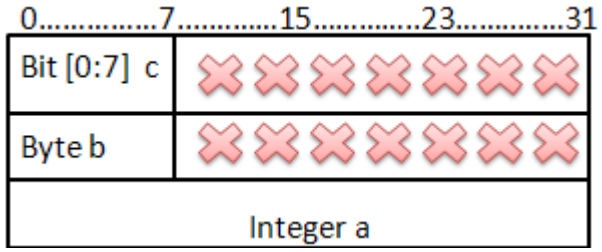
    // Change fruit to pineapple, and expiry to 7
    st_fruit.fruit = "pineapple";
    st_fruit.expiry = 7;
    $display ("st_fruit = %p", st_fruit);
  end
endmodule
```

```
ncsim> run
st_fruit = '{fruit:"apple", count:4, expiry:'hf}
st_fruit = '{fruit:"pineapple", count:4, expiry:'h7}
ncsim: *W,RNQUIE: Simulation is complete.
```

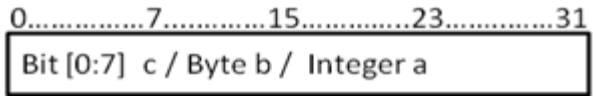
Union

- Unions like structure contain members whose individual data types may differ from one another.
- The members that compose a union all share the same storage area.
- A union allows us to treat the same space in memory as a number of different variables.

```
union {
int a;
byte b;
bit [7:0] c;
} my_data;
```



"my_data_struct".



"my_data_union"

Advantages Of Using Typedef :

- Shorter names are easier to type and reduce typing errors.
- Improves readability by shortening complex declarations.
- Improves understanding by clarifying the meaning of data.
- Changing a data type in one place is easier than changing all of its uses throughout the code.
- Allows defining new data types using structs, unions and Enumerations also.
- Increases reusability.
- Useful is type casting.

```
typedef enum {NO, YES} boolean;
typedef union { int i; shortreal f; } num; // named union
type
typedef struct {
bit isfloat;
union { int i; shortreal f; } n; // anonymous type
} tagged_st; // named structure
```

ARRAYS:

- Arrays hold a fixed number of equally-sized data elements.
- Individual elements are accessed by index using a consecutive range of integers.

Fixed Arrays:

"Packed array" to refer to the dimensions declared before the object name.
"unpacked array" refers to the dimensions declared after the object name.

int Array[8][32]; is the same as: **int** Array[0:7][0:31];

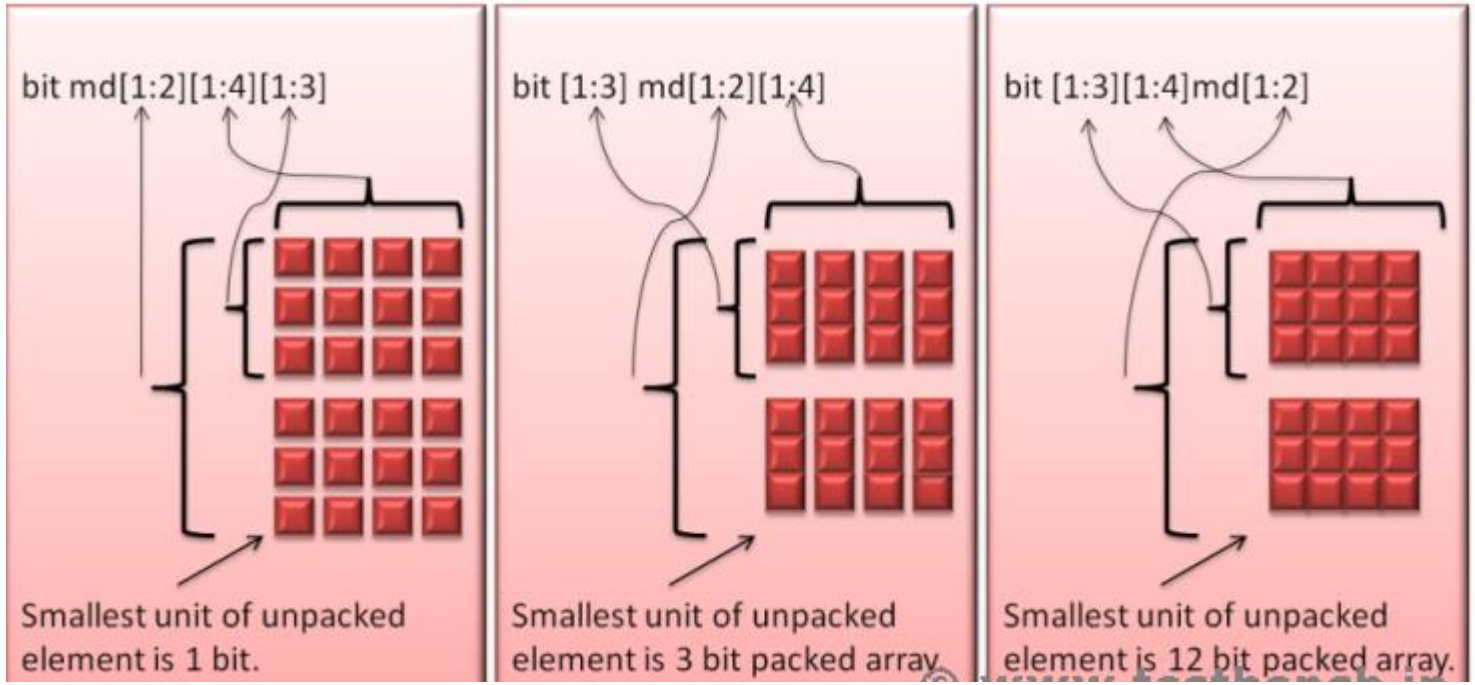
```
// Packed Arrays
reg [0:10] vari; // packed array of 4-bits
wire [31:0] [1:0] vari; // 2-dimensional packed array

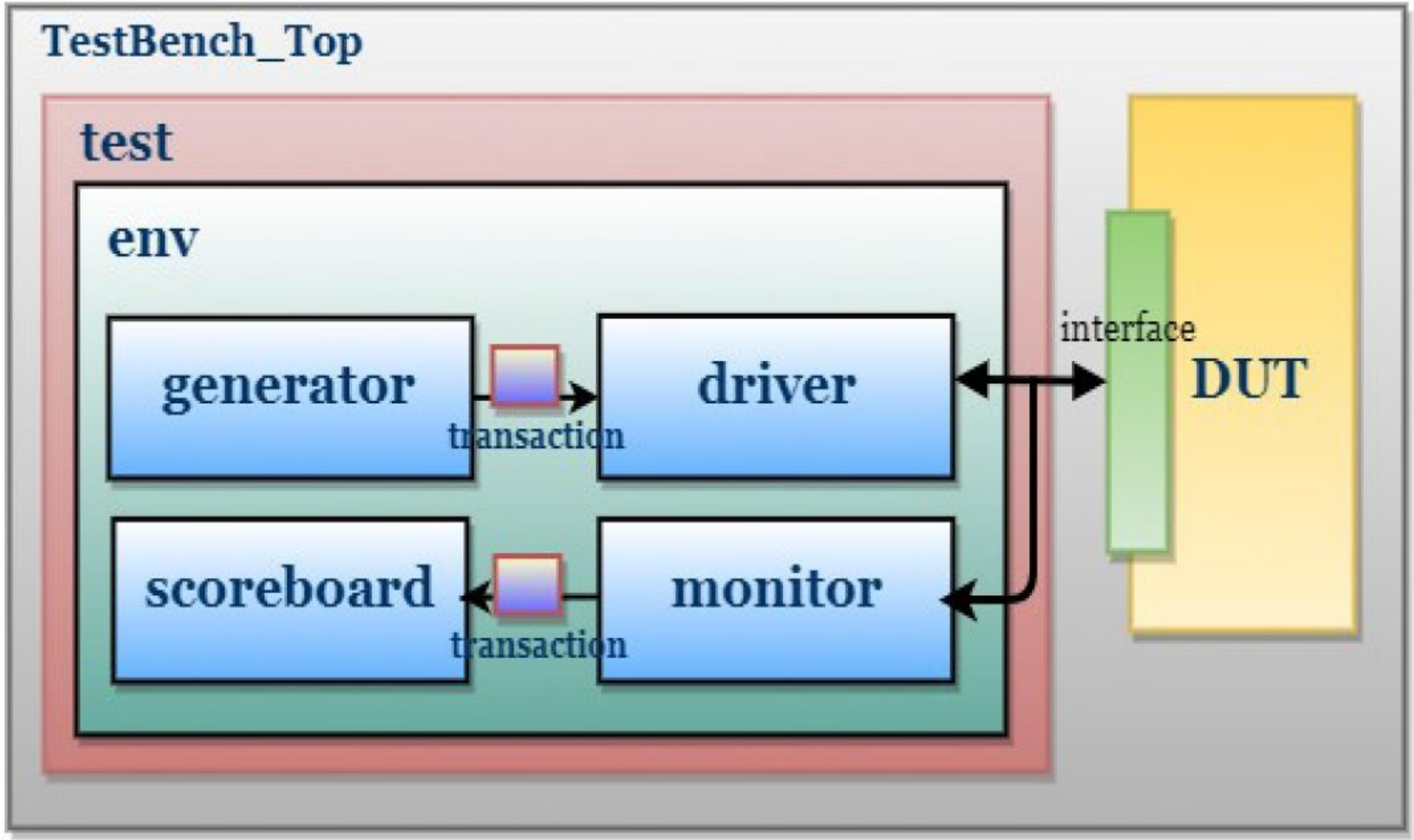
// Unpacked Arrays
wire status [31:0]; // 1 dimensional unpacked array
wire status [32]; // 1 dimensional unpacked array

integer matrix[7:0][0:31][15:0]; // 3-dimensional unpacked array of integers
integer matrix[8][32][16]; // 3-dimensional unpacked array of integers

reg [31:0] registers1 [0:255]; // unpacked array of 256 registers; each
reg [31:0] registers2 [256]; // register is packed 32 bit wide
```

ARRAYS:


bit md[1:2][1:4][1:3]
Smallest unit of unpacked element is 1 bit.
bit [1:3] md[1:2][1:4]
Smallest unit of unpacked element is 3 bit packed array.
bit [1:3][1:4]md[1:2]
Smallest unit of unpacked element is 12 bit packed array.


TestBench_Top
test
env
generator
transaction
driver
interface
DUT
scoreboard
transaction
monitor

TRANSACTION: Transaction class Contains all the interface signals. Input signals declared as "**rand**" to generate stimulus.

INTERFACE: If the design contained hundreds of port signals it would be connect, maintain and re-use those signals. Instead, we can place all the design input-output ports into a container which becomes an *interface* to the DUT. The design can then be driven with values through this interface. A simple interface without modport and clocking block. print function included for debugging purpose.

GENERATOR: Generator class is used to Generate the stimulus (create and randomize the transaction class) and send it to Driver.

DRIVER: Driver class Receives the stimulus (transaction) from a generator and drives the packet level data inside the transaction into pin level (to DUT).

MONITOR: Monitor class observes pin level activity on interface signals and converts into packet level which is sent to the components such as scoreboard.

SCOREBOARD: Receives input and output transactions from monitor's. Computes the expected result and compares it with DUT's output. It Send an error message if there is any mismatch.

ENVIRONMENT: The environment is a container class it Creates generator, driver, monitor, scoreboard and mailboxes. For better controllability.

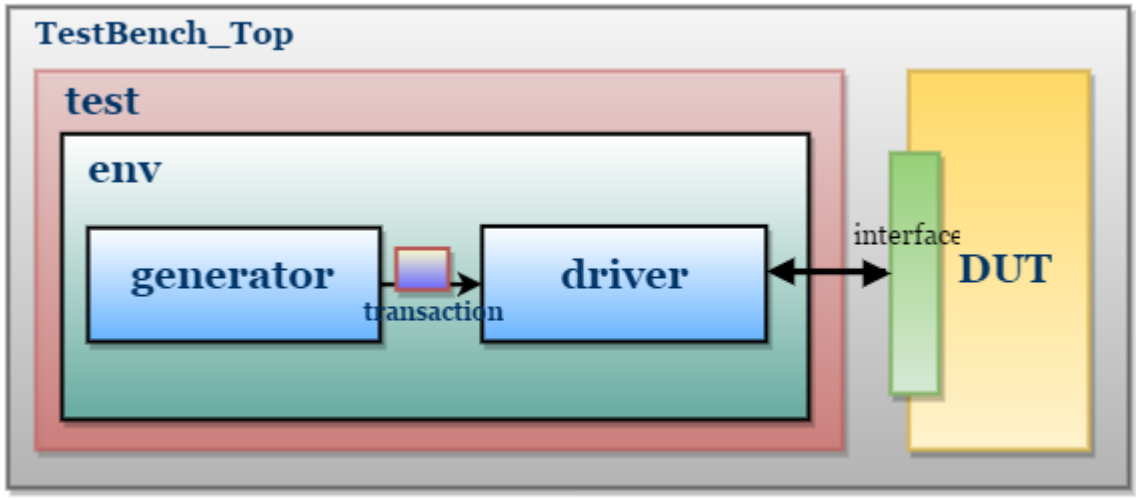
TEST: Test is a program is responsible for,

- Creates the environment.
- Configuring the testbench.
- Initiate the testbench components construction process.
- Initiate the stimulus driving.

TESTBENCH TOP: This is the topmost file, which Contains DUT, TEST and Interface instances. The interface connects the DUT and TestBench.

‘ADDER’ TestBench Without Monitor, Agent and Scoreboard

TestBench Architecture



Transaction Class

•Fields required to generate the stimulus are declared in the transaction class.
•Transaction class can also be used as a placeholder for the activity monitored by the monitor on DUT signals.
•So, the first step is to declare the ‘Fields‘ in the transaction class.
•Below are the steps to write the transaction class.

1. Declaring the fields.

```
class transaction;

  //declaring the transaction items
  bit [3:0] a;
  bit [3:0] b;
  bit [6:0] c;

endclass
```

2. To generate the random stimulus, declare the fields as '**rand**'.

```
class transaction;

  //declaring the transaction items
  rand bit [3:0] a;
  rand bit [3:0] b;
       bit [7:0] c;

endclass
```

3. Adding display() method to display Transaction properti

```
class transaction;

  //declaring the transaction items
  rand bit [3:0] a;
  rand bit [3:0] b;
       bit [6:0] c;
  function void display(string name);
    $display("-------------------------");
    $display("- %s ",name);
    $display("-------------------------");
    $display("- a = %0d, b = %0d",a,b);
    $display("- c = %0d",c);
    $display("-------------------------");
  endfunction
endclass
```

Generator Class

Generator class is responsible for,
•Generating the stimulus by randomizing the transaction class
•Sending the randomized class to driver

```
class generator;

  ------

endclass
```

1. Declare the transaction class handle,

```
class generator;

  //declaring transaction class
  rand transaction trans;

endclass
```

2. ‘Randomize’ the transaction class,

```
class generator;

  //declaring transaction class
  rand transaction trans;

  //main task, generates(create and randomizes) the packets and puts into mailbox
  task main();
      trans = new();
      if( !trans.randomize() ) $fatal("Gen:: trans randomization failed");
      gen2driv.put(trans);
  endtask

endclass
```

3. Adding Mailbox and event,

Mailbox is used to send the randomized transaction to Driver.

Eve

Thi

•De

•Ge

and

nerator

```
class generator;

  //declaring transaction class
  rand transaction trans;

  //declaring mailbox
  mailbox gen2driv;

  //event, to indicate the end of transaction generation
  event ended;

  //constructor
  function new(mailbox gen2driv);
    //getting the mailbox handle from env
    this.gen2driv = gen2driv;
  endfunction

  //main task, generates(create and randomizes) the packets and puts into mailbox
  task main();
      trans = new();
      if( !trans.randomize() ) $fatal("Gen:: trans randomization failed");
      gen2driv.put(trans);
      -> ended; //triggering indicatesthe end of generation
  endtask
```

4. Adding

```
class generator;

  //declaring transaction class
  rand transaction trans;

  //declaring mailbox
  mailbox gen2driv;

  //event, to indicate the end of transaction generation
  event ended;

  //repeat count, to specify number of items to generate
  int repeat_count;

  //constructor
  function new(mailbox gen2driv);
    //getting the mailbox handle from env
    this.gen2driv = gen2driv;
  endfunction

  //main task, generates(create and randomizes) the repeat_count number of transaction
packets and puts into mailbox
  task main();
    repeat(repeat_count) begin
      trans = new();
      if( !trans.randomize() ) $fatal("Gen:: trans randomization failed");
      gen2driv.put(trans);
    end
    -> ended; //triggering indicatesthe end of generation
  endtask
```

5. Adding an event to indicate the completion of the generation process, the event will be triggered on the completion of the Generation process.

```
class generator;

  //declaring transaction class
  rand transaction trans;

  //declaring mailbox
  mailbox gen2driv;

  //repeat count, to specify number of items to generate
  int repeat_count;

  //event, to indicate the end of transaction generation
  event ended;

  //constructor
  function new(mailbox gen2driv);
    //getting the mailbox handle from env
    this.gen2driv = gen2driv;
  endfunction
```

```
  //main task, generates(create and randomizes) the repeat_count number of
packets and puts into mailbox
  task main();

    repeat(repeat_count) begin
      trans = new();
      if( !trans.randomize() ) $fatal("Gen:: trans randomization failed");
      gen2driv.put(trans);
    end
    -> ended; //triggering indicatesthe end of generation
  endtask
endclass
```

Interface

Interface will group the signals.
This is a simple interface without modport and clocking block.

```
interface intf(input logic clk,reset);
  ----
endinterface
```

1. Complete Interface code,

```
interface intf(input logic clk,reset);

  //declaring the signals
  logic    valid;
  logic [3:0] a;
  logic [3:0] b;
  logic [6:0] c;

endinterface
```

Driver Class

Driver class is responsible for,

- receive the stimulus generated from the generator and drive to DUT by assigning transaction class values to interface signals.

```
class driver;
  ----
endclass
```

1. Declare interface and mailbox, Get the interface and mailbox handle through a constructor.

```
//creating virtual interface handle
virtual intf vif;

//creating mailbox handle
mailbox gen2driv;

//constructor
function new(virtual intf vif,mailbox gen2driv);
  //getting the interface
  this.vif = vif;
  //getting the mailbox handle from  environment
  this.gen2driv = gen2driv;
endfunction
```

2. Adding a **reset task**, which initializes the Interface signals to default values.

```
//Reset task, Reset the Interface signals to default/initial values
task reset;
  wait(vif.reset);
  $display("[ DRIVER ] ----- Reset Started -----");
  vif.a <= 0;
  vif.b <= 0;
  vif.valid <= 0;
  wait(!vif.reset);
  $display("[ DRIVER ] ----- Reset Ended   -----");
endtask
```

3. Adding a **drive** task to drive the transaction packet to the interface signal.

```
//drive the transaction items to interface signals
task drive;
    transaction trans;
    gen2driv.get(trans);
    @(posedge vif.clk);
    vif.valid <= 1;
    vif.a     <= trans.a;
    vif.b     <= trans.b;
    @(posedge vif.clk);
    vif.valid <= 0;
    trans.c   <= vif.c;
    @(posedge vif.clk);
    trans.display("[ Driver ]");
    no_transactions++;
  end
endtask
```

4. Adding a local variable to track the number of packets driven, and increment the variable in the drive task. (This
will be useful to end the test-case/Simulation. i.e compare the
generated pkt's and driven pkt's if both are same then end the
simulation)

```
//used to count the number of transactions
int no_transactions;

//drive the transaction items to interface signals
task drive;
  ------
  ------
  no_transactions++;
endtask
```

```
class driver;
  //used to count the number of transacti
  int no_transactions;

  //creating virtual interface handle
  virtual intf vif;

  //creating mailbox handle
  mailbox gen2driv;

  //constructor
  function new(virtual intf vif,mailbox gen2driv);
    //getting the interface
    this.vif = vif;
    //getting the mailbox handles from  environment
    this.gen2driv = gen2driv;
  endfunction
```

```
  //Reset task, Reset the Interface signals to default/i
  task reset;
    wait(vif.reset);
    $display("[ DRIVER ] ----- Reset Started -----");
    vif.a <= 0;
    vif.b <= 0;
    vif.valid <= 0;
    wait(!vif.reset);
    $display("[ DRIVER ] ----- Reset Ended   -----");
  endtask
```

```
  //drivers the transaction items to inter
  task main;
    forever begin
      transaction trans;
      gen2driv.get(trans);
      @(posedge vif.clk);
      vif.valid <= 1;
      vif.a     <= trans.a;
      vif.b     <= trans.b;
      @(posedge vif.clk);
      vif.valid <= 0;
      trans.c   <= vif.c;
      @(posedge vif.clk);
      trans.display("[ Driver ]");
      no_transactions++;
    end
  endtask
```

Environment

Environment is container class contains **Mailbox**, **Generator** and **Driver**.

Creates the mailbox, generator and driver shares the mailbox handle across the Generator and Driver.

```
class environment;
  ---
endclass
```

1. Declare the handles,

```
//generator and driver instance
generator gen;
driver    driv;

//mailbox handle's
mailbox gen2driv;

//virtual interface
virtual intf vif;
```

2. In Construct Method, Create

- Mailbox
- Generator
- Driver

and pass the interface handle through the new() method.

```
//constructor
function new(virtual intf vif);
  //get the interface from test
  this.vif = vif;

  //creating the mailbox (Same handle will be shared across generator and driver)
  gen2driv = new();

  //creating generator and driver
  gen  = new(gen2driv);
  driv = new(vif,gen2driv);
endfunction
```

3. For better accessibility.

Generator and Driver activity can be divided and controlled in three methods.

- pre_test() – Method to call Initialization. i.e, reset method.
- test() – Method to call Stimulus Generation and Stimulus Driving.
- post_test() – Method to wait the completion of generation and driving.

```
task pre_test();
  driv.reset();
endtask

task test();
  fork
    gen.main();
    driv.main();
  join_any
endtask

task post_test();
  wait(gen.ended.triggered);
  wait(gen.repeat_count == driv.no_transactions);
endtask
```

4. Add a run task to call the above methods,

call $finish after post_test() to end the simulation.

```
task run;
  pre_test();
  test();
  post_test();
  $finish;
endtask
```

```
`include "transaction.sv"
`include "generator.sv"
`include "driver.sv"
class environment;

  //generator and driver instance
  generator gen;
  driver    driv;

  //mailbox handle's
  mailbox gen2driv;

  //virtual interface
  virtual intf vif;

  //constructor
  function new(virtual intf vif);
    //get the interface from test
    this.vif = vif;

    //creating the mailbox (Same handle
    gen2driv = new();

    //creating generator and driver
    gen  = new(gen2driv);
    driv = new(vif,gen2driv);
  endfunction

  //
  task pre_test();
    driv.reset();
  endtask

  task test();
    fork
    gen.main();
    driv.main();
    join_any
  endtask

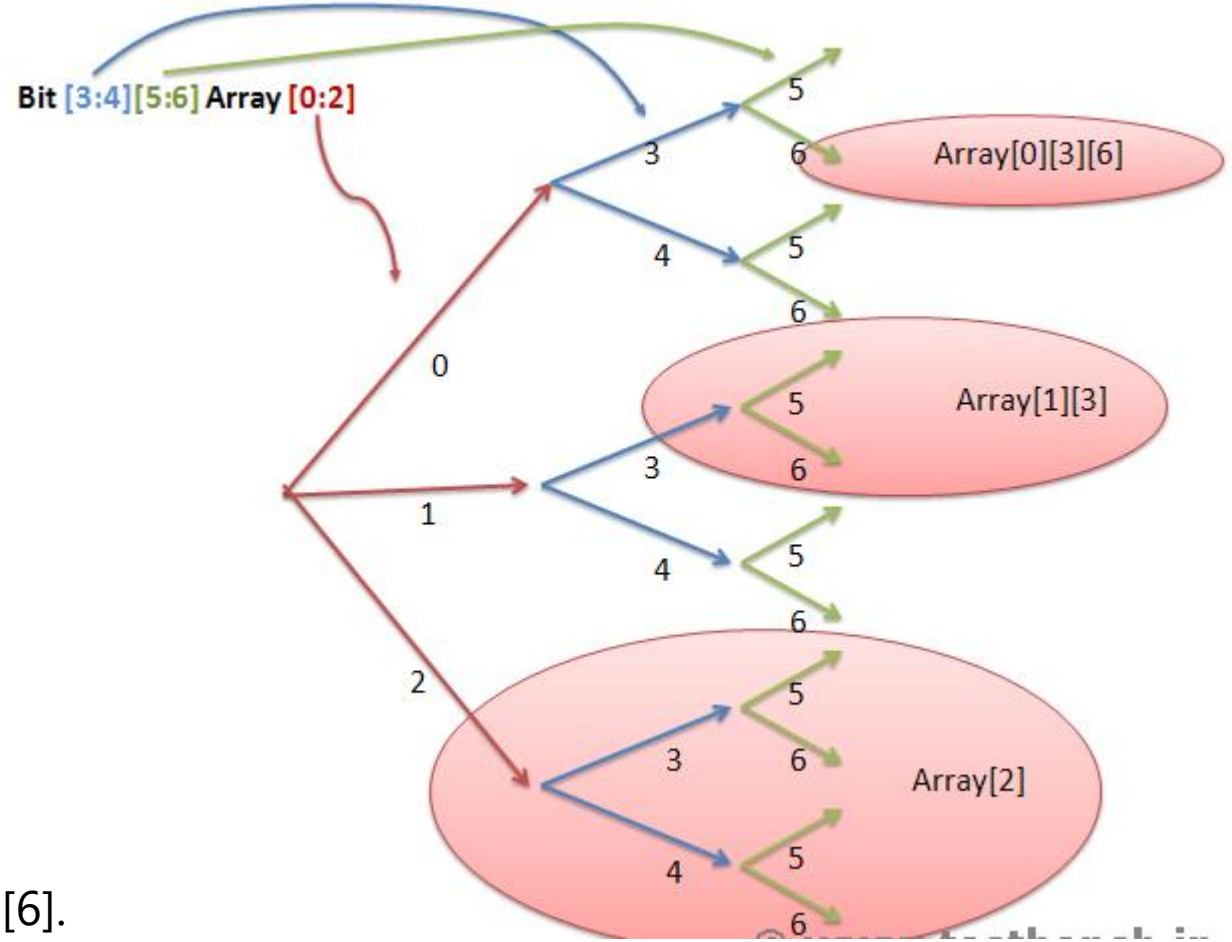
  task post_test();
    wait(gen.ended.triggered);
    wait(gen.repeat_count == driv.no_transactions);
  endtask

  //run task
  task run;
    pre_test();
    test();
    post_test();
    $finish;
  endtask

endclass
```

Accessing Individual Elements Of Multidimensional Arrays:

bit [3:4][5:6]Array [0:2];



Accessing "Array[2]" will access 4 elements
Array[2][3][5],Array[2][3][6],Array[2][4][5] and Array[2][4][6].
Accessing "Array[1][3]" will access 2 elements Array[1][3][5] and Array[1][3][6].
Accessing "Array[0][3][6]" will access one element.

DYNAMIC ARRAYS:

A *dynamic* array dimensions are specified by the empty square brackets [ ] .

```
[data_type] [identifier_name]  [];

bit [7:0]   stack [];      // A dynamic array of 8-
string     names [];      // A dynamic array that c
```

The new() function is used to allocate a size for the array and initialize its elements if required.

Dynamic Array Example

```
module tb;
  // Create a dynamic array that can hold elements o
  int   array [];

  initial begin
    // Create a size for the dynamic array -> size h
    // so that it can hold 5 values
    array = new [5];

    // Initialize the array with five values
    array = '{31, 67, 10, 4, 99};

    // Loop through the array and print their values
    foreach (array[i])
      $display ("array[%0d] = %0d", i, array[i]);
  end
endmodule
```

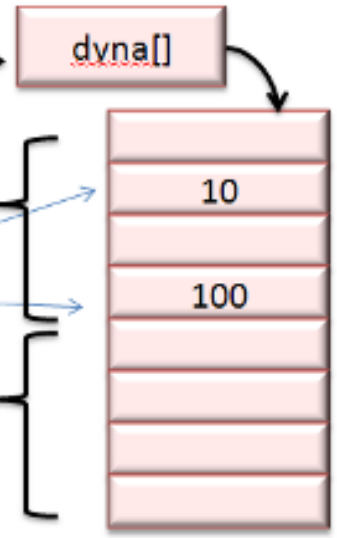
```
ncsim> run
array[0] = 31
array[1] = 67
array[2] = 10
array[3] = 4
array[4] = 99
ncsim: *W,RNQUIE: Simulation is complete.
```

Dynamic Array Methods

Function	Description
`function int size ();`	Returns the current size of the array, 0 if array has not been created
`function void delete ();`	Empties the array resulting in a zero-sized array

DYNAMIC ARRAYS:

```
module dynamic_array();
integer dyna[];
initial
begin
dyna = new[4];
$display(dyna.size);
Dyna[1] = 10; Dyna[3] = 100;
dyna = new[8](dyna);
$display(dyna.size);
dyna.delete;
$display(dyna.size);
end
endmodule
```



RESULT

4
8
0

```
module tb;
  // Create a dynamic array that can hold elements o
  string   fruits [];

  initial begin
    // Create a size for the dynamic array -> size h
    // so that it can hold 5 values
        fruits = new [3];

    // Initialize the array with five values
        fruits = '{"apple", "orange", "mango"};

        // Print size of the dynamic array
    $display ("fruits.size() = %0d", fruits.size());

    // Empty the dynamic array by deleting all items
    fruits.delete();
    $display ("fruits.size() = %0d", fruits.size());
  end
endmodule
```

```
ncsim> run
fruits.size() = 3
fruits.size() = 0
ncsim: *W,RNQUIE: Simulation is complete.
```

How to add new items to a dynamic array ?

- Many times we may need to add new elements to an existing dyanmic array without losing its original contents.

- Since the new() operator is used to allocate a particular size for the array, we also have to copy the old array contents into the new one after creation.

```
int array [];
array = new [10];

// This creates one more slot in the array, while
array = new [array.size() + 1] (array);
```

```
module tb;
  // Create two dynamic arrays of type int
  int array [];
  int id [];

  initial begin
    // Allocate 5 memory locations to "array" and in
    array = new [5];
    array = '{1, 2, 3, 4, 5};

    // Point "id" to "array"
    id = array;

    // Display contents of "id"
    $display ("id = %p", id);

    // Grow size by 1 and copy existing elements to
    id = new [id.size() + 1] (id);

    // Assign value 6 to the newly added location [i
    id [id.size() - 1] = 6;

    // Display contents of new "id"
    $display ("New id = %p", id);

    // Display size of both arrays
    $display ("array.size() = %0d, id.size() = %0d",
  end
endmodule
```

```
run -all;
ncsim> run
id = '{1, 2, 3, 4, 5}
New id = '{1, 2, 3, 4, 5, 6}
array.size() = 5, id.size() = 6
ncsim: *W,RNQUIE: Simulation is complete.
```

System Verilog Array Manipulation

- There are many built-in methods in SystemVerilog to help in array searching and ordering.
- Array manipulation methods simply iterate through the array elements and each element is used to evaluate the expression specified by the with clause

- The iterator argument specifies a local variable that can be used within the with expression to refer to the current element in the iteration.

- If an argument is not provided, item is the name used by default.

Mandatory 'with' clause

Method name	Description
find()	Returns all elements satisfying the given expression
find_index()	Returns the indices of all elements satisfying the given expression
find_first()	Returns the first element satisfying the given expression
find_first_index()	Returns the index of the first element satisfying the given expression
find_last()	Returns the last element satisfying the given expression
find_last_index()	Returns the index of the last element satisfying the given expression

```
module tb;
  int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};
  int res[$];

  initial begin
    res = array.find(x) with (x > 3);
    $display ("find(x)         : %p", res);

    res = array.find_index with (item == 4);
    $display ("find_index      : res[%0d] = 4", res[

    res = array.find_first with (item < 5 & item >=
    $display ("find_first      : %p", res);

    res = array.find_first_index(x) with (x > 5);
    $display ("find_first_index: %p", res);

    res = array.find_last with (item <= 7 & item > 3
    $display ("find_last       : %p", res);

    res = array.find_last_index(x) with (x < 3);
    $display ("find_last_index : %p", res);
  end
endmodule
```

```
ncsim> run
find(x)         : '{4, 7, 5, 7, 6}
find_index      : res[0] = 4
find_first      : '{4}
find_first_index: '{1}
find_last       : '{6}
find_last_index : '{8}
ncsim: *W,RNQUIE: Simulation is complete.
```

Optional 'with' clause

Methods	Description
min()	Returns the element with minimum value or
max()	Returns the element with maximum value or whose expression evaluates to a maximum
unique()	Returns all elements with unique values or whose expression evaluates to a unique value
unique_index()	Returns the indices of all elements with unique values or whose expression evaluates to a unique value

Example

```
module tb;
  int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};
  int res[$];

  initial begin
    res = array.min();
    $display ("min          : %p", res);

    res = array.max();
    $display ("max          : %p", res);

    res = array.unique();
    $display ("unique       : %p", res);

    res = array.unique(x) with (x < 3);
    $display ("unique       : %p", res);

    res = array.unique_index;
    $display ("unique_index : %p", res);
  end
endmodule
```

Simulation Log

```
ncsim> run
min          : '{1}
max          : '{7}
unique       : '{4, 7, 2, 5, 1, 6, 3}
unique       : '{4, 2}
unique_index : '{0, 1, 2, 3, 5, 6, 7}
ncsim: *W,RNQUIE: Simulation is complete.
```

Array Ordering Methods

Method	Description
reverse()	Reverses the order of elements in the array
sort()	Sorts the array in ascending order, optionally using `with` clause
rsort()	Sorts the array in descending order, optionally using `with` clause
shuffle()	Randomizes the order of the elements in the array. `with` clause is not allowed here.

Example

```
module tb;
  int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};

  initial begin
    array.reverse();
    $display ("reverse  : %p", array);

    array.sort();
    $display ("sort     : %p", array);

    array.rsort();
    $display ("rsort    : %p", array);

    for (int i = 0; i < 5; i++) begin
      array.shuffle();
      $display ("shuffle Iter:%0d  = %p", i, array);
    end
  end
endmodule
```

```
ncsim> run
reverse  : '{1, 3, 6, 1, 7, 5, 2, 7, 4}
sort     : '{1, 1, 2, 3, 4, 5, 6, 7, 7}
rsort    : '{7, 7, 6, 5, 4, 3, 2, 1, 1}
shuffle Iter:0  = '{6, 7, 1, 7, 3, 2, 1, 4, 5}
shuffle Iter:1  = '{5, 1, 3, 4, 2, 7, 1, 7, 6}
shuffle Iter:2  = '{3, 1, 7, 4, 6, 7, 1, 2, 5}
shuffle Iter:3  = '{6, 4, 7, 3, 1, 7, 5, 2, 1}
shuffle Iter:4  = '{3, 6, 2, 5, 4, 7, 7, 1, 1}
ncsim: *W,RNQUIE: Simulation is complete.
```

Array Reduction Methods

Method	Description
sum()	Returns the sum of all array elements
product()	Returns the product of all array elements
and()	Returns the bitwise AND (&) of all array elements
or()	Returns the bitwise OR (|) of all array elements
xor()	Returns the bitwise XOR (^) of all array elements

```
module tb;
  int array[4] = '{1, 2, 3, 4};
  int res[$];

  initial begin
    $display ("sum     = %0d", array.sum());
    $display ("product = %0d", array.product());
    $display ("and     = 0x%0h", array.and());
    $display ("or      = 0x%0h", array.or());
    $display ("xor     = 0x%0h", array.xor());
  end
endmodule
```

Simulation Log

```
ncsim> run
sum     = 10
product = 24
and     = 0x0
or      = 0x7
xor     = 0x4
ncsim: *W,RNQUIE: Simulation is complete.
```

SystemVerilog Queue

- A SystemVerilog *queue* is a *First In First Out* scheme which can have a variable size to store elements of the same data type
- It is similar to a one-dimensional unpacked array that grows and shrinks automatically.
- They can also be manipulated by indexing, concatenation and slicing operators. Queues can be passed to tasks/functions as ref or non-ref arguments.

Syntax and Usage

A queue is distinguished by it's specification of the size using **$** operator.

Function	Description
`function int size ();`	Returns the number of items in the queue, 0 if empty
`function void insert (input integer index, input element_t item);`	Inserts the given item at the specified index position
`function void delete ( [input integer index] );`	Deletes the element at the specified index, and if not provided all elements will be deleted
`function element_t pop_front ();`	Removes and returns the first element of the queue
`function element_t pop_back ();`	Removes and returns the last element of the queue
`function void push_front (input element_t item);`	Inserts the given element at the front of the queue

```
[data_type]  [name_of_queue] [$];

string       name_list [$];         // A queue of string elements
bit [3:0]    data [$];              // A queue of 4-bit elements

logic [7:0]  elements [$:127];      // A bounded queue of 8-bits with maximum size of 128

int  q1 [$] =  { 1, 2, 3, 4, 5 };   // Integer queue, initialize elements
int  q2 [$];                        // Integer queue, empty
int  tmp;                           // Temporary variable to store values

tmp = q1 [0];             // Get first item of q1 (index 0) and store in tmp
tmp = q1 [$];             // Get last item of q1 (index 4) and store in tmp
q2  = q1;                 // Copy all elements in q1 into q2
q1  = {};                 // Empty the queue (delete all items)

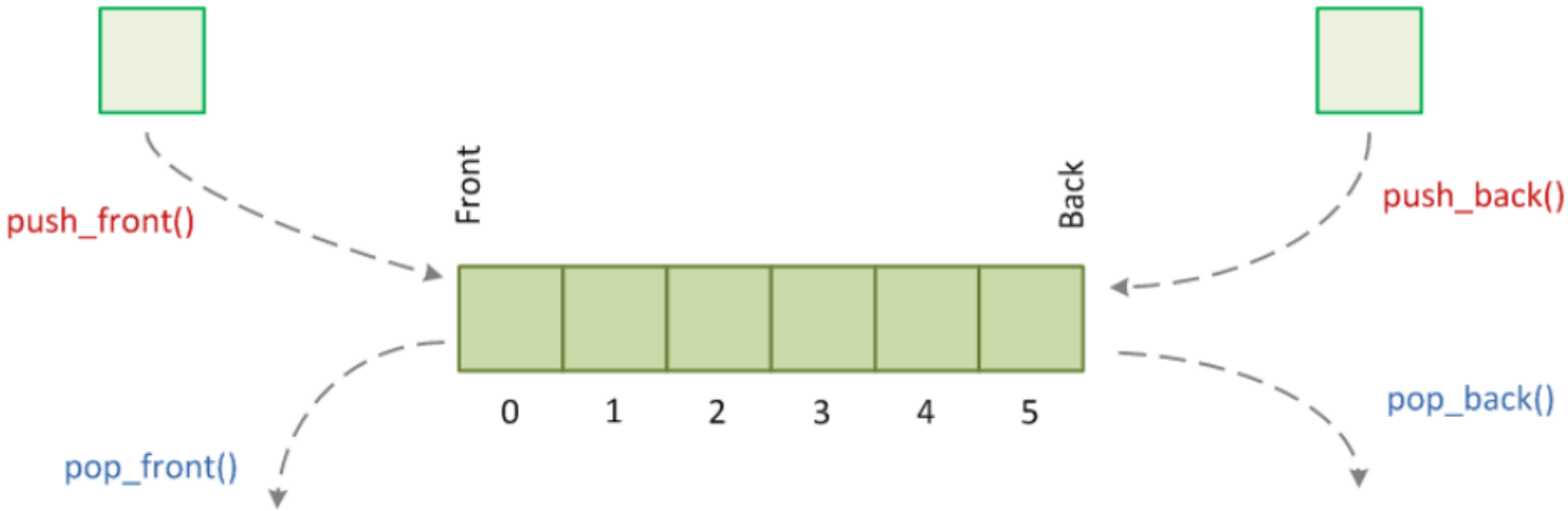
q2[2] = 15;               // Replace element at index 2 with 15
q2.insert (2, 15);        // Inserts value 15 to index# 2
q2 = { q2, 22 };          // Append 22 to q2
q2 = { 99, q2 };          // Put 99 as the first element of q2
q2 = q2 [1:$];            // Delete first item
q2 = q2 [0:$-1];          // Delete last item
q2 = q2 [1:$-1];          // Delete first and last item
```

```
module tb;
    // Create a queue that can store "string" values
    string   fruits[$] =  { "orange", "apple", "kiwi" };

  initial begin
       // Iterate and access each queue element
      foreach (fruits[i])
        $display ("fruits[%0d] = %s", i, fruits[i]);

      // Display elements in a queue
      $display ("fruits = %p", fruits);

        // Delete all elements in the queue
        fruits = {};
        $display ("After deletion, fruits = %p", fruits);
  end
endmodule
```


push_front()
Front
Back
push_back()
0
1
2
3
4
5
pop_front()
pop_back()

SystemVerilog Loops

forever	Runs the given set of statements forever
repeat	Repeats the given set of statements for a given number
while	Repeats the given set of statments as long as given condition is true
for	Similar to while loop, but more condense and popular form
do while	Repeats the given set of statements atleast once, and then loops as long as condition is true
foreach	Used mainly to iterate through all elements in an array

forever

This is an infinite loop, just like `while (1)` .
Note that your simulation will hang unless you include a time delay inside the `forever` block to advance simulation time.

```
module tb;

  // This initial block has a forever loop which wil
  // Hence this block will never finish in simulatio
  initial begin
    forever begin
      #5 $display ("Hello World !");
    end
  end

  // Because the other initial block will run foreve
  // To avoid that, we will explicity terminate simu
  initial
    #50 $finish;
endmodule
```

```
ncsim> run
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Simulation complete via $finish(1) at time 50 NS + 0
```

repeat

Used to repeat statements in a block a certain number of times.

```
module tb;

    // This initial block will execute a repeat stat
  initial begin

      // Repeat everything within begin end 5 time
    repeat(5) begin
      $display ("Hello World !");
    end
  end
endmodule
```

Simulation Log

```
ncsim> run
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
ncsim: *W,RNQUIE: Simulation is complete.
```

while

It'll repeat the block as long as the condition is true.

```
initial begin
  bit [3:0] counter;

  while (counter < 10) begin
    @(posedge clk);
    counter++;
  end
  $display ("Counter = %0d", counter);       // Count
end
```

For

```
module tb;
  string array [5] = '{"apple", "orange", "pear", "blueberry", "lemon"};

  initial begin
    for (int i = 0; i < $size(array); i++)
      $display ("array[%0d] = %s", i, array[i]);
  end
endmodule
```

```
ncsim> run
array[0] = apple
array[1] = orange
array[2] = pear
array[3] = blueberry
array[4] = lemon
ncsim: *W,RNQUIE: Simulation is complete.
```

while loop

```
module tb;
  initial begin
    int cnt = 0;
    while (cnt < 5) begin
      $display("cnt = %0d", cnt);
      cnt++;
    end
  end
endmodule
```

```
ncsim> run
cnt = 0
cnt = 1
cnt = 2
cnt = 3
cnt = 4
ncsim: *W,RNQUIE: Simulation is complete.
```

do while loop

```
module tb;
  initial begin
    int cnt = 0;
    do begin
      $display("cnt = %0d", cnt);
      cnt++;
    end while (cnt < 5);
  end
endmodule
```

```
ncsim> run
cnt = 0
cnt = 1
cnt = 2
cnt = 3
cnt = 4
ncsim: *W,RNQUIE: Simulation is complete.
```

/

SystemVerilog 'break’and 'continue'

```
module tb;
    initial begin

      // This for loop increments i from 0 to 9 and
      for (int i = 0 ; i < 10; i++) begin
        $display ("Iteration [%0d]", i);

        // Let's create a condition such that the
        // for loop exits when i becomes 7
        if (i == 7)
          break;
      end
    end
endmodule
```

```
ncsim> run
Iteration [0]
Iteration [1]
Iteration [2]
Iteration [3]
Iteration [4]
Iteration [5]
Iteration [6]
Iteration [7]
ncsim: *W,RNQUIE: Simulation is complete.
```

```
module tb;
    initial begin

      // This for loop increments i from 0 to 9 and
      for (int i = 0 ; i < 10; i++) begin

        // Let's create a condition such that the
        // for loop
        if (i == 7)
          continue;

        $display ("Iteration [%0d]", i);
      end
    end
endmodule
```

```
ncsim> run
Iteration [0]
Iteration [1]
Iteration [2]
Iteration [3]
Iteration [4]
Iteration [5]
Iteration [6]
Iteration [8]
Iteration [9]
ncsim: *W,RNQUIE: Simulation is complete.
```

SystemVerilog 'unique' and 'priority' if-else

unique-if, unique0-if

`unique-if` evaluates conditions in any order and does the following :

- report an error when none of the `if` conditions match unless there is an explicit `else` .
- report an erorr when there is more than 1 match found in the if else conditions

Unlike *unique-if*, *unique**0**-if* does not report a violation if none of the conditions match

No else block for unique-if

```
module tb;
  int x = 4;

    initial begin
        // This if else if construct is declared to
    // Error is not reported here because there is a
        // clause in the end which will be triggered
        // the conditions match
      unique if (x == 3)
          $display ("x is %0d", x);
      else if (x == 5)
          $display ("x is %0d", x);
        else
          $display ("x is neither 3 nor 5");

        // When none of the conditions become true a
        // is no "else" clause, then an error is rep
      unique if (x == 3)
          $display ("x is %0d", x);
      else if (x == 5)
          $display ("x is %0d", x);
    end
endmodule
```

```
ncsim> run
x is neither 3 nor 5
ncsim: *W,NOCOND: Unique if violation:  Every if clause
            File: ./testbench.sv, line = 18, pos = 13
           Scope: tb
            Time: 0 FS + 1

ncsim: *W,RNQUIE: Simulation is complete.
```

Multiple matches in unique-if

```
module tb;
  int x = 4;

    initial begin

        // This if else if construct is declared to
    // When multiple if blocks match, then error is
        unique if (x == 4)
          $display ("1. x is %0d", x);
        else if (x == 4)
          $display ("2. x is %0d", x);
        else
          $display ("x is not 4");
    end
endmodule
```

```
ncsim> run
1. x is 4
ncsim: *W,MCONDE: Unique if violation:  Multiple true if
            File: ./testbench.sv, line = 8, pos = 15
           Scope: tb
            Time: 0 FS + 1

ncsim: *W,RNQUIE: Simulation is complete.
```

priority-if

`priority-if` evaluates all conditions in sequential order and a violation is reported when:

None of the conditions are true or if there's no `else` clause to the final `if` construct

```
module tb;
  int x = 4;

    initial begin
        // This if else if construct is declared to
    // Error is not reported here because there is a
        // clause in the end which will be triggered
        // the conditions match
      priority if (x == 3)
          $display ("x is %0d", x);
      else if (x == 5)
          $display ("x is %0d", x);
        else
          $display ("x is neither 3 nor 5");

        // When none of the conditions become true a
        // is no "else" clause, then an error is rep
      priority if (x == 3)
          $display ("x is %0d", x);
      else if (x == 5)
          $display ("x is %0d", x);
    end
endmodule
```

```
ncsim> run
x is neither 3 nor 5
ncsim: *W,NOCOND: Priority if violation:  Every if claus
            File: ./testbench.sv, line = 18, pos = 15
           Scope: tb
            Time: 0 FS + 1

ncsim: *W,RNQUIE: Simulation is complete.
```

Exit after first match in priority-if

```
module tb;
  int x = 4;

    initial begin
        // Exits if-else block once the first match
        priority if (x == 4)
          $display ("x is %0d", x);
      else if (x != 5)
          $display ("x is %0d", x);
    end
endmodule
```

```
ncsim> run
x is 4
ncsim: *W,RNQUIE: Simulation is complete.
```

unique : No items match for given expression

```
module tb;
  bit [1:0]   abc;

  initial begin
    abc = 1;

    // None of the case items match the value in "ab
    // A violation is reported here
    unique case (abc)
      0 : $display ("Found to be 0");
      2 : $display ("Found to be 2");
    endcase
  end
endmodule
```

```
ncsim> run
ncsim: *W,NOCOND: Unique case violation:  Every case ite
            File: ./testbench.sv, line = 9, pos = 14
           Scope: tb
            Time: 0 FS + 1

ncsim: *W,RNQUIE: Simulation is complete.
```

unique : More than one case item matches

```
module tb;
  bit [1:0]   abc;

  initial begin
    abc = 0;

    // Multiple case items match the value in "abc"
    // A violation is reported here
    unique case (abc)
      0 : $display ("Found to be 0");
      0 : $display ("Again found to be 0");
      2 : $display ("Found to be 2");
    endcase
  end
endmodule
```

```
ncsim> run
Found to be 0
ncsim: *W,MCONDE: Unique case violation:  Multiple match
            File: ./testbench.sv, line = 9, pos = 14
           Scope: tb
            Time: 0 FS + 1

ncsim: *W,RNQUIE: Simulation is complete.
```

priority case

```
module tb;
  bit [1:0]     abc;

  initial begin
    abc = 0;

    // First match is executed
    priority case (abc)
      0 : $display ("Found to be 0");
      0 : $display ("Again found to be 0");
      2 : $display ("Found to be 2");
    endcase
  end
endmodule
```

```
ncsim> run
Found to be 0
ncsim: *W,RNQUIE: Simulation is complete.
```

SystemVerilog fork join

- SystemVerilog provides support for parallel or concurrent threads through fork join construct.
- Multiple procedural blocks can be spawned off at the same time using fork and join .
- There are variations to fork join that allow the main thread to continue executing rest of the statements based on when child threads finish.

```
module tb;
  initial begin
      $display ("[%0t] Main Thread: Fork join going to start", $time);
    fork
      // Thread 1
      #30 $display ("[%0t] Thread1 finished", $time);

      // Thread 2
            begin
                #5 $display ("[%0t] Thread2 ...", $time);
        #10 $display ("[%0t] Thread2 finished", $time);
            end

            // Thread 3
      #20 $display ("[%0t] Thread3 finished", $time);
    join
      $display ("[%0t] Main Thread: Fork join has finished", $time);
  end
endmodule
```

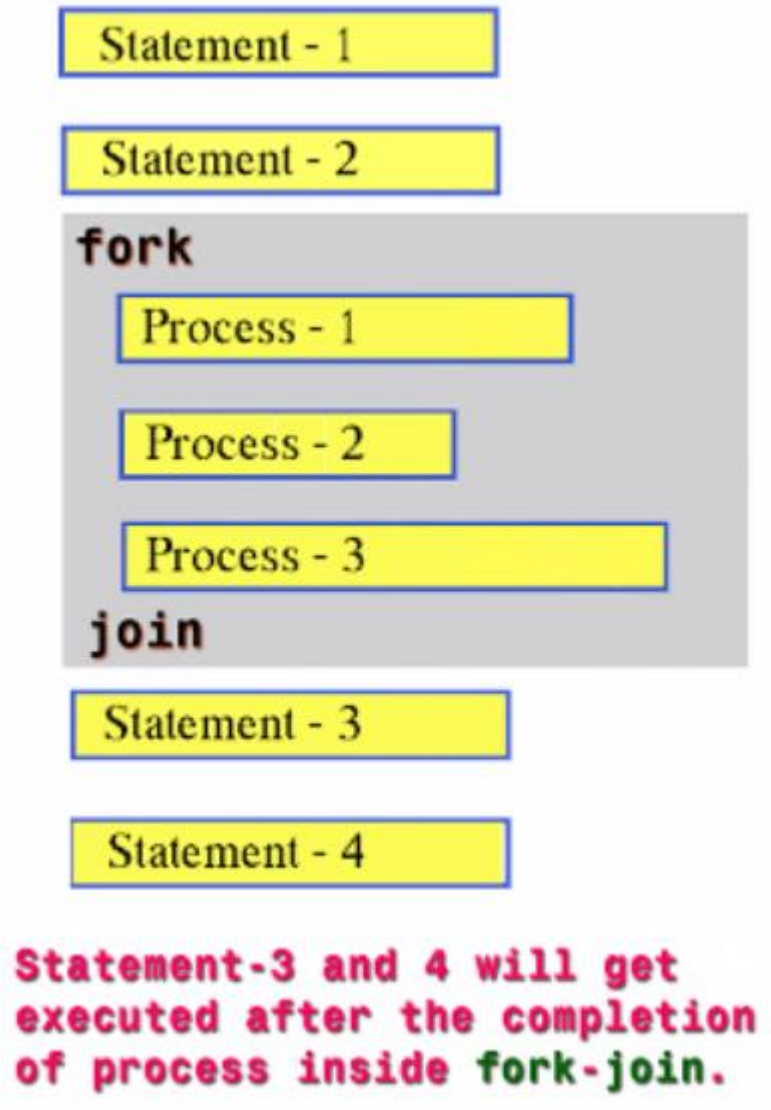
```
ncsim> run
[0] Main Thread: Fork join going to start
[5] Thread2 ...
[15] Thread2 finished
[20] Thread3 finished
[30] Thread1 finished
[30] Main Thread: Fork join has finished
ncsim: *W,RNQUIE: Simulation is complete.
```

SystemVerilog Fork Join

fork join

Fork-Join will start all the processes inside it parallel and wait for the completion of all the processes.

- fork block will be blocked until the completion of process-1 and Process-2.
- Both process-1 and Process-2 will start at the same time,
- Process-1 will finish at 5ns and Process-2 will finish at 20ns.
- fork-join will be unblocked at 20ns.



SystemVerilog Fork Join

```
module fork_join;

  initial begin
    $display("------------------------------------------------------------------");
    fork
      //--------------------
      //Process-1
      //--------------------
      begin
        $display($time,"\tProcess-1 Started");
        #5;
        $display($time,"\tProcess-1 Finished");
      end

      //--------------------
      //Process-2
      //--------------------
      begin
        $display($time,"\tProcess-2 Started");
        #20;
        $display($time,"\tProcess-2 Finished");
      end
    join

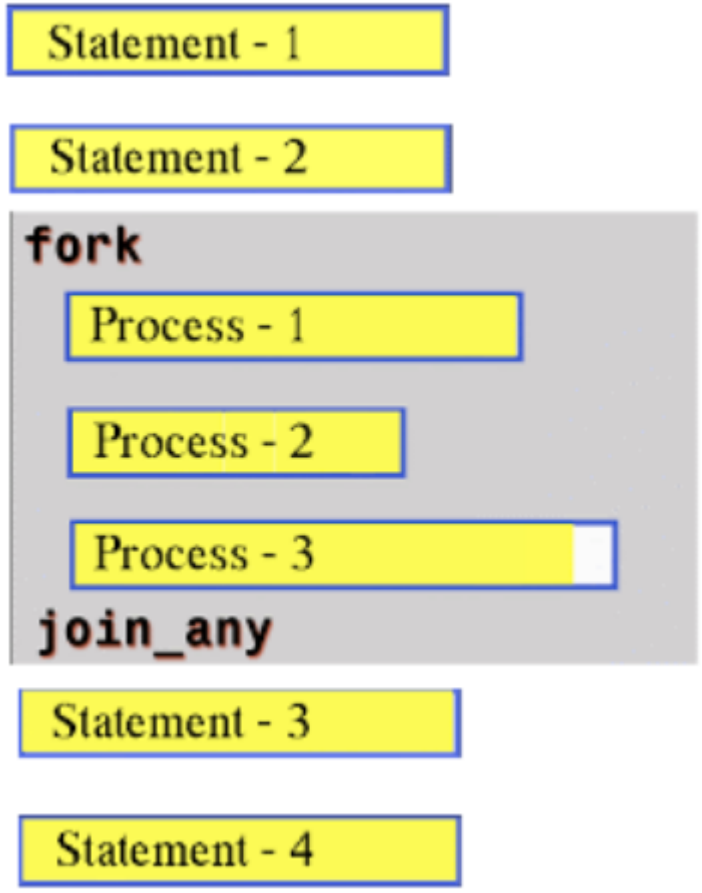
    $display($time,"\tOutside Fork-Join");
    $display("------------------------------------------------------------------");
    $finish;
  end
endmodule
```

```
0 Process-1 Started
0 Process-2 Startedt
5 Process-1 Finished
20 Process-2 Finished
20 Outside Fork-Join
```

SystemVerilog fork join_any

Fork-Join_any will be unblocked after the completion of any of the Processes.

fork block will be blocked until the completion of any of the Process Process-1 or Process-2.
fork-join_any will be unblocked at 5ns



```
module fork_join;
  initial begin
    $display("------------------------------------------------------------");
    fork
      //Process-1
      begin
        $display($time,"\tProcess-1 Started");
        #5;
        $display($time,"\tProcess-1 Finished");
      end

      //Process-2
      begin
        $display($time,"\tProcess-2 Started");
        #20;
        $display($time,"\tProcess-2 Finished");
      end
    join_any

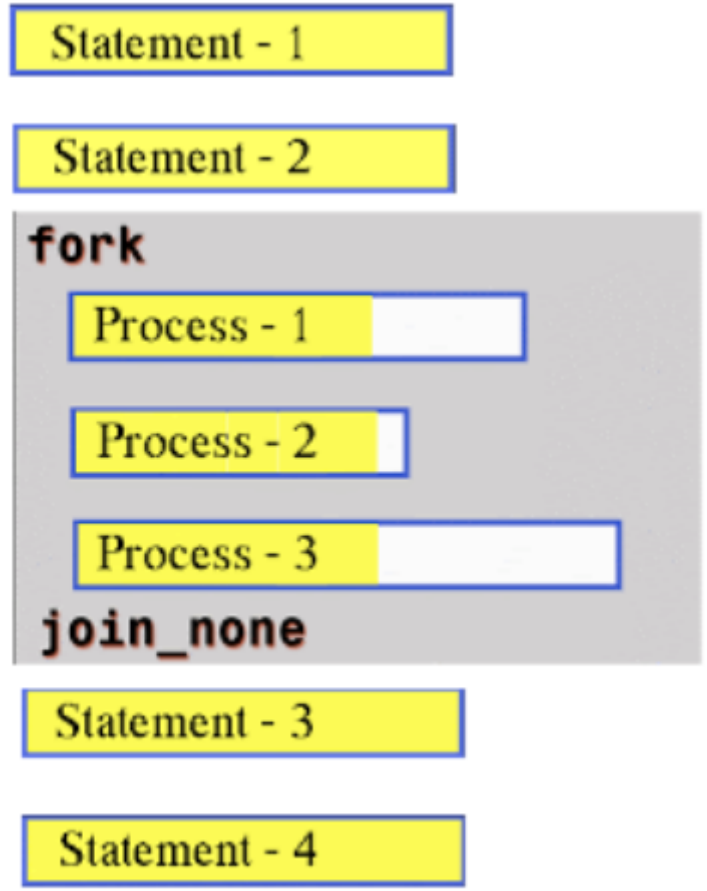
    $display($time,"\tOutside Fork-Join");
    $display("------------------------------------------------
  end
endmodule
```

```
------------------------------------------------
0 Thread-1 Started
0 Thread-2 Started
5 Thread-1 Finished
5 Outside Fork-Join
------------------------------------------------
20 Process-2 Finished
```

SystemVerilog fork join_none

fork join_none

- As in the case of Fork-Join and Fork-Join_any fork block is blocking, but in case of Fork-Join_none fork block will be non-blocking.
- Processes inside the fork-join_none block will be started at the same time, fork block will not wait for the completion of the Process inside the fork-join_none.



```
module fork_join_none;
  initial begin
    $display("------------------------------------------------------------");

    fork
      //Process-1
      begin
        $display($time,"\tProcess-1 Started");
        #5;
        $display($time,"\tProcess-1 Finished");
      end
      //Process-2
      begin
        $display($time,"\tProcess-2 Startedt");
        #20;
        $display($time,"\tProcess-2 Finished");
      end
    join_none

    $display($time,"\tOutside Fork-Join_none");
    $display("------------------------------------------
  end
endmodule
```

```
------------------------------------------------------
        0 Outside Fork-Join_none
------------------------------------------------------
        0 Process-1 Started
        0 Process-2 Startedt
        5 Process-1 Finished
       20 Process-2 Finished
```

wait fork

wait fork; causes the process to block until the completion of all processes started from fork blocks.

```
module wait_fork;

  initial begin
    $display("------------------------------------------------------------------");


    fork
      //Process-1
      begin
        $display($time,"\tProcess-1 Started");
        #5;
        $display($time,"\tProcess-1 Finished");
      end

      //Process-2
      begin
        $display($time,"\tProcess-2 Started");
        #20;
        $display($time,"\tProcess-2 Finished");
      end
    join_any

   $display("------------------------------------------------------------------");
$finish; //ends the simulation

  end
endmodule
```

```
------------------------------------------------------------

                  0 Process-1 Started

                  0 Process-2 Started

                  5 Process-1 Finished

------------------------------------------------------------
```

```
module wait_fork;

  initial begin
    $display("------------------------------------------------------------");

    fork
      //Process-1
      begin
        $display($time,"\tProcess-1 Started");
        #5;
        $display($time,"\tProcess-1 Finished");
      end

      //Process-2
      begin
        $display($time,"\tProcess-2 Started");
        #20;
        $display($time,"\tProcess-2 Finished");
      end
    join_any
    wait fork; //waiting for the completion of active fork threads

    $display("------------------------------------------------------------");
    $finish; //ends the simulation
  end
endmodule
```

```
------------------------------------------------------------
                   0 Process-1 Started
                   0 Process-2 Started
                   5 Process-1 Finished
                  20 Process-2 Finished
------------------------------------------------------------
```

```
module tb;
  initial begin
      $display ("[%0t] Main Thread: Fork join going to start", $time);
    fork
      fork
              display (20, "Thread1_0");
              display (30, "Thread1_1");
            join
          display (10, "Thread2");
    join
      $display ("[%0t] Main Thread: Fork join has finished", $time);
  end

  task display (int _time, string t_name);
    #(_time) $display ("[%0t] %s", $time, t_name);
  endtask
endmodule
```

```
ncsim> run
[0] Main Thread: Fork join going to start
[10] Thread2
[20] Thread1_0
[30] Thread1_1
[30] Main Thread: Fork join has finished
ncsim: *W,RNQUIE: Simulation is complete.
```

```
module tb;
    initial begin
      $display ("[%0t] Main Thread: Fork join going to start", $time);
        fork
            fork // Thread 1
              #50 $display ("[%0t] Thread1_0 ...", $time);
              #70 $display ("[%0t] Thread1_1 ...", $time);
              begin
                #10 $display ("[%0t] Thread1_2 ...", $time);
                #100 $display ("[%0t] Thread1_2 finished", $time);
              end
            join

            // Thread 2
            begin
                #5 $display ("[%0t] Thread2 ...", $time);
                #10 $display ("[%0t] Thread2 finished", $time);
            end

            // Thread 3
            #20 $display ("[%0t] Thread3 finished", $time);
        join
      $display ("[%0t] Main Thread: Fork join has finished", $time);
    end
endmodule
```

```
ncsim> run
[0] Main Thread: Fork join going to start
[5] Thread2 ...
[10] Thread1_2 ...
[15] Thread2 finished
[20] Thread3 finished
[50] Thread1_0 ...
[70] Thread1_1 ...
[110] Thread1_2 finished
[110] Main Thread: Fork join has finished
ncsim: *W,RNQUIE: Simulation is complete.
```

Tasks and Functions

- Tasks
- Functions
- Tasks and Functions argument passing
- **Import and Export functions**

Difference between `function` and `task`

Function	Task
Cannot have time-controlling statements/delay, and hence executes in the same simulation time unit	Can contain time-controlling statements/delay and may only complete at some other time
Cannot enable a task	Can enable other tasks and functions
Should have atleast one input argument and cannot have output or inout arguments	Can have zero or more arguments of any type
Can return only a single value	Cannot return a value, but can achieve the same effect using output arguments

SystemVerilog Tasks

- Tasks and Functions provide a means of splitting code into small parts.
- A Task can contain a declaration of parameters, input arguments, output arguments, in-out arguments, registers, events, and zero or more behavioral statements.

SystemVerilog task can be,

- Static: Static tasks share the same storage space for all task calls.
- Automatic: Automatic tasks allocate unique, stacked storage for each task call

SystemVerilog allows,

- to declare an automatic variable in a static task
- to declare a static variable in an automatic task
- more capabilities for declaring task ports
- multiple statements within task without requiring a begin...end or fork...join block
- returning from the task before reaching the end of the task
- passing values by reference, value, names, and position
- default argument values
- the default direction of argument is input if no direction has been specified
- default arguments type is logic if no type has been specified

task examples

task arguments in parentheses

```
module sv_task;
  int x;

  //task to add two integer numbers.
  task sum(input int a,b,output int c);
    c = a+b;
  endtask

  initial begin
    sum(10,5,x);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator output:

Value of x=15

task arguments in declarations and mentioning directions

```
module sv_task;
  int x;

  //task to add two integer numbers.
  task sum;
    input int a,b;
    output int c;
    c = a+b;
  endtask

  initial begin
    sum(10,5,x);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator output:

Value of x = 15

SystemVerilog Functions

A Function can contain declarations of range, returned type, parameters, input arguments, registers, and events.

- A function without a range or return type declaration returns a one-bit value
- Any expression can be used as a function call argument
- Functions cannot contain any time-controlled statements, and they cannot enable tasks
- Functions can return only one value

SystemVerilog allows,

- to declare an automatic variable in static functions
- to declare the static variable in automatic functions
- more capabilities for declaring function ports
- multiple statements within a function without requiring a begin...end or fork...join block
- returning from the function before reaching the end of the function
- Passing values by reference, value, names, and position
- default argument values
- function output and inout ports
- the default direction of argument is input if no direction has been specified.
- default arguments type is logic if no type has been specified.

function arguments in parentheses

```
module sv_function;
  int x;
  //function to add two integer numbers.
  function int sum(input int a,b);
    sum = a+b;
  endfunction

  initial begin
    x=sum(10,5);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator output:

Value of x=15

function arguments in declarations and mentioning directions

```
module sv_function;
  int x;

  //function to add two integer numbers.
  function int sum;
    input int a,b;
    sum = a+b;
  endfunction
  initial begin
    x=sum(10,5);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator output:

Value of x = 15

function with return value with the return keyword

```
module sv_function;
  int x;

  //function to add two integer numbers.
  function int sum;
    input int a,b;
    return a+b;
  endfunction

  initial begin
    x=sum(10,5);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator Output

Value of x = 15

Void function

```
module sv_function;
int x;

  //void function to display current simulation time
  function void current_time;
    $display("\tCurrent simulation time is %0d",$time);
  endfunction

  initial begin
    #10;
    current_time();
    #20;
    current_time();
  end
endmodule
```

Simulator Output

```
Current simulation time is 10
Current simulation time is 30
```

function call as an expression

```
module sv_function;
  int x;
  //function to add two integer numbers.
  function int sum;
    input int a,b;
  return a+b;
  endfunction
  initial begin
    x = 10 + sum(10,5);
    $display("\tValue of x = %0d",x);
  end
endmodule
```

Simulator Output

```
Value of x = 25
```

argument pass by value

In argument pass by value,

- the argument passing mechanism works by copying each argument into the subroutine area.
- if any changes to arguments within the subroutine, those changes will not be visible outside the subroutine

```
module argument_passing;
  int x,y,z;
  //function to add two integer numbers.
  function int sum(int x,y);
    x = x+y;
    return x+y;
  endfunction

  initial begin
    x = 20;
    y = 30;
    z = sum(x,y);
    $display("-----------------------------------------------------------");
    $display("\tValue of x = %0d",x);
    $display("\tValue of y = %0d",y);
    $display("\tValue of z = %0d",z);
    $display("-----------------------------------------------------------");
  end
endmodule
```

Simulator Output

```
-----------------------------------------------------------
        Value of x = 20
        Value of y = 30
        Value of z = 80
-----------------------------------------------------------
```

argument pass by reference

- In pass by reference, a reference to the original argument is passed to the subroutine.
- As the argument within a subroutine is pointing to an original argument, any changes to the argument within subroutine will be visible outside.
- To indicate argument pass by reference, the argument declaration is preceded by keyword **ref**.

```
module argument_passing;
  int x,y,z;

  //function to add two integer numbers.
  function int sum(ref int x,y);
    x = x+y;
    return x+y;
  endfunction

  initial begin
    x = 20;
    y = 30;
    z = sum(x,y);
    $display("-----------------------------------------------------------");
    $display("\tValue of x = %0d",x);
    $display("\tValue of y = %0d",y);
    $display("\tValue of z = %0d",z);
    $display("-----------------------------------------------------------");
  end
endmodule
```

Simulator Output

```
-----------------------------------------------------------
        Value of x = 50
        Value of y = 30
        Value of z = 80
-----------------------------------------------------------
```

```
module argument_passing;
  int q;

  //function to add three integer numbers.
  function int sum(int x=5,y=10,z=20);
    return x+y+z;
  endfunction

  initial begin
    q = sum( , ,10);
    $display("-----------------------------------------------------------------");
    $display("\tValue of z = %0d",q);
    $display("-----------------------------------------------------------------");
  end
endmodule
```

Value of z = 25

argument pass by name

In argument pass by name, arguments can be passed in any order by specifying the name of the subroutine argument.

```
module argument_passing;
  int x,y,z;

  function void display(int x,string y);
    $display("\tValue of x = %0d, y = %0s",x,y);
  endfunction

  initial begin
    display(.y("Hello World"),.x(2016));
  end
endmodule
```

```
Value of x = 2016, y = Hello World
```

SystemVerilog Class

- A class is a user-defined data type that includes data (class properties), functions and tasks that operate on data.
- functions and tasks are called as methods, both are members of the class.
- classes allow objects to be dynamically created, deleted, assigned and accessed via object handles.

Class Declaration

```
class sv_class;
  //class properties
  int x;

  //method-1
  task set(int i);
    x = i;
  endtask

  //method-2
  function int get();
    return x;
  endfunction
endclass
```

Class declaration/Class Instance

sv_class class_1;

Object Creation

```
class_1 = new();
```

```
sv_class class_1 = new();
```

Accessing class properties and methods

```
class_1.set(10); //calling set method to set value 10 to x
$display("Vlaue of x = %0d",class_1.get(););
```

```
class sv_class;
  //class properties
  int x;

  //method-1
  task set(int i);
    x = i;
  endtask

  //method-2
  function int get();
    return x;
  endfunction
endclass

module sv_class_ex;
 sv_class class_1; //Creating Handle

  initial begin
    sv_class class_2 = new(); //Creating handle and Object
    class_1 = new(); //Creating Object for the Handle

    //Accessing Class methods
    class_1.set(10);
    class_2.set(20);
    $display("\tclass_1 :: Value of x = %0d",class_1.get());
    $display("\tclass_2 :: Value of x = %0d",class_2.get());
  end
endmodule
```

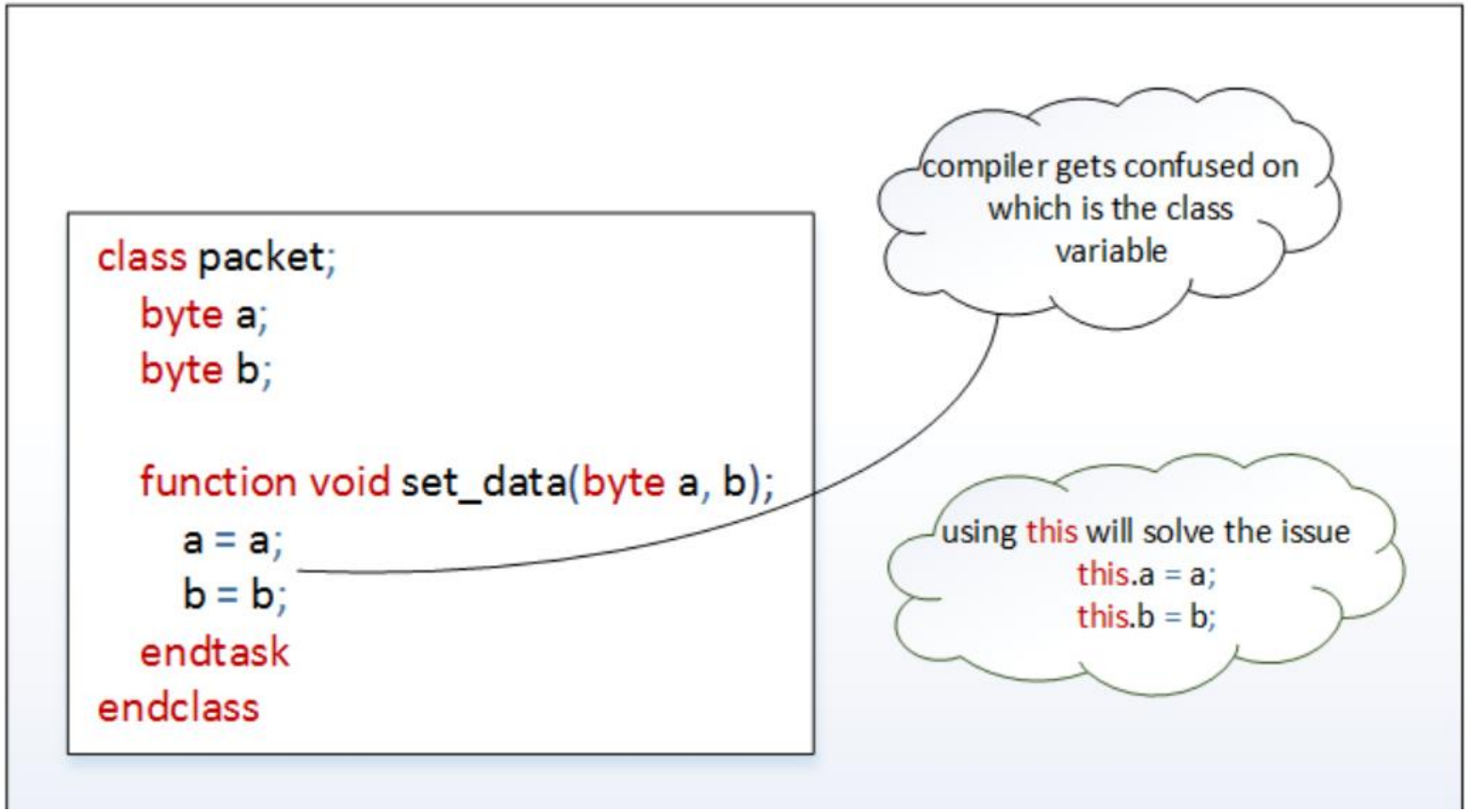
Simulator Output

class_1 :: Value of x = 10

class_1 :: Value of x = 20

SystemVerilog this keyword

- this keyword is used to refer to class properties.
- this keyword is used to unambiguously refer to class properties or methods of the current instance.
- this is a pre-defined class handle referring to the object from which it is used, calling this.variable means object.v



```
class packet;

  //class properties
  bit [31:0] addr;
  bit [31:0] data;
  bit  write;
  string pkt_type;

  //constructor
  function new(bit [31:0] addr,data,bit write,string pkt_type);
    addr   = addr;
    data   = data;
    write  = write;
    pkt_type  = pkt_type;
  endfunction

  //method to display class prperties
  function void display();
    $display("-----------------------------------------------------");
    $display("\t addr   = %0h",addr);
    $display("\t data   = %0h",data);
    $display("\t write  = %0h",write);
    $display("\t pkt_type   = %0s",pkt_type);
    $display("-----------------------------------------------------");
  endfunction

endclass
```

```
module sv_constructor;
  packet pkt;

  initial begin
    pkt = new(32'h10,32'hFF,1,"GOOD_PKT");
    pkt.display();
  end

endmodule
```

```
addr = 0
data = 0
write = 0
pkt_type =
```

```
class packet;

  //class properties
  bit [31:0] addr;
  bit [31:0] data;
  bit  write;
  string pkt_type;

  //constructor
  function new(bit [31:0] addr,data,bit write,string pkt_type);
    this.addr  = addr;
    this.data  = data;
    this.write = write;
    this.pkt_type = pkt_type;
  endfunction

  //method to display class prperties
  function void display();
    $display("-----------------------------------------------------");
    $display("\t addr  = %0h",addr);
    $display("\t data  = %0h",data);
    $display("\t write = %0h",write);
    $display("\t pkt_type  = %0s",pkt_type);
    $display("-----------------------------------------------------");
  endfunction

endclass
```

```
module sv_constructor;
  packet pkt;

  initial begin
    pkt = new(32'h10,32'hFF,1,"GOOD_PKT");
    pkt.display();
  end

endmodule
```

```
-----------------------------------------------------
         addr  = 10
         data  = ff
         write = 1
         pkt_type  = GOOD_PKT
-----------------------------------------------------
```

SystemVerilog Class Constructors

- The new function is called as class constructor.
- On calling the new method it allocates the memory and returns the address to the class handle.

```
//class declaration
class packet;
 //class properties
 bit [31:0] addr;
 bit [31:0] data;
endlass
```

```
//declare the handle
packet  pkt_1;

//construct the object
pkt_1 = new();
```



Calling **new()** allocates memory, returns the address and initialize the variables to default value.

Note: Default value is,

0 – for 2 state variable's

X – for 4 state variable's

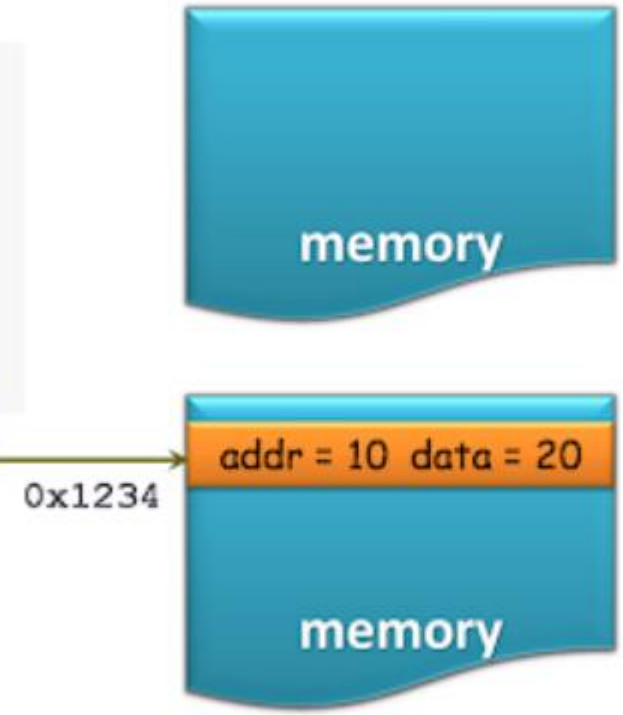
- The new operation is defined as a function with no return type
- every class has a built-in new method, calling the constructor of class without the explicit definition of the new method will invoke the default built-in new method
- specifying return type to the constructor shall give a compilation error (even specifying void shall give a compilation error)
- The constructor can be used for initializing the class properties. In case of any initialization required, those can be placed in the constructor and It is also possible to pass arguments to the constructor, which allows run-time customization of an object.

Variable's Initialization by Constructor:

```
//class declaration
class packet;
 //class properties
 bit [31:0] addr;
 bit [31:0] data;
//constructor
 function new(bit [31:0] a, b );
   addr = a;
   data = b;
 endfunction
endlass
```

```
//declare the handle
packet = pkt_1;

//construct the object
pkt_1 = new(10,20);
```



```
class packet;
  //class properties
  bit [31:0] addr;
  bit [31:0] data;
  bit write;
  string pkt_type;
  //constructor
  function new();
    addr   = 32'h10;
    data   = 32'hFF;
    write  = 1;
    pkt_type = "GOOD_PKT";
  endfunction

  //method to display class prperties
  function void display();
    $display("-----------------------------------------------------");
    $display("\t addr   = %0d",addr);
    $display("\t data   = %0h",data);
    $display("\t write  = %0d",write);
    $display("\t pkt_type   = %0s",pkt_type);
    $display("-----------------------------------------------------");
  endfunction
endclass
```

```
module sv_constructor;
  packet pkt;
  initial begin
    pkt = new();
    pkt.display();
  end
endmodule
```

addr = 16

data = ff

write = 1

pkt_type = GOOD_PKT

System Verilog Class Assignment

Object will be created only after doing new to an class handle,

```
packet   pkt_1;
pkt_1  = new();
packet   pkt_2;
pkt_2  = pkt_1;
```

- an object is created only for pkt_1, pkt_2 is just a handle to the packet
- pkt_1 is assigned to the pkt_2. so only one object has been created, pkt_1 and pkt_2 are two handles both are pointing to the same object
- As both the handles are pointing to the same object any changes made with respect to pkt_1 will **reflect** on pkt_2

Class Assignment :

```
//class declaration
class packet;
 //class properties
 bit [31:0] addr;
 bit [31:0] data;
endlass
```

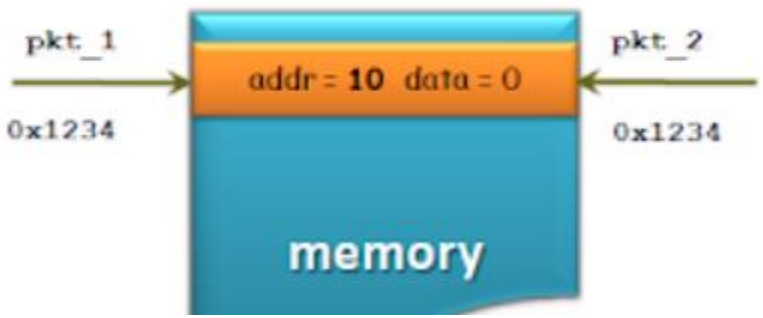
```
//declare the handle
packet pkt_1;
packet pkt_2;

//construct the object
pkt_1 = new();
pkt_2 = pkt_1;//assignment
```



With the assignment, both the objects will point the to same memory.

```
//changing the value of addr using pkt_2 handle
pkt_2 .addr = 10;
```



```
class packet;
  //class properties
  bit [31:0] addr;
  bit [31:0] data;
  bit write;
  string pkt_type;
  //constructor
  function new();
    addr   = 32'h10;
    data   = 32'hFF;
    write  = 1;
    pkt_type = "GOOD_PKT";
  endfunction

  //method to display class properties
  function void display();
    $display("-----------------------------
    $display("\t addr   = %0d",addr);
    $display("\t data   = %0h",data);
    $display("\t write  = %0d",write);
    $display("\t pkt_type   = %0s",pkt_type)
    $display("-----------------------------
  endfunction
endclass
```

```
module class_assignment;
  packet pkt_1;
  packet pkt_2;

  initial begin
    pkt_1 = new();
    $display("\t****  calling pkt_1 display  ****");
    pkt_1.display();
    //assigning pkt_1 to pkt_2
    pkt_2 = pkt_1;
    $display("\t****  calling pkt_2 display  ****");
    pkt_2.display();
    //changing values with pkt_2 handle
    pkt_2.addr = 32'hAB;
    pkt_2.pkt_type = "BAD_PKT";

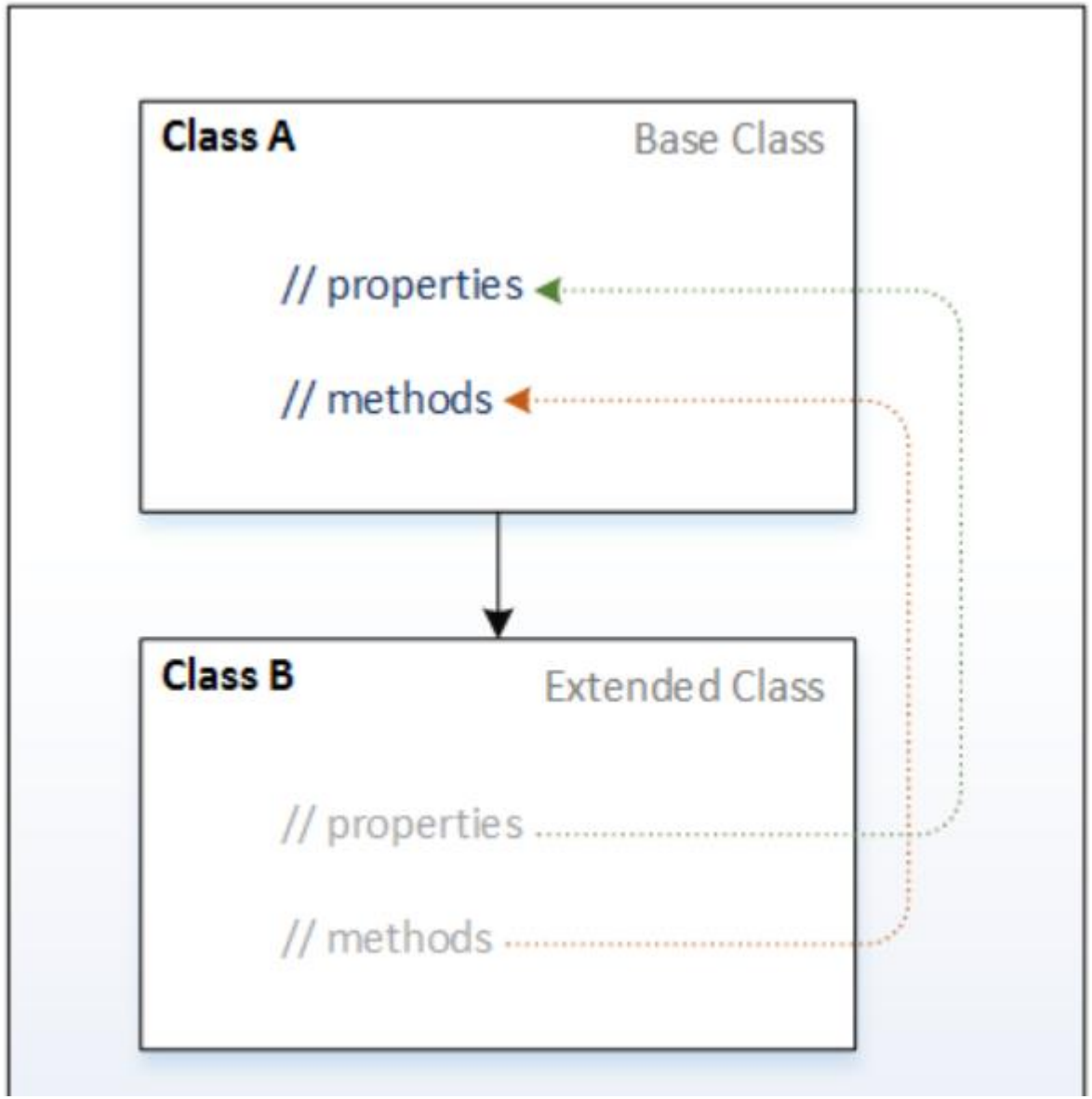
    //changes made with pkt_2 handle will reflect on pkt_1
    $display("\t****  calling pkt_1 display  ****");
    pkt_1.display();
  end
endmodule
```

```
****calling pkt_1 display****
-----------------------------------------------------
addr = 16
data = ff
write = 1
pkt_type = GOOD_PKT
-----------------------------------------------------
****calling pkt_2 display****
-----------------------------------------------------
addr = 16
data = ff
write = 1
pkt_type = GOOD_PKT
-----------------------------------------------------
****calling pkt_1 display****
-----------------------------------------------------
addr = 171
data = ff
write = 1
pkt_type = BAD_PKT
-----------------------------------------------------
```

SystemVerilog Inheritance

- Inheritance is an OOP concept that allows the user to create classes that are built upon existing classes.
- The new class will be with new properties and methods along with having access to all the properties and methods of the original class.
- Inheritance is about inheriting base class members to the extended class.

•New classes can be created based on existing classes, this is referred to as class inheritance
•A derived class by default inherits the properties and methods of its parent class
•An inherited class is called a subclass of its parent class
•A derived class may add new properties and methods, or modify the inherited properties and methods
•Inheritance allows re-usability. i.e. derived class by default includes the properties and methods, which is ready to use
•If the class is derived from a derived class, then it is referred to as Multilevel inheritance


Class A
Base Class
// properties
// methods
Class B
Extended Class
// properties
// methods

Inheritance Terminology

Parent Class

- It's an existing class;
- The class whose features are inherited
- The parent class is also known as a base class, superclass

Child Class

- It's an extended class;
- The class that inherits the other class is known as subclass
- The child class is also known as an extended class, derived class, subclass

```
class parent_class;
  bit [31:0] addr;
endclass

class child_class  extends parent_class;
  bit [31:0] data;
endclass

module inheritence;
  initial begin
    child_class c = new();
    c.addr = 10;
    c.data = 20;
    $display("Value of addr = %0d data = %0d",c.addr,c.data);
  end
endmodule
```

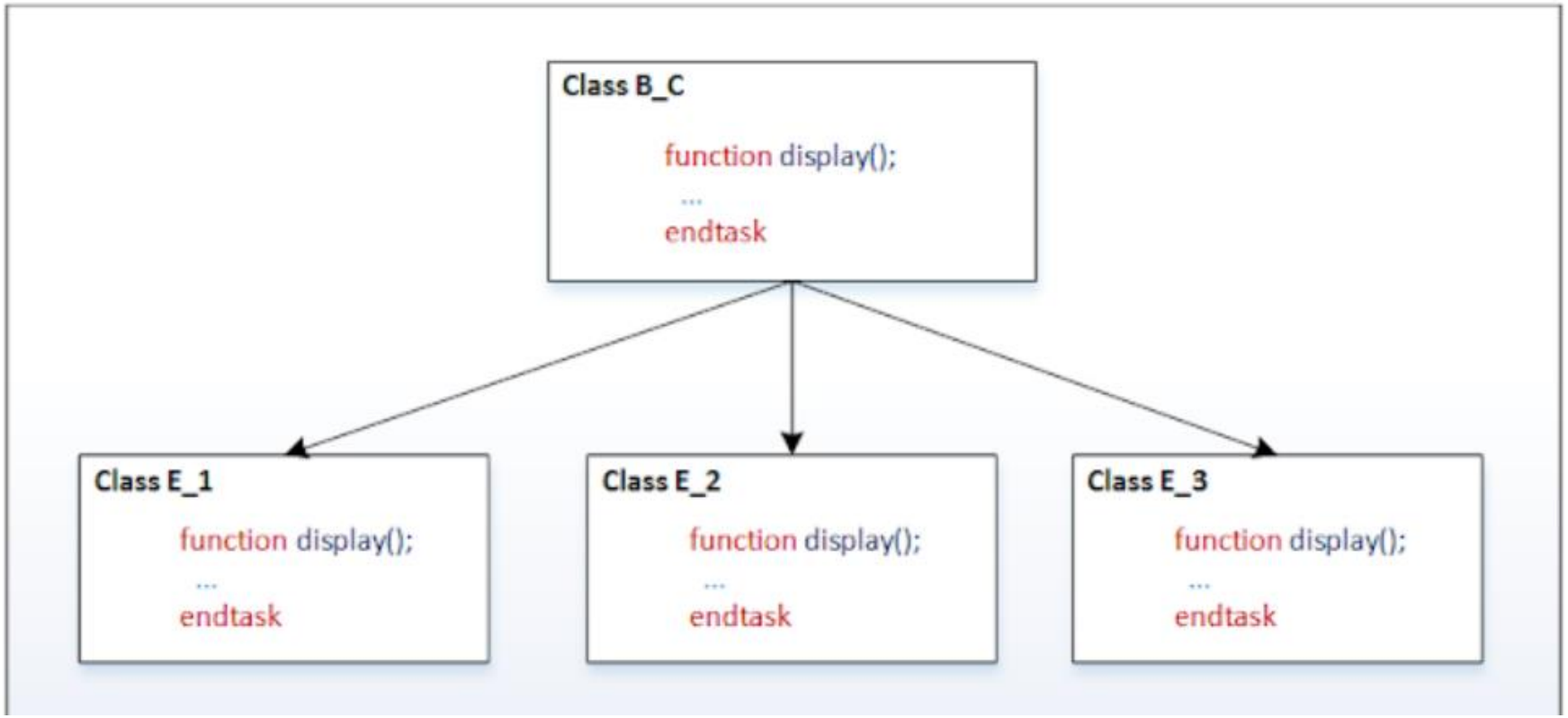
Value of addr = 10 data = 20

Though the addr is not declared in child_class, it is accessible. because it is inherited from the parent class.

Polymorphism in SystemVerilog

- Polymorphism means many forms.
- Polymorphism in SystemVerilog provides an ability to an object to take on many forms.
- Method handle of super-class can be made to refer to the subclass method, this allows polymorphism or different forms of the same method.

Polymorphism example



```
class base_class;
  virtual function void display();
    $display("Inside base class");
  endfunction
endclass
```

Let's write the base_class with a method display();

```
class ext_class_1 extends base_class;
  function void display();
    $display("Inside extended class 1");
  endfunction
endclass

class ext_class_2 extends base_class;
  function void display();
    $display("Inside extended class 2");
  endfunction
endclass

class ext_class_3 extends base_class;
  function void display();
    $display("Inside extended class 3");
  endfunction
endclass
```

Writing three extended classes of base_class, with display method overridden in it.

Create an object of each extended class,

```
ext_class_1 ec_1 = new();
ext_class_2 ec_2 = new();
ext_class_3 ec_3 = new();
```

Declare an array of a base class,

```
base_class b_c[3];
```

Assign extend class handles to base class handles.

```
b_c[0] = ec_1;
b_c[1] = ec_2;
b_c[2] = ec_3;
```

Call the display method using base_class handle,

```
b_c[0].display();
b_c[1].display();
b_c[2].display();
```

Though all the methods are called using base_class handle, different methods are getting called. this shows the many forms of the same method, this is called *polymorphism*.

```
// base class
class base_class;
  virtual function void display();
    $display("Inside base class");
  endfunction
endclass

// extended class 1
class ext_class_1 extends base_class;
  function void display();
    $display("Inside extended class 1");
  endfunction
endclass

// extended class 2
class ext_class_2 extends base_class;
  function void display();
    $display("Inside extended class 2");
  endfunction
endclass

// extended class 3
class ext_class_3 extends base_class;
  function void display();
    $display("Inside extended class 3");
  endfunction
endclass
```

```
// module
module class_polymorphism;

  initial begin

    //declare and create extended class
    ext_class_1 ec_1 = new();
    ext_class_2 ec_2 = new();
    ext_class_3 ec_3 = new();

    //base class handle
    base_class b_c[3];

    //assigning extended class to base class
    b_c[0] = ec_1;
    b_c[1] = ec_2;
    b_c[2] = ec_3;

    //accessing extended class methods using base class har
    b_c[0].display();
    b_c[1].display();
    b_c[2].display();
  end

endmodule
```

Simulator Output

```
Inside extended class 1
Inside extended class 2
Inside extended class 3
```

SystemVerilog Overriding class members

- Base class or parent class properties and methods can be overridden in the child class or extended class.
- Defining the class properties and methods with the same name as parent class in the child class will override the class members.

```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class extends parent_class;
  bit [31:0] data;
  function display();
    $display("Data = %0d",data);
  endfunction
endclass

module inheritence;
  initial begin
    child_class c=new();
    c.addr = 10;
    c.data = 20;
    c.display();
  end
endmodule
```

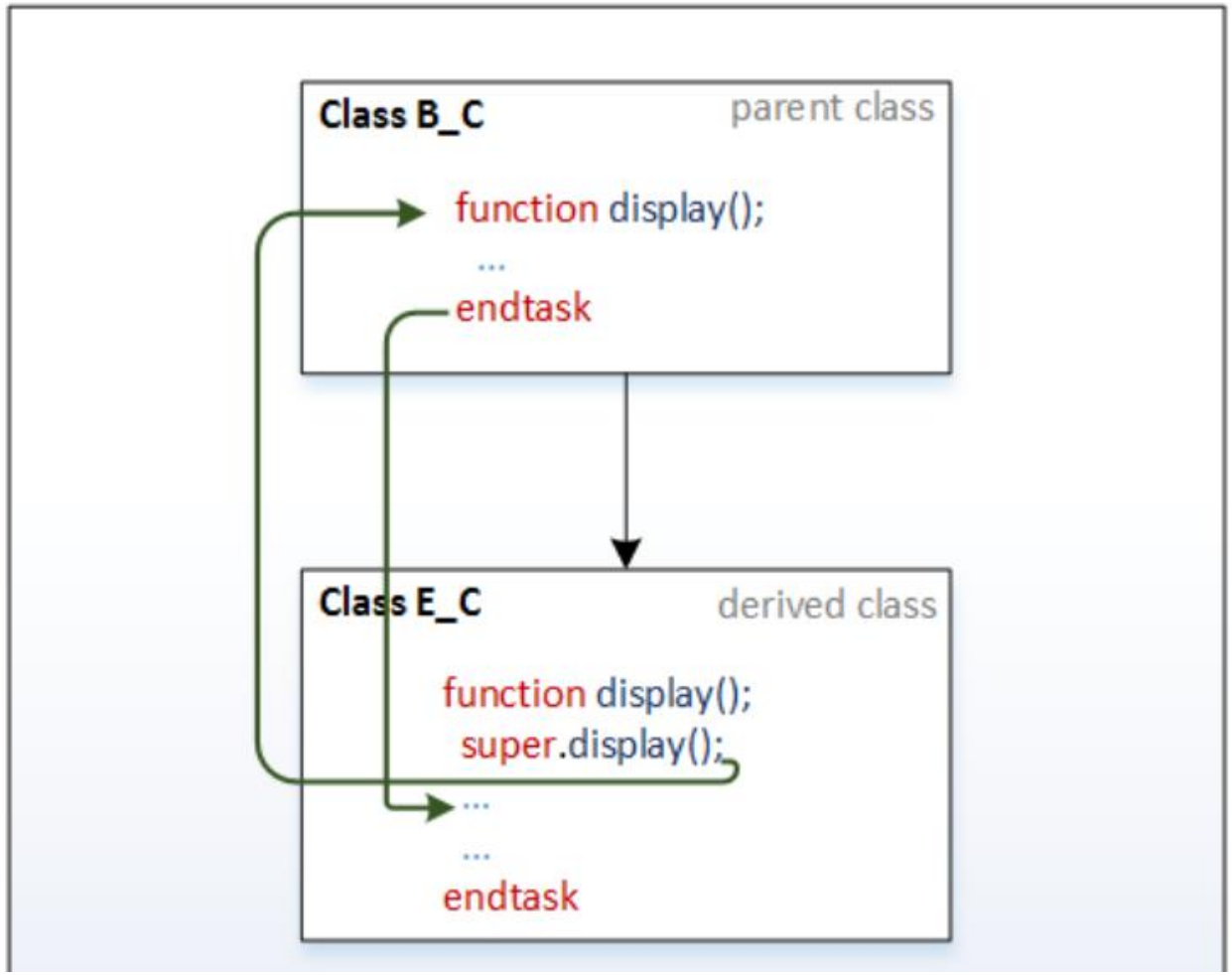
Simulator Output

Data = 20

SystemVerilog Super keyword

The ***super*** keyword is used in a derived class to refer to the members of the parent class.

- When class members are overridden in the derived class, It is necessary to use the *super* keyword to access members of a parent class
- With super keyword, it is allowed to access the class members of parent class which is only one level up



```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class  extends parent_class;
  bit [31:0] data;

  function display();
    super.display();
    $display("Data = %0d",data);
  endfunction

endclass

module inheritence;
  initial begin
    child_class c=new();
    c.addr = 10;
    c.data = 20;
    c.display();
  end
endmodule
```

Simulator Output

Addr = 10

Data = 20

```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class  extends parent_class;
  bit [31:0] data;

  function display();
    $display("Data = %0d",data);
  endfunction

endclass

module inheritence;
  initial begin
    child_class c=new();
    c.addr = 10;
    c.data = 20;
    c.display();
  end
endmodule
```

Simulator Output

Data = 20

SystemVerilog Casting

- SystemVerilog casting means the conversion of one data type to another datatype.
- During value or variable assignment to a variable, it is required to assign value or variable of the same data type.
- Some situations need assignment of different data type, in such situations, it is necessary to convert data type and assign. Otherwise, the assignment of different data type results in a compilation error.

- Static casting
- Dynamic casting

Static casting

- SystemVerilog static casting is not applicable to OOP
- Static casting converts one data type to another compatible data types (example string to int)
- As the name says 'Static', the conversion data type is fixed.
- Static casting will be checked during compilation, so there won't be run-time checking and error reporting
- Casting is applicable to value, variable or to an expression
- A data type can be changed by using a cast (') operation
- The vale/variable/expression to be cast must be enclosed in parentheses or within concatenation or replication braces

```
module casting;

  real r_a;
  int i_a;

  initial begin

    r_a = (2.1 * 3.2);

    //real to integer conversion
    i_a = int'(2.1 * 3.2); //or i_a = int'(r_a);

    $display("real value is %f",r_a);
    $display("int  value is %d",i_a);
  end
endmodule
```

Simulator Output

```
real value is 6.720000
int  value is         7
```

Dynamic casting

• Dynamic casting is used to, safely cast a super-class pointer (reference) into a subclass pointer (reference) in a class hierarchy
•Dynamic casting will be checked during run time, an attempt to cast an object to an incompatible object will result in a run-time error
•Dynamic casting is done using the $cast(destination, source) method
•With $cast compatibility of the assignment will not be checked during compile time, it will be checked during run-time

```
parent_class = child_class; //allowed

child_class  = parent_class; //not-allowed
```

```
parent_class = child_class ;
child_class  = parent_class; //allowed because parent_class is pointing to child_class.
```

Though parent_class is pointing to the child_class, we will get a *compilation error* saying its not compatible type for the assignment.

```
$cast(child_class,parent_class);
```

- In the above parent class assignment with child class example. type of parent class is changing dynamically i.e on declaration it is of parent class type, on child class assignment it is of child class type.
- Parent class handle during $cast execution is considered for the assignment, so it referred to as dynamic casting.

assigning child class handle to parent class handle

```
class parent_class;
  bit [31:0] addr;
  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class  extends parent_class;
  bit [31:0] data;
  function display();
    super.display();
    $display("Data = %0d",data);
  endfunction
endclass

module inheritence;
  initial begin
    parent_class p=new();
    child_class  c=new();
    c.addr = 10;
    c.data = 20;
    p = c;          //assigning child class handle to parent class handle
    c.display();
  end
endmodule
```

Simulator Output

Addr = 10

Data = 20

assigning parent class handle to child class handle

```
class parent_class;
  bit [31:0] addr;
  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class extends parent_class;
  bit [31:0] data;

  function display();
    super.display();
    $display("Data = %0d",data);
  endfunction
endclass

module inheritence;
  initial begin
    parent_class p=new();
    child_class  c=new();
    c.addr = 10;
    c.data = 20;
    c = p;          //assigning child class handle to parent class handle
    c.display();
  end
endmodule
```

Simulator Output

"c = p;"

Expression 'p' on rhs is not a class or a compatible class and hence cannot be assigned to a class handle on lhs.

Please make sure that the lhs and rhs expressions are compatible.

assigning parent class handle to child class handle

```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class extends parent_class;
  bit [31:0] data;
  function display();
    super.display();
    $display("Data = %0d",data);
  endfunction
endclass
module inheritence;
  initial begin
    parent_class p;
    child_class  c=new();
    child_class  c1;
    c.addr = 10;
    c.data = 20;
    p  = c;          //p is pointing to child class handle c.
    c1 = p;          //type check fails during compile time.
    c1.display();
  end
endmodule
```

Simulator Output

"c1 = p;"
Expression 'p' on rhs is not a class or a compatible class and hence cannot be assigned to a class handle on lhs.
Please make sure that the lhs and rhs expressions are compatible.

Use of $cast or casting

```
class parent_class;
  bit [31:0] addr;

  function display();
    $display("Addr = %0d",addr);
  endfunction
endclass

class child_class extends parent_class
  bit [31:0] data;

  function display();
    super.display();
    $display("Data = %0d",data);
  endfunction
endclass
```

```
module inheritence;
  initial begin
    parent_class p;
    child_class  c=new();
    child_class  c1;
    c.addr = 10;
    c.data = 20;

    p = c;          //p is pointing to child class handle c.
    $cast(c1,p);  //with the use of $cast, type chek will occu

    c1.display();
  end
endmodule
```

Simulator Output

Addr = 10

Data = 20

Systemverilog Data Hiding and Encapsulation

The technique of hiding the data within the class and making it available only through the methods, is known as encapsulation.

Access Control

- local
- protected

local class members

External access to the class members can be avoided by declaring members as ***local***.

Any violation could result in a compilation error.

Syntax:

```
local integer x;
```

Accessing local variable outside the class (Not allowed)

```
class parent_class;
  local bit [31:0] tmp_addr;

  function new(bit [31:0] r_addr);
    tmp_addr = r_addr + 10;
  endfunction

  function display();
    $display("tmp_addr = %0d",tmp_addr);
  endfunction
endclass


//    module
module encapsulation;
  initial begin
    parent_class p_c = new(5);

    p_c.tmp_addr = 20;  //Accessing local variable outside the class
    p_c.display();
  end
endmodule
```

Simulator Output

```
Error- Illegal class variable access
testbench.sv,
  Local member 'tmp_addr' of class 'parent_class' is not visible to scope
  'encapsulation'.
```

Accessing local variable within the class (Allowed)

```
class parent_class;
  local bit [31:0] tmp_addr;

  function new(bit [31:0] r_addr);
    tmp_addr = r_addr + 10;
  endfunction

  function display();
    $display("tmp_addr = %0d",tmp_addr);
  endfunction
endclass

//    module
module encapsulation;
  initial begin
    parent_class p_c = new(5);
    p_c.display();
  end
endmodule
```

Simulator Output

```
Addr = 15
```

Protected class members

- In some use cases, it is required to access the class members only by the derived class's,
- this can be done by prefixing the class members with the protected keyword.
- Any violation could result in a compilation error.

Syntax:

protected integer x;

Accessing a protected variable outside the class (Not allowed)

```
class parent_class;
  protected bit [31:0] tmp_addr;

  function new(bit [31:0] r_addr);
    tmp_addr = r_addr + 10;
  endfunction

  function display();
    $display("tmp_addr = %0d",tmp_addr);
  endfunction
endclass

class child_class  extends parent_class;
  function new(bit [31:0] r_addr);
    super.new(r_addr);
  endfunction

  function void incr_addr();
    tmp_addr++;
  endfunction
endclass
```

```
//    module
module encapsulation;
  initial begin
    parent_class p_c = new(5);
    child_class  c_c = new(10);

    // variable declared as protected cannot be accessed ou
    p_c.tmp_addr = 10;
    p_c.display();

    c_c.incr_addr(); //Accessing protected variable in exte
    c_c.display();
  end
endmodule
```

Simulator Output

```
Error- Illegal class variable access
testbench.sv,
  Protected member 'tmp_addr' of class 'parent_class' is not visible to scope
  'encapsulation'.
```

Accessing a protected variable in the extended class (allowed)

```
class parent_class;
  protected bit [31:0] tmp_addr;

  function new(bit [31:0] r_addr);
    tmp_addr = r_addr + 10;
  endfunction

  function display();
    $display("tmp_addr = %0d",tmp_addr);
  endfunction
endclass

class child_class  extends parent_class;
  function new(bit [31:0] r_addr);
    super.new(r_addr);
  endfunction

  function void incr_addr();
    tmp_addr++;
  endfunction
endclass
```

```
//    module
module encapsulation;
  initial begin
    child_class  c_c = new(10);

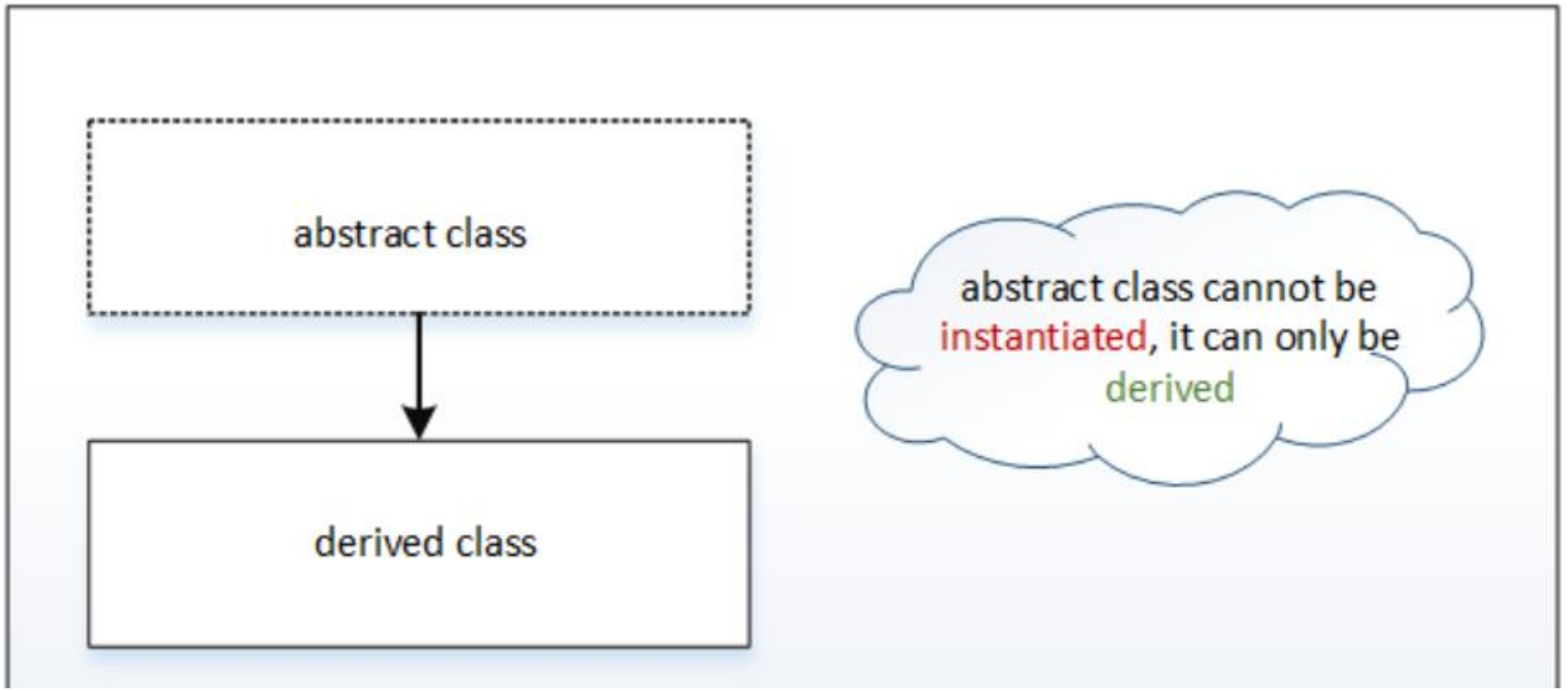
    c_c.incr_addr(); //Accessing protected variable in exten
    c_c.display();
  end
endmodule
```

Simulator Output

```
Addr = 21
```

Abstract Class

- SystemVerilog class declared with the keyword virtual is referred to as an *abstract* class.
- An abstract class sets out the prototype for the sub-classes.
- An abstract class cannot be instantiated, it can only be derived.
- An abstract class can contain methods for which there are only a prototype and no implementation, just a method declaration.



Abstract class Syntax

```
virtual class abc;
  //Class defination
endclass
```

Instantiating virtual class

```
//abstract class
virtual class packet;
  bit [31:0] addr;
endclass


module virtual_class;
  initial begin
    packet p;
    p = new();
  end
endmodule
```

An abstract class can only be derived, creating an object of a virtual class leads to a compilation error.

Simulator Output

```
virtual_class, "p = new();"
Instantiation of the object 'p' can not be done because its type 'packet' is
an abstract base class.
Perhaps there is a derived class that should be used.
```

Deriving virtual class

```
//abstract class
virtual class packet;
  bit [31:0] addr;
endclass

class extended_packet extends packet;
  function void display;
    $display("Value of addr is %0d", addr);
  endfunction
endclass

module virtual_class;
  initial begin
    extended_packet p;
    p = new();
    p.addr = 10;
    p.display();
  end
endmodule
```

An abstract class is derived and written extend the class and creating it.

Simulator Output

```
Value of addr is 10
```

Virtual Methods in SystemVerilog

SystemVerilog Methods declared with the keyword *virtual* are referred to as virtual methods.

Virtual Methods,

Virtual Functions
•Virtual Tasks

Virtual Functions

A function declared with a virtual keyword before the function keyword is referred to as virtual Function

Virtual Task

Task declared with a virtual keyword before the task keyword is referred to as virtual task

About Virtual Method

In a virtual method,
If the base_class handle is referring to the extended class, then the extended class method handle will get assigned to the base class handle.

Virtual function syntax

```
virtual function function_name;

  //Function definition
endfunction
```

Virtual task syntax

```
virtual task task_name;
  //task definition
endtask
```

```
class base_class;

  function void display;
    $display("Inside base_class");
  endfunction

endclass

class extended_class  extends base_class;

  function void display;
    $display("Inside extended class");
  endfunction

endclass


module virtual_class;
  initial begin
    base_class     b_c;
    extended_class e_c;

    e_c = new();
    b_c = e_c;

    b_c.display();
  end
endmodule
```

the method inside the base class is declared without a virtual keyword, on calling method of the base class which is pointing to the extended class will call the base class method.

Simulator Output

Inside base_class

```
class base_class;

  virtual function void display;
    $display("Inside base_class");
  endfunction

endclass

class extended_class  extends base_class;

  function void display;
    $display("Inside extended_class");
  endfunction

endclass


module virtual_class;
  initial begin
    base_class     b_c;
    extended_class e_c;

    e_c = new();
    b_c = e_c;

    b_c.display();
  end
endmodule
```

the method inside the base class is declared with a virtual keyword, on calling method of the base class which is pointing to an extended class will call the extended class method.

Simulator Output

```
Inside extended_class
```

Scope Resolution Operator ::

- The class scope operator **::** is used to specify an identifier defined within the scope of a class.
- Classes and other scopes can have the same identifiers
- The scope resolution operator uniquely identifies a member of a particular class

Class Resolution operator allows access to static members (class properties and methods) from outside the class, as well as access to public or protected elements of super classes from within the derived classes

```
//class
class packet;
          bit [31:0] addr;
  static bit [31:0] id;

  function display(bit [31:0] a,b);
    $display("Values are %0d %0d",a,b);
  endfunction
endclass

module sro_class;
  int id=10;
  initial begin
    packet p;
    p = new();
    packet::id = 20;
    p.display(packet::id,id);
  end
endmodule
```

A static member of the class is accessed outside the class by using class resolution operator ::

Simulator Output

Values are 20 10

SystemVerilog typedef class

A typedef is used to provide a forward declaration of the class.

In some cases, the class needs to be instantiated before the class declaration. In these kinds of situations, the typedef is used to provide a forward declaration of the class.

typedef syntax

```
typedef class class_name;
```

```
//class-1
class c1;
  c2 c;    //using class c2 handle before declaring it.
endclass

//class-2
class c2;
  c1 c;
endclass

module typedef_class;
  initial begin
    c1 class1;
    c2 class2;
    $display("Inside typedef_class");
  end
endmodule
```

Simulator Output

```
Error-[SE] Syntax error
  Following verilog source has syntax error :
   token 'c2' should be a valid type. Please declare it vi
  is an Interface.
  "testbench.sv", 6: token is ';'
    c2 c;
```

```
typedef class c2;
//class-1
class c1;
  c2 c;    //using class c2 handle before declaring it.
endclass

//class-2
class c2;
  c1 c;
endclass

module typedef_class;
  initial begin
    c1 class1;
    c2 class2;
    $display("Inside typedef_class");
  end
endmodule
```

Simulator Output

Inside typedef_class

SystemVerilog Randomization and SystemVerilog Constraint

- Randomization
- Disable Randomization
- Randomization methods

- **Constraints**
  - Constraint Block, External Constraint Blocks and Constraint Inheritance
  - Inside Operator
  - Weighted Distribution
  - Implication Operator and if-else
  - Iterative in Constraint Blocks **(foreach constraints)**
  - Disable Constraint
  - Static Constraints
  - In line Constraints
  - Functions in Constraints
  - Soft Constraints
  - Unique Constraints
  - Bidirectional Constraints
  - Solve-Before

Random System Methods **$urandom(), $random and $urandom_range();**

randomization in SystemVerilog

- Randomization is the process of making something random;
- SystemVerilog randomization is the process of generating random values to a variable.
- Verilog has a $random method for generating the random integer values.
- This is good for randomizing the variables alone, but it is hard to use in case of class object randomization.
- for easy randomization of class properties, SystemVerilog provides rand keyword and randomize() method.

•Following types can be declared as rand and randc,
singular variables of any integral type
•arrays
•arrays size
•object handle's

rand keyword

- Variables declared with the rand keyword are standard random variables.
- Their values are uniformly distributed over their range.

```
rand bit [3:0] addr;
```

addr is a 4-bit unsigned integer with a range of 0 to 15. on randomization this variable shall be assigned any value in the range 0 to 15 with equal probability.

randc keyword

- randc is random-cyclic.
- For the variables declared with the randc keyword, on randomization variable values don't repeat a random value until every possible value has been assigned.

```
randc bit wr_rd;
```

In order to randomize the object variables, the user needs to call randomize() method.

example:

```
object.randomize();
```

```
//class
class packet;
  rand bit [2:0] addr1;
  randc bit [2:0] addr2;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();
    repeat(10) begin
      pkt.randomize();
      $display("\taddr1 = %0d \t addr2 = %0d",pkt.addr1,pkt.addr2);
    end
  end
endmodule
```

Simulator Output

addr1 = 6 addr2 = 4
addr1 = 4 addr2 = 3
addr1 = 2 addr2 = 0
addr1 = 6 addr2 = 6
addr1 = 1 addr2 = 7
addr1 = 3 addr2 = 5
addr1 = 7 addr2 = 1
addr1 = 6 addr2 = 2
addr1 = 4 addr2 = 7
addr1 = 4 addr2 = 6

Disable randomization

The rand_mode() method is used to disable the randomization of a variable declared with the rand/randc keyword.

- rand_mode(1) means randomization enabled
- rand_mode(0) means randomization disabled
- The default value of rand_mode is 1, i.e enabled
- Once the randomization is disabled, it is required to make rand_mode(1) enable back the randomization
- rand_mode can be called as SystemVerilog method, the randomization enables/disable status of a variable can be obtained by calling vairble.rand_mode().
- the rand_mode method returns 1 if randomization is enabled else returns 0

rand_mode syntax

```
<object_hanlde>.<variable_name>.rand_mode(enable);
//enable = 1, randomization enable
//enable = 0, randomization disable
```

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data = %0d",pkt.addr,pkt.data);
  end
endmodule
```

Simulator Output

addr = 110 data = 116

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    //disable rand_mode of addr variable of pkt
    pkt.addr.rand_mode(0);

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data = %0d",pkt.addr,pkt.data);

    $display("\taddr.rand_mode() = %0d \t data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());
  end
endmodule
```

Simulator Output

addr = 0 data = 110

addr.rand_mode() = 0 data.rand_mode() = 1

randomization disable for all class variable

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    $display("\taddr.rand_mode() = %0d \t data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());

    //disable rand_mode of object
    pkt.rand_mode(0);

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data = %0d",pkt.addr,pkt.data);

    $display("\taddr.rand_mode() = %0d \t data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());
  end
endmodule
```

Simulator Output

addr.rand_mode() = 1 data.rand_mode() = 1

addr = 0 data = 0

addr.rand_mode() = 0 data.rand_mode() = 0

Constrained randomization

- In some situations it is required to control the values getting assigned on randomization, this can be achieved by writing constraints
- By writing constraints to a random variable, the user can get specific value on randomization.
- constraints to a random variable shall be written in constraint blocks.

Constraint blocks

- Constraint blocks are class members like tasks, functions, and variables
- Constraint blocks will have a unique name within a class
- Constraint blocks consist of conditions or expressions to limit or control the values for a random variable
- Constraint blocks are enclosed within curly braces { }
- Constraint blocks can be defined inside the class or outside the class like extern methods, constraint block defined outside the class is called as extern constraint block

Constraint block syntax

```
constraint <constraint_block_name> { <condition/expression>;
                                     ...
                                     <condition/expression>; }
```

Constraint block inside the class

```
class packet;
  rand bit [3:0] addr;

  constraint addr_range { addr > 5; }
endclass

module constr_blocks;
  initial begin
    packet pkt;
    pkt = new();
    repeat(10) begin
      pkt.randomize();
      $display("\taddr = %0d",pkt.addr);
    end
  end
endmodule
```

Simulator Output

```
addr = 14
addr = 10
addr = 9
addr = 8
addr = 9
addr = 6
addr = 10
addr = 14
addr = 12
addr = 8
```

Constraint block outside the class

```
class packet;
  rand bit [3:0] addr;
//constraint block declaration
  constraint addr_range;
endclass

//constraint implementation outside class body
constraint packet::addr_range { addr > 5; }

module extern_constr;
  initial begin
    packet pkt;
    pkt = new();

    repeat(10) begin
      pkt.randomize();
      $display("\taddr = %0d",pkt.addr);
    end
  end
endmodule
```

Simulator Output

```
addr = 14
addr = 10
addr = 9
addr = 8
addr = 9
addr = 6
addr = 10
addr = 14
addr = 12
addr = 8
```

Constraint Inheritance

- Like class members, constraints also will get inherited from parent class to child class.
- Constraint blocks can be overridden by writing constraint block with the same name as in parent class.

```
class packet;
  rand bit [3:0] addr;
  constraint addr_range { addr > 5; }
endclass

class packet2 extends packet;
  constraint addr_range { addr < 5; } //overriding constraint of parent class
endclass

module const_inhe;
  initial begin
    packet pkt1;
    packet2 pkt2;

    pkt1 = new();
    pkt2 = new();

    $display("------------------------------------");
    repeat(5) begin
      pkt1.randomize();
      $display("\tpkt1:: addr = %0d",pkt1.addr);
    end

    $display("------------------------------------");
    repeat(5) begin
      pkt2.randomize();
      $display("\tpkt2:: addr = %0d",pkt2.addr);
    end
    $display("------------------------------------");
  end
endmodule
```

Simulator Output

```
--------------------------------
pkt1:: addr = 14
pkt1:: addr = 10
pkt1:: addr = 9
pkt1:: addr = 8
pkt1:: addr = 9
--------------------------------
pkt2:: addr = 0
pkt2:: addr = 1
pkt2:: addr = 2
pkt2:: addr = 0
pkt2:: addr = 2
--------------------------------
```

Constraint inside SystemVerilog

- During randomization, it might require to randomize the variable within a range of values or with inset of values or other than a range of values.
- this can be achieved by using constraint inside operator.

SystemVerilog inside operator, random variables will get values specified within the inside block.

- values within the inside block can be variable, constant or range
- the inside block is written with an inside keyword followed by curly braces {}

```
constraint addr_range { addr inside { ... }; }
```

- the range is specified by [ ]

```
constraint addr_range { addr inside { [5:10]}; }
```

- set of values are specified by 'comma',

```
constraint addr_range { addr inside { 1,3,5,7,9}; }
```

- it is allowed to mix range and set of values

```
constraint addr_range { addr inside {1,3,[5:10],12,[13:15]}; }
```

- if the value needs to be outside the range, then it can be specified as inverse (!) of inside

```
constraint addr_range { addr !(inside {[5:10]}); }
```

- Other random variables can be used in inside block

```
rand bit [3:0] start_addr;
rand bit [3:0] end_addr;
rand bit [3:0] addr;
constraint addr_range { addr inside {[start_addr:end_addr]}; }
```

constraint inside example

```
class packet;
  rand bit [3:0] addr;
  rand bit [3:0] start_addr;
  rand bit [3:0] end_addr;

  constraint addr_1_range { addr inside {[start_addr:end_addr]}; }
endclass

module constr_inside;
  initial begin
    packet pkt;
    pkt = new();
    $display("------------------------------------");
    repeat(3) begin
      pkt.randomize();
      $display("\tstart_addr = %0d,end_addr = %0d",pkt.start_addr,pkt.end_addr);
      $display("\taddr = %0d",pkt.addr);
      $display("------------------------------------");
    end
  end
endmodule
```

Simulator Output

```
------------------------------
start_addr = 12,end_addr = 13
addr = 12
------------------------------
start_addr = 3,end_addr = 7
addr = 7
------------------------------
start_addr = 5,end_addr = 11
addr = 9
------------------------------
```

inverted inside example

```
class packet;
  rand bit [3:0] addr;
  rand bit [3:0] start_addr;
  rand bit [3:0] end_addr;

  constraint addr_1_range { !(addr inside {[start_addr:end_addr]}); }
endclass

module constr_inside;
  initial begin
    packet pkt;
    pkt = new();
    $display("------------------------------------");
    repeat(3) begin
      pkt.randomize();
      $display("\tstart_addr = %0d,end_addr = %0d",pkt.start_addr,pkt.end_addr);
      $display("\taddr = %0d",pkt.addr);
      $display("------------------------------------");
    end
  end
endmodule
```

Simulator Output

```
--------------------------------
start_addr = 12,end_addr = 4
addr = 0
--------------------------------
start_addr = 6,end_addr = 7
addr = 13
--------------------------------
start_addr = 5,end_addr = 9
addr = 11
--------------------------------
```

dist Constraint in SystemVerilog

- Constraint provides control on randomization, from which the user can control the values on randomization.
- it would be good if it's possible to control the occurrence or repetition of the same value on randomization.
- its possible, with dist operator, some values can be allocated more often to a random variable.
- this is called a **weighted distribution**.
- dist is an operator, it takes a list of values and weights, separated by := or :/ operator.

weighted distribution

As the name says, in weighted distribution weight will be specified to the values inside the constraint block.
Value with the more weight will get allocated more often to a random variable.

syntax

```
value := weight   or
value :/ weight
```

Value - desired value to a random variable
weight - indicates how often the value needs to be considered on randomization

- The values and weights can be constants or variables,
- value can be single or a range
- the default weight of an unspecified value is := 1
- the sum of weights need not be a 100

The := operator assigns the specified weight to the item, or if the item is a range, specified weight to every value in the range.

```
addr dist { 2 := 5, [10:12] := 8 };

for addr == 2 , weight 5
    addr == 10, weight 8
    addr == 11, weight 8
    addr == 12, weight 8
```

The :/ operator assigns the specified weight to the item, or if the item is a range, specified weight/n to every value in the range. where n is the number of values in the range.

```
addr dist { 2 :/ 5, [10:12] :/ 8 };

for addr == 2 , weight 5
    addr == 10, weight 8/3
    addr == 11, weight 8/3
    addr == 12, weight 8/3
```

weighted distribution constraint examples

randomization with dist operator

```
class packet;
  rand bit [3:0] addr;

  constraint addr_range { addr dist { 2 := 5, 7 := 8, 10 := 12 }; }
endclass

module constr_dist;
  initial begin
    packet pkt;
    pkt = new();
    $display("-------------------------------------");
    repeat(10) begin
      pkt.randomize();
      $display("\taddr = %0d",pkt.addr);
    end
    $display("-------------------------------------");
  end
endmodule
```

Simulator Output

```
---------------------------------
addr = 10
addr = 7
addr = 10
addr = 10
addr = 10
addr = 7
addr = 7
addr = 2
addr = 10
addr = 10
---------------------------------
```

difference between := and :/ dist operator

```
class packet;
  rand bit [3:0] addr_1;
  rand bit [3:0] addr_2;

  constraint addr_1_range {  addr_1 dist { 2 := 5, [10:12] := 8 }; }
  constraint addr_2_range {  addr_2 dist { 2 :/ 5, [10:12] :/ 8 }; }
endclass

module constr_dist;
  initial begin
    packet pkt;
    pkt = new();

    $display("------------------------------------");
    repeat(10) begin
      pkt.randomize();
      $display("\taddr_1 = %0d",pkt.addr_1);
    end
    $display("------------------------------------");
    $display("------------------------------------");
    repeat(10) begin
      pkt.randomize();
      $display("\taddr_2 = %0d",pkt.addr_2);
    end
    $display("------------------------------------");
  end
endmodule
```

Simulator Output

```
----------------------------------
 addr_1 = 2
 addr_1 = 12
 addr_1 = 12
 addr_1 = 12
 addr_1 = 11
 addr_1 = 10
 addr_1 = 11
 addr_1 = 2
 addr_1 = 11
 addr_1 = 12
----------------------------------

----------------------------------
 addr_2 = 10
 addr_2 = 12
 addr_2 = 2
 addr_2 = 2
 addr_2 = 10
 addr_2 = 10
 addr_2 = 2
 addr_2 = 11
 addr_2 = 11
 addr_2 = 12
----------------------------------
```

SystemVerilog implication if else Constraints

- The implication operator can be used to declaring conditional relations between two variables.
- implication operator is denoted by the symbol ->.
- The implication operator is placed between the expression and constraint.

```
expression -> constraint
```

If the expression on the LHS of implication operator (->) is true, then the only constraint on the RHS will be considered.

```
class packet;
  rand bit [3:0] addr;
       string   addr_range;
  constraint address_range { (addr_range == "small") -> (addr < 8);}
endclass

module constr_implication;
  initial begin
    packet pkt;
    pkt = new();

    pkt.addr_range = "small";
    $display("-------------------------------------");
    repeat(4) begin
      pkt.randomize();
      $display("\taddr_range = %s addr = %0d",pkt.addr_range,pkt.addr);
    end
    $display("-------------------------------------");
  end
endmodule
```

Simulator Output

```
---------------------------------
 addr_range = small addr = 6
 addr_range = small addr = 2
 addr_range = small addr = 1
 addr_range = small addr = 4
---------------------------------
```

if else constraints

- if else block allows conditional executions of constraints.
- If the expression is true, all the constraints in the first constraint/constraint-block must be satisfied, otherwise all the constraints in the optional else constraint/constraint-block must be satisfied.

```
class packet;
  rand bit [3:0] addr;
       string  addr_range;

  constraint address_range { if(addr_range == "small")
                               addr < 8;
                             else
                               addr > 8;
                           }
endclass

module constr_if_else;
  initial begin
    packet pkt;
    pkt = new();
    pkt.addr_range = "small";
    $display("------------------------------------");
    repeat(3) begin
      pkt.randomize();
      $display("\taddr_range = %s addr = %0d",pkt.addr_range,pkt.addr);
    end
    $display("------------------------------------");

    pkt.addr_range = "high";
    $display("------------------------------------");
    repeat(3) begin
      pkt.randomize();
      $display("\taddr_range = %s addr = %0d",pkt.addr_range,pkt.addr);
    end
    $display("------------------------------------");
  end
endmodule
```

Simulator Output

```
------------------------------------
 addr_range = small addr = 1
 addr_range = small addr = 4
 addr_range = small addr = 6
------------------------------------

------------------------------------
 addr_range = high addr = 12
 addr_range = high addr = 15
 addr_range = high addr = 9
------------------------------------
```

SystemVerilog foreach loop Constraint Blocks

- SystemVerilog supports using the foreach loop inside a constraint block.
- using the foreach loop within the constraint block will make easy to constrain an array.
- The foreach loop iterates over the elements of an array, so constraints with the foreach loop are called Iterative constraints.

```
constraint constraint_name { foreach ( variable[iterator] ) variable[iterator]
<..conditions..>  }
```

```
class packet;
  rand byte addr [];
  rand byte data [];

  constraint avalues { foreach( addr[i] ) addr[i] inside {4,8,12,16}; }
  constraint dvalues { foreach( data[j] ) data[j] > 4 * j; }
  constraint asize   { addr.size < 4; }
  constraint dsize   { data.size == addr.size; }
endclass

module constr_iteration;
  initial begin
    packet pkt;
    pkt = new();

    $display("------------------------------------");
    repeat(2) begin
      pkt.randomize();
      $display("\taddr-size = %0d data-size = %0d",pkt.addr.size(),pkt.data.size());
      foreach(pkt.addr[i]) $display("\taddr = %0d data = %0d",pkt.addr[i],pkt.data[i]);
      $display("------------------------------------");
    end
  end
endmodule
```

Simulator Output

```
------------------------------------
addr-size = 2 data-size = 2
addr = 4 data = 101
addr = 16 data = 21
------------------------------------
addr-size = 3 data-size = 3
addr = 16 data = 89
addr = 4 data = 19
addr = 16 data = 13
------------------------------------
```

Constraint modes and Static Constraints

Constraint Modes

The *constraint_mode()* method can be used to disable any particular constraint block.
By default constraint_mode value for all the constraint blocks will be 1.

constraint_mode() can be used as follow,

```
addr_range.constraint_mode(0); //disable
```

```
addr_range constraint
    packet.addr_range.constraint_mode(0);
```

Static Constraints

A constraint block can be defined as *static*, by including static keyword in its definition.
static constraint addr_range { addr > 5; }

Any mode change of static constraint will afect in all the objects of same class type.

```
class packet;
  rand bit [3:0] addr;

  static constraint addr_range { addr > 5; }
endclass

module static_constr;
  initial begin
    packet pkt1;
    packet pkt2;

    pkt1 = new();
    pkt2 = new();

    pkt1.randomize();
    $display("\taddr = %0d",pkt1.addr);
    pkt1.addr_range.constraint_mode(0);
    $display("\tAfter disabling constraint");
    pkt1.randomize();
    $display("\taddr = %0d",pkt1.addr);
    pkt2.randomize();
    $display("\taddr = %0d",pkt2.addr);
  end
endmodule
```

Simulator Output

addr = 14

After disabling constraint

addr = 2

addr = 6

SystemVerilog Inline Constraints

Inline constraint Syntax

```
object.randomize() with { .... };
```

```
class packet;
  rand bit [3:0] addr;
endclass

module inline_constr;
  initial begin
    packet pkt;
    pkt = new();

    repeat(2) begin
      pkt.randomize() with { addr == 8;};
      $display("\taddr = %0d",pkt.addr);
    end
  end
endmodule
```

Class doesn't have constraints defined in it. the inline constraint is used to constrain the variable addr.

Simulator Output

```
addr = 8
addr = 8
```

Constraint inside the class and inline constraint

```
class packet;
  rand bit [3:0] addr;
  rand bit [3:0] data;

  constraint data_range { data > 0;
                          data < 10; }
endclass

module inline_constr;
  initial begin
    packet pkt;
    pkt = new();
    repeat(2) begin
      pkt.randomize() with { addr == 8;};
      $display("\taddr = %0d data = %0d",pkt.addr,pkt.data);
    end
  end
endmodule
```

Simulator Output

addr = 8 data = 2

addr = 8 data = 5

Functions in Constraints

- In some cases constraint can't be expressed in a single line,
- in such cases function call can be used to constrain a random variable.
- Calling the function inside the constraint is referred to as function in constraints.

•The function will be written outside the constraint block
•Constraint logic shall be written inside the function as function definition and function call shall be placed inside the constraint block
•Functions shall be called before constraints are solved, and their return values shall be treated as state variables.

```
constraint constraint_name { var = function_call(); };
```

```
class packet;
  rand bit [3:0] start_addr;
  rand bit [3:0] end_addr;

  constraint start_addr_c { start_addr == s_addr(end_addr); }

  function bit [3:0] s_addr(bit [3:0] e_addr);
    if(e_addr < 4)
      s_addr = 0;
    else
      s_addr = e_addr - 4;
  endfunction

endclass

module func_constr;
  initial begin
    packet pkt;
    pkt = new();
    repeat(3) begin
      pkt.randomize();
      $display("\tstart_addr = %0d end_addr =",pkt.start_addr,pkt.end_addr);
    end
  end
endmodule
```

Simulator Output

start_addr = 9 end_addr =13

start_addr = 2 end_addr = 6

start_addr = 0 end_addr = 1

SystemVerilog Random System Methods

$urandom( )

The system function $urandom provides a mechanism for generating pseudorandom numbers.
The function returns a new 32-bit random number each time it is called.
The number shall be unsigned.

```
variable = $urandom(seed);
```

The seed is an optional argument that determines the sequence of random numbers generated.
The seed can be an integral expression.

$random( )

$random() is same as $urandom() but it generates signed numbers.

$urandom_range( )

The $urandom_range() function returns an unsigned integer within a specified range.

```
module system_funcations;
  bit [31:0] addr1;
  bit [31:0] addr2;
  bit [64:0] addr3;
  bit [31:0] data;

  initial begin
    addr1 = $urandom();
    addr2 = $urandom(89);
    addr3 = {$urandom(),$urandom()};
    data  = $urandom * 6;

    $display("addr1=%0d, addr2=%0d, addr3=%0d, data=%0d",addr1,addr2,addr3,data);

    addr1 = $urandom_range(30,20);
    addr2 = $urandom_range(20); //takes max value as '0'
    addr3 = $urandom_range(20,30); //considers max value as '30' and min value as '20'
    $display("addr1=%0d, addr2=%0d, addr3=%0d",addr1,addr2,addr3);
  end
endmodule
```

Simulator Output

addr1=303379748, addr2=2153631232, addr3=42395982244962108, data=546103870

addr1=27, addr2=6, addr3=25

Synchronization Communication Mechanisms

- Semaphore
- Mailbox
- Events

Semaphore

- Semaphore is a SystemVerilog built-in class, used for access control to shared resources, and for basic synchronization.
- A semaphore is like a bucket with the number of keys.
- processes using semaphores must first procure a key from the bucket before they can continue to execute
- All other processes must wait until a sufficient number of keys are returned to the bucket.

Imagine a situation where two processes try to access a shared memory area.
where one process tries to write and the other process is trying to read the same memory location.
this leads to an unexpected result. A semaphore can be used to overcome this situation.

Semaphore syntax

```
semaphore semaphore_name;
```

Semaphore methods

Semaphore is a built-in class that provides the following methods,

- new(); Create a semaphore with a specified number of keys
- get(); Obtain one or more keys from the bucket
- put(); Return one or more keys into the bucket
- try_get(); Try to obtain one or more keys without blocking

new( );

The new() method is used to create the Semaphore.

```
semaphore_name = new(numbers_of_keys);
```

- the new method will create the semaphore with number_of_keys keys in a bucket; where number_of_keys is integer variable.
- the default number of keys is '0'
- the new() method will return the semaphore handle or null if the semaphore cannot be created

put( );

The semaphore put() method is used to return key/keys to a semaphore.

```
semaphore_name.put(number_of_keys); or semaphore_name.put();
```

When the semaphore_name.put() method is called, the specified number of keys are returned to the semaphore. The default number of keys returned is 1.

get( );

The semaphore get() method is used to get key/keys from a semaphore.

```
semaphore_name.get(number_of_keys); or semaphore_name.get();
```

When the semaphore_name.get() method is called,

- If the specified number of keys are available, then the method returns and execution continues
- If the specified number of keys are not available, then the process blocks until the keys become available
- The default number of keys requested is 1

try_get();

The semaphore try_get() method is used to procure a specified number of keys from a semaphore, but without blocking.

```
semaphore_name.try_get(number_of_keys); or semaphore_name.try_get();
```

When the semaphore_name.try_get() method is called,

- If the specified number of keys are available, the method returns 1 and execution continues
- If the specified number of keys are not available, the method returns 0 and execution continues
- The default number of keys requested is 1

```
program main ;
semaphore sema = new(1);
initial begin
repeat(3) begin
fork
////////// PROCESS 1 ////////////////
begin
$display("1: Waiting for key");
sema.get(1);
$display("1: Got the Key");
#(10);// Do some work
sema.put(1);
$display("1: Returning back key ");
#(10);
end
////////// PROCESS 2 ////////////////
begin
$display("2: Waiting for Key");
sema.get(1);
$display("2: Got the Key");
#(10);//Do some work
sema.put(1);
$display("2: Returning back key ");
#(10);
end
join
end
#1000;
end
endprogram
```

RESULTS:

```
1: Waiting for key
1: Got the Key
2: Waiting for Key
1: Returning back key
2: Got the Key
2: Returning back key
1: Waiting for key
1: Got the Key
2: Waiting for Key
1: Returning back key
2: Got the Key
2: Returning back key
1: Waiting for key
1: Got the Key
2: Waiting for Key
1: Returning back key
2: Got the Key
2: Returning back key
```

```
module tb_top;
    semaphore key;

    initial begin
        key = new (1);
        fork
            personA ();
            personB ();
            #25 personA ();
        join_none
    end

    task getRoom (bit [1:0] id);
        $display ("[%0t] Trying to get a room for id[%0d] ...", $time, id);
        key.get (1);
        $ display ("[%0t] Room Key retrieved for id[%0d]", $time, id);
    endtask

    task putRoom (bit [1:0] id);
        $display ("[%0t] Leaving room id[%0d] ...", $time, id);
        key.put (1);
        $display ("[%0t] Room Key put back id[%0d]", $time, id);
    endtask

    task personA ();
        getRoom (1);
        #20 putRoom (1);
    endtask

    task personB ();
        #5  getRoom (2);
        #10 putRoom (2);
    endtask
endmodule
```

```
[0] Trying to get a room for id[1] ...
[0] Room Key retrieved for id[1]
[5] Trying to get a room for id[2] ...
[20] Leaving room id[1] ...
[20] Room Key put back id[1]
[20] Room Key retrieved for id[2]
[25] Trying to get a room for id[1] ...
[30] Leaving room id[2] ...
[30] Room Key put back id[2]
[30] Room Key retrieved for id[1]
[50] Leaving room id[1] ...
[50] Room Key put back id[1]
```

```
[0] Trying to get a room for id[1] ...
[0] Room Key retrieved for id[1]
[5] Trying to get a room for id[2] ...
[20] Leaving room id[1] ...
[20] Room Key put back id[1]
[20] Room Key retrieved for id[2]
[25] Trying to get a room for id[1] ...
[30] Leaving room id[2] ...
[30] Room Key put back id[2]
[30] Room Key retrieved for id[1]
[50] Leaving room id[1] ...
[50] Room Key put back id[1]
```

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema
  initial begin
    sema=new(1); //creating sema with '1' key
    fork
      display(); //process-1
      display(); //process-2
    join
  end

  //display method
  task automatic display();
    sema.get(); //getting '1' key from sema
    $display($time,"\tCurrent Simulation Time");
    #30;
    sema.put(); //putting '1' key to sema
  endtask
endmodule
```

Simulator Output

```
0 Current Simulation Time
30 Current Simulation Time
```

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema
  initial begin
    sema=new(4); //creating sema with '4' keys
    fork
      display(); //process-1
      display(); //process-2
    join
  end
  //display method
  task automatic display();
    sema.get(4); //getting '4' keys from sema
    $display($time,"\tCurent Simulation Time");
    #30;
    sema.put(4); //putting '4' keys to sema
  endtask
endmodule
```

Simulator Output

0 Current Simulation Time

30 Current Simulation Time

Putting back more keys

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema

  initial begin
    sema=new(1); //creating sema with '1' keys
    fork
      display(1); //process-1
      display(2); //process-2
      display(3); //process-3
    join
  end

  //display method
  task automatic display(int key);
    sema.get(key); //getting 'key' number of keys from sema
    $display($time,"\tCurrent Simulation Time, Got %0d keys",key);
    #30;
    sema.put(key+1); //putting 'key' number of keys to sema
  endtask
endmodule
```

Simulator Output

```
0 Current Simulation Time, Got 1 keys
30 Current Simulation Time, Got 2 keys
60 Current Simulation Time, Got 3 keys
```

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema

  initial begin
    sema=new(4); //creating sema with '4' keys
    fork
      display(); //process-1
      display(); //process-2
      display(); //process-3
    join
  end

  //display method
  task automatic display();
    sema.get(2); //getting '2' keys from sema
    $display($time,"\tCurrent Simulation Time");
    #30;
    sema.put(2); //putting '2' keys to sema
  endtask
endmodule
```

Simulator Output

```
0 Current Simulation Time
0 Current Simulation Time
30 Current Simulation Time
```

At the same time, two processes will get access to the method and the other process will be blocked until the one other process puts the key.

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema

  initial begin
    sema=new(1); //creating sema with '1' keys
    fork
      display(1); //process-1
      display(2); //process-2
      display(3); //process-3
    join
  end

  //display method
  task automatic display(int key);
    sema.get(key); //getting 'key' number of keys from sema
    $display($time,"\tCurrent Simulation Time, Got %0d keys",key);
    #30;
    sema.put(key+1); //putting 'key' number of keys to sema
  endtask
endmodule
```

Simulator Output

0 Current Simulation Time, Got 1 keys

30 Current Simulation Time, Got 2 keys

60 Current Simulation Time, Got 3 keys

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema

  initial begin
    sema=new(4); //creating sema with '4' keys
    fork
      display(2); //process-1
      display(3); //process-2
      display(2); //process-3
      display(1); //process-4
    join
  end

  //display method
  task automatic display(int key);
    sema.get(key); //getting 'key' number of keys from sema
    $display($time,"\tCurrent Simulation Time, Got %0d keys",key);
    #30;
    sema.put(key); //putting 'key' number of keys to sema
  endtask
endmodule
```

```
0 Current Simulation Time, Got 2 keys
0 Current Simulation Time, Got 2 keys
30 Current Simulation Time, Got 1 keys
30 Current Simulation Time, Got 3 keys
```

```
module semaphore_ex;
  semaphore sema; //declaring semaphore sema

  initial begin
    sema=new(4); //creating sema with '4' keys
    fork
      display(4); //process-1
      display(4); //process-2
    join
  end

  //display method
  task automatic display(int key);
    sema.try_get(key); //getting 'key' number of keys from sema
    $display($time,"\tCurrent Simulation Time, Got %0d keys",key);
    #30;
    sema.put(key); //putting 'key' number of keys to sema
  endtask
endmodule
```

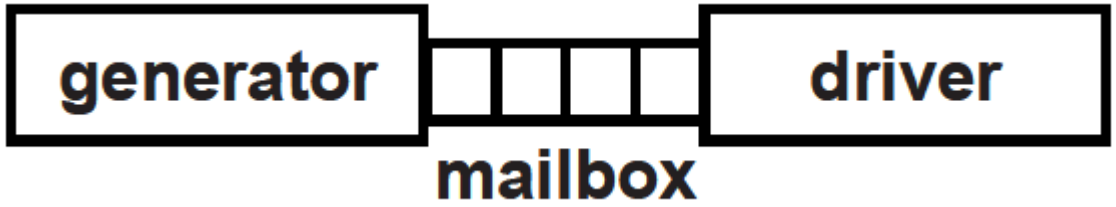
Simulator Output

```
0 Current Simulation Time, Got 4 keys
0 Current Simulation Time, Got 4 keys
```

Creating semaphore with '4' key, try_get() will check for the keys if the keys are not available simulation will proceed(non-blocking).

SystemVerilog Mailbox

- A SystemVerilog mailbox is a way to allow different processes to exchange data between each other.
- It is similar to a real postbox where letters can be put into the box and a person can retrieve those letters later on.
- SystemVerilog mailboxes are created as having either a *bounded* or *unbounded* queue size.
- A bounded mailbox can only store a limited amount of data, and if a process attempts to store more messages into a full mailbox, it will be suspended until there's enough room in the mailbox.
- An *unbounded* mailbox has unlimited size.



There are two types:

- Generic Mailbox that can accept items of any data type
- Parameterized Mailbox that can accept items of only a specific data type

Generic Mailbox (type-less mailbox)

The default mailbox is type-less. that is, a single mailbox can send and receive data of any type.

```
mailbox mailbox_name;
```

Parameterized mailbox (mailbox with particular type)

Parameterized mailbox is used to transfer a data of particular type.

```
mailbox#(type) mailbox_name;
```

- new() : Create a mailbox
- put() : Place a message in a mailbox
- try_put() : Try to place a message in a mailbox without blocking
- get() : Retrieve a message from a mailbox
- try_get() : Try to retrieve a message from a mailbox without blocking
- peek() : Copy a message from a mailbox without removing
- try_peek() : Try to copy a message from a mailbox without blocking & removing
- num() : Retrieve the number of messages in the mailbox

new( );

Mailboxes are created with the new() method.

```
mailbox_name = new();     // Creates unbounded mailbox and returns mailbox handle

mailbox_name = new(m_size); //Creates bounded mailbox with size m_size and returns mailbox
handle ,where m_size is integer  variable
```

SystemVerilog Mailbox vs Queue

- Although a SystemVerilog mailbox essentially behaves like a queue, it is quite different from the queue data type.
- A simple queue can only push and pop items from either the front or the back. However, a mailbox is a builtin class that uses semaphores (/systemverilog/systemverilog-semaphore) to have atomic control the push and pop from the queue.
- Moreover, you cannot access a given index within the mailbox queue, but only retrieve items in FIFO order.
- you cannot access a given index within the mailbox queue, but only retrieve items in FIFO order.

Where is a mailbox used ?

A SystemVerilog mailbox is typically used when there are multiple threads running in parallel and want to share data for which a certain level of determinism is required.

```
module tb;
  // Create a new mailbox that can hold utmost 2 items
    mailbox   mbx = new(2);

    // Block1: This block keeps putting items into the mailbox
    // The rate of items being put into the mailbox is 1 every ns
    initial begin
    for (int i=0; i < 5; i++) begin
          #1 mbx.put (i);
          $display ("[%0t] Thread0: Put item #%0d, size=%0d", $time, i,
mbx.num());
        end
    end

    // Block2: This block keeps getting items from the mailbox
    // The rate of items received from the mailbox is 2 every ns
  initial begin
    forever begin
      int idx;
      #2 mbx.get (idx);
            $display ("[%0t] Thread1: Got item #%0d, size=%0d", $time,
idx, mbx.num());
    end
  end
endmodule
```

```
ncsim> run
[1] Thread0: Put item #0, size=1
[2] Thread1:   Got item #0, size=0
[2] Thread0: Put item #1, size=1
[3] Thread0: Put item #2, size=2
[4] Thread1:   Got item #1, size=1
[4] Thread0: Put item #3, size=2
[6] Thread1:   Got item #2, size=2
[6] Thread0: Put item #4, size=2
[8] Thread1:   Got item #3, size=1
[10] Thread1:   Got item #4, size=0
ncsim: *W,RNQUIE: Simulation is complete.
```

```
program mailbox_check;

mailbox mbx = new();
int msg,num;

initial begin
  for(int i=0; i<5; i++) #10 send_to_mbx(i*100);

  num = mbx.num();
  $display($time,"Number of packets in the Mailbox : %0d",num);

  for(int i=0; i<5; i++) #20 retrive_from_mbx(msg);

  #20 send_to_mbx(500);
  #10 retrive_from_mbx(msg);

  send_to_mbx(700);
  send_to_mbx(900);

  retrive_from_mbx(msg);
  copy_from_mbx(msg);
  retrive_from_mbx(msg);

  #500 $finish;
end
```

```
task send_to_mbx(int i);
 $display($time,"Sending message to Mailbox : %0d", i);
 mbx.put(i);
endtask

task retrive_from_mbx(int msg);
 mbx.get(msg);
 $display($time,"Retrieved message using get : %0d", msg);
endtask

task copy_from_mbx(int msgt);
 mbx.peek(msg);
 $display($time,"Copied message using peek : %0d", msg);
endtask

endprogram
```

```
                      10Sending message to Mailbox : 0
                      20Sending message to Mailbox : 100
                      30Sending message to Mailbox : 200
                      40Sending message to Mailbox : 300
                      50Sending message to Mailbox : 400
                      50Number of packets in the Mailbox : 5
                      70Retrieved message using get : 0
                      90Retrieved message using get : 100
                     110Retrieved message using get : 200
                     130Retrieved message using get : 300
                     150Retrieved message using get : 400
                     170Sending message to Mailbox : 500
                     180Retrieved message using get : 500
                     180Sending message to Mailbox : 700
                     180Sending message to Mailbox : 900
                     180Retrieved message using get : 700
                     180Copied message using peek : 900
                     180Retrieved message using get : 900
Simulation complete via $finish(1) at time 680 NS + 1
```

- The try_get() method attempts to retrieve a message from a mailbox without blocking.If the mailbox is empty, then the method returns 0.
- If the type of the message variable and the type of the message in the mailbox are different, the method returns a negative integer.
- The try_peek() method attempts to copy a message from a mailbox without blocking & without removing from mailbox queue.If the mailbox is empty, then the method returns 0. If there is a type mismatch the method returns a negative integer.

```
program mailbox_try_check;

mailbox mbx = new();
int msg,ret;
string str;

initial begin

  //As there is no message in the mbox, try_get returns 0
  ret = mbx.try_get(msg);
  $display("  Empty Mailbox - Return %0d", ret);

  //send a message to mbox
  send_to_mbx(555);

  //As type is different, try_get returns a negative number
  ret = mbx.try_get(str);
  $display("  try_get fails due to type mismatch - Return %0d", ret);

  //send another message to mbox
  send_to_mbx(666);

  //try_peek
  ret = mbx.try_peek(msg);
  $display("  try_peek() Message copied  - Return %0d : Message = %0d", ret, msg);

  //try_get
  ret = mbx.try_get(msg);
  $display("  try_get() Message retrieved - Return %0d : Message = %0d", ret, msg);
```

```
  //try_peek
  ret = mbx.try_peek(msg);
  $display("  try_peek() Message copied  - Return %0d : Message = %0d", ret, msg);

  //try_get
  ret = mbx.try_get(msg);
  $display("  try_get() Message retrieved - Return %0d : Message = %0d", ret, msg);

  #50 $finish;
end

task send_to_mbx(int i);
 $display("  Sending Packet to Mailbox - put() : %0d", i);
 mbx.put(i);
endtask

endprogram
```

```
  Empty Mailbox - Return 0
  Sending Packet to Mailbox - put() : 555
  try_get fails due to type mismatch - Return -1
  Sending Packet to Mailbox - put() : 666
  try_peek() Message copied  - Return 1 : Message = 555
  try_get() Message retrieved - Return 1 : Message = 555
  try_peek() Message copied  - Return 1 : Message = 666
  try_get() Message retrieved - Return 1 : Message = 666
Simulation complete via $finish(1) at time 50 NS + 1
./b.sv:40    #50 $finish;
```

- The try_get() method attempts to retrieve a message from a mailbox without blocking.If the mailbox is empty, then the method returns 0.
- If the type of the message variable and the type of the message in the mailbox are different, the method returns a negative integer.
- The try_peek() method attempts to copy a message from a mailbox without blocking & without removing from mailbox queue.If the mailbox is empty, then the method returns 0. If there is a type mismatch the method returns a negative integer.

```
program mailbox_bounded_check;

mailbox mbx = new(4);
int msg,ret;
string str;

initial begin

  for(int i=0; i<7; i++) #10 try_put_to_mbx(i*100);
  for(int i=0; i<5; i++) #20 try_get_from_mbx(msg);

  #50 $finish;
end

task try_put_to_mbx(int i);
 int  ok;
 ok = mbx.try_put(i);
 if(ok > 0) $display(" Sent message to Mailbox - try_put() : %0d  Return : %0d", i, ok);
 else  $display(" Fails sending message to Mailbox - try_put() : %0d  Return : %0d", i, ok);
endtask

task try_get_from_mbx(int msg);
 int ok;
 ok = mbx.try_get(msg);
 if(ok > 0) $display(" Retrieved message from Mailbox - try_get() : %0d  Return : %0d", msg, ok);
 else  $display(" Fails retrieving message from Mailbox - try_get() : %0d  Return : %0d", msg, ok);
endtask

endprogram
```

```
ncsim> run
 Sent message to Mailbox - try_put() : 0  Return : 1
 Sent message to Mailbox - try_put() : 100  Return : 1
 Sent message to Mailbox - try_put() : 200  Return : 1
 Sent message to Mailbox - try_put() : 300  Return : 1
 Fails sending message to Mailbox - try_put() : 400  Return : 0
 Fails sending message to Mailbox - try_put() : 500  Return : 0
 Fails sending message to Mailbox - try_put() : 600  Return : 0
 Retrieved message from Mailbox - try_get() : 0  Return : 1
 Retrieved message from Mailbox - try_get() : 100  Return : 1
 Retrieved message from Mailbox - try_get() : 200  Return : 1
 Retrieved message from Mailbox - try_get() : 300  Return : 1
 Fails retrieving message from Mailbox - try_get() : 0  Return : 0
Simulation complete via $finish(1) at time 220 NS + 1
*/
```

```
//-------------------------------------------------
// Packet
//-------------------------------------------------
class packet;
  rand bit [7:0] addr;
  rand bit [7:0] data;

  //Displaying randomized values
  function void post_randomize();
    $display("Packet::Packet Generated");
    $display("Packet::Addr=%0d,Data=%0d",addr,data);
  endfunction
endclass
```

```
//-------------------------------------------------------------
//Generator - Generates the transaction packet and send to driver
//-------------------------------------------------------------
class generator;
  packet pkt;
  mailbox m_box;
  //constructor, getting mailbox handle
  function new(mailbox m_box);
    this.m_box = m_box;
  endfunction
  task run;
    repeat(2) begin
      pkt = new();
      pkt.randomize(); //generating packet
      m_box.put(pkt); //putting packet into mailbox
      $display("Generator::Packet Put into Mailbox");
      #5;
    end
  endtask
endclass
```

```
//-------------------------------------------------------------
// Driver - Gets the packet from generator and display's the pack
//-------------------------------------------------------------
class driver;
  packet pkt;
  mailbox m_box;

  //constructor, getting mailbox handle
  function new(mailbox m_box);
    this.m_box = m_box;
  endfunction

  task run;
    repeat(2) begin
      m_box.get(pkt); //getting packet from mailbox
      $display("Driver::Packet Recived");
      $display("Driver::Addr=%0d,Data=%0d\n",pkt.addr,pkt.data);
    end
  endtask
endclass
```

```
//------------------------------------------------------------------------
//     tbench_top
//------------------------------------------------------------------------
module mailbox_ex;
  generator gen;
  driver    dri;
  mailbox m_box; //declaring mailbox m_box

  initial begin
    //Creating the mailbox, Passing the same handle to generator and driver,
    //because same mailbox should be shared in-order to communicate.
    m_box = new(); //creating mailbox

    gen = new(m_box); //creating generator and passing mailbox handle
    dri = new(m_box); //creating driver and passing mailbox handle
    $display("----------------------------------------");
    fork
      gen.run(); //Process-1
      dri.run(); //Process-2
    join
    $display("----------------------------------------");
  end
endmodule
```

```
----------------------------------
Packet::Packet Generated
Packet::Addr=3,Data=38
Generator::Packet Put into Mailbox
Driver::Packet Recived
Driver::Addr=3,Data=38

Packet::Packet Generated
Packet::Addr=118,Data=92
Generator::Packet Put into Mailbox
Driver::Packet Recived
Driver::Addr=118,Data=92
```

```
program meain ;
mailbox my_mailbox;
initial begin
my_mailbox = new();
if (my_mailbox)
begin
fork
put_packets();
get_packets();
#10000;
join_any
end

#(1000);
$display("END of Program");
end
```

```
task put_packets();
integer i;
begin
for (i=0; i<10; i++)
begin
#(10);
my_mailbox.put(i);
$display("Done putting packet %d @time %d",i, $time);

end
end
endtask

task get_packets();
integer i,packet;
begin
for (int i=0; i<10; i++)
begin
my_mailbox.get(packet);
$display("Got packet %d @time %d", packet, $time);
end
end
endtask
endprogram
```

```
Done putting packet 0 @time 10
Got packet 0 @time 10
Done putting packet 1 @time 20
Got packet 1 @time 20
Done putting packet 2 @time 30
Got packet 2 @time 30
Done putting packet 3 @time 40
Got packet 3 @time 40
Done putting packet 4 @time 50
Got packet 4 @time 50
Done putting packet 5 @time 60
Got packet 5 @time 60
Done putting packet 6 @time 70
Got packet 6 @time 70
Done putting packet 7 @time 80
Got packet 7 @time 80
Done putting packet 8 @time 90
Got packet 8 @time 90
Done putting packet 9 @time 100
Got packet 9 @time 100
END of Program
```

SystemVerilog Events

- Events are static objects useful for synchronization between the process.
- Events operations are of two staged processes in which one process will trigger the event, and the other processes will wait for an event to be triggered.

•Events are triggered using -> operator or ->> operator
•wait for an event to be triggered using @ operator or wait() construct

-> operator

Named events are triggered via the -> operator.
Triggering an event unblocks all processes currently waiting on that event.

->> operator

Non-blocking events are triggered using the ->> operator.

@ operator

- wait for an event to be triggered is via the event control operator, @.

```
@(event_name.triggered);
```

•The @ operator blocks the calling process until the given event is triggered.
•For a trigger to unblock a process waiting on an event, the waiting process must execute the @ statement before the triggering process executes the trigger operator, ->

Note: If the trigger executes first, then the waiting process remains blocked.

wait operator

- If the event triggering and waiting for event trigger with @ operator happens at the same time, @ operator may miss detecting the event trigger.
- Whereas wait(); construct will detect the event triggering.

```
wait(event_name.triggered);
```

wait_order();

The wait_order construct is blocking the process until all of the specified events are triggered in the given order (left to right).
event trigger with out of order will not unblock the process.

```
wait_order(a,b,c);
```

Blocks the process until events a, b, and c trigger in the order a –> b –> c. If the events trigger out of order, a run-time error is generated.

Example:

```
wait_order( a, b, c ) else $display( "Error: events out of order" );
```

```
bit success;
wait_order( a, b, c ) success = 1; else success = 0;
```

the event waiting with @ operator

```
module events_ex;
  event ev_1; //declaring event ev_1

  initial begin
    fork
      //process-1, triggers the event
      begin
        #40;
        $display($time,"\tTriggering The Event");
        ->ev_1;
      end

      //process-2, wait for the event to trigger
      begin
        $display($time,"\tWaiting for the Event to trigger");
        @(ev_1.triggered);
        $display($time,"\tEvent triggered");
      end
    join
  end
endmodule
```

Simulator Output

```
 0 Waiting for the Event to trigger
40 Triggering The Event
40 Event triggered
```

trigger first and then waiting for a trigger

```
module events_ex;
  event ev_1; //declaring event ev_1

  initial begin
    fork
      //process-1, triggers the event
      begin
        #40;
        $display($time,"\tTriggering The Event");
        ->ev_1;
      end

      //process-2, wait for the event to trigger
      begin
        $display($time,"\tWaiting for the Event to trigger");
        #60;
        @(ev_1.triggered);
        $display($time,"\tEvent triggered");
      end
    join
  end
  initial begin
    #100;
    $display($time,"\tEnding the Simulation");
    $finish;
  end
endmodule
```

- event triggering happens first and then the waiting for trigger happens.
- As the waiting happens later, it will be blocking, so the statements after the wait for the trigger will not be executed.

Simulator Output

```
0  Waiting for the Event to trigger
40  Triggering The Event
100 Ending the Simulation
```

```
module events_ex;
  event ev_1; //declaring event ev_1
  event ev_2; //declaring event ev_2
  event ev_3; //declaring event ev_3

  initial begin
    fork
      //process-1, triggers the event ev_1
      begin
        #6;
        $display($time,"\tTriggering The Event ev_1");
        ->ev_1;
      end
      //process-2, triggers the event ev_2
      begin
        #2;
        $display($time,"\tTriggering The Event ev_2");
        ->ev_2;
      end
      //process-3, triggers the event ev_3
      begin
        #8;
        $display($time,"\tTriggering The Event ev_3");
        ->ev_3;
      end
      //process-4, wait for the events to trigger in order of ev_2,ev_1 and ev_3
      begin
        $display($time,"\tWaiting for the Event's to trigger");
        wait_order(ev_2,ev_1,ev_3)
          $display($time,"\tEvent's triggered Inorder");
        else
          $display($time,"\tEvent's triggered Out-Of-Order");
      end
    join
  end
endmodule
```

Simulator Output

```
0 Waiting for the Event's to trigger
2 Triggering The Event ev_2
6 Triggering The Event ev_1
8 Triggering The Event ev_3
8 Event's triggered Inorder
```

```
module events_ex;
  event ev_1; //declaring event ev_1
  event ev_2; //declaring event ev_2
  event ev_3; //declaring event ev_3

  initial begin
    fork
      //process-1, triggers the event ev_1
      begin
        #6;
        $display($time,"\tTriggering The Event ev_1");
        ->ev_1;
      end

      //process-2, triggers the event ev_2
      begin
        #2;
        $display($time,"\tTriggering The Event ev_2");
        ->ev_2;
      end

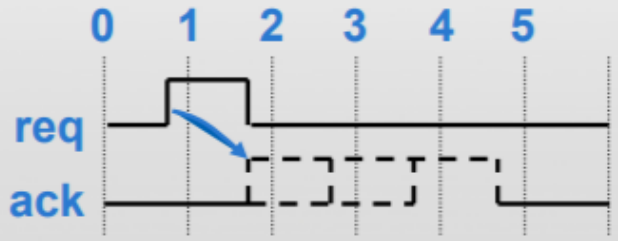
      //process-3, triggers the event ev_3
      begin
        #1;
        $display($time,"\tTriggering The Event ev_3");
        ->ev_3;
      end
      //process-4, wait for the events to trigger in order of ev_2,ev_1 and ev_3
      begin
        $display($time,"\tWaiting for the Event's to trigger");
        wait_order(ev_2,ev_1,ev_3)
          $display($time,"\tEvent's triggered Inorder");
        else
          $display($time,"\tEvent's triggered Out-Of-Order");
      end
    join
  end
endmodule
```

Simulator Output

```
0 Waiting for the Event's to trigger
1 Triggering The Event ev_3
1 Event's triggered Out-Of-Order
2 Triggering The Event ev_2
6 Triggering The Event ev_1
```

What Is An Assertion?

- **An assertion is a statement that a certain property must be true**



Design Specification:
After the request signal is asserted, the acknowledge signal must arrive 1 to 3 clocks later

- **Assertions are used to:**
  - Document design intent (e.g.: every request has an acknowledge)
  - Verify design meets the specification over simulation time
  - Verify design assumptions (e.g.: state value is one-hot)
  - Localize where failures occur in the design instead of at the output
  - Provide semantics for formal verification
  - Describe functional coverage points
  - And… requires clarifying ambiguities in spec

Which one of these is the most important to you in your projects?

accellera

SystemVerilog Assertions

- Assertions are primarily used to validate the behavior of a design.
- An assertion is a check embedded in design or bound to a design unit during the simulation.
- Warnings or errors are generated on the failure of a specific condition or sequence of events.

Assertions are used to,

- Check the occurrence of a specific condition or sequence of events.
- Provide functional coverage.

There are two kinds of assertions:

- Immediate Assertions
- Concurrent Assertions

Immediate Assertions:

- *Immediate assertions* check for a condition at the current simulation time.
- An *immediate assertion* is the same as an if..else statement with assertion control.
- Immediate assertions have to be placed in a procedural block definition.

Syntax

```
label: assert(expression) action_block;
```

- *The optional statement label (identifier and colon) creates a named block around the assertion statement*
- *The action block is executed immediately after the evaluation of the assert expression*
- *The action_block specifies what actions are taken upon success or failure of the assertion*

action_block;

```
pass_statement; else fail_statement;
```

•The **pass statement** is executed if the expression evaluates to true
•The statement associated with else is called a **fail statement** and is executed if the expression evaluates to false
•Both pass and fail statements are **optional**
•Since the assertion is a statement that something must be true, the failure of an assertion shall have a severity associated with it.
•By **default**, the severity of an assertion failure is an **error**.
•Other severity levels can be specified by including one of the following severity system tasks in the fail statement:

- $fatal is a run-time fatal.
- $error is a run-time error.
- $warning is a run-time warning, which can be suppressed in a tool-specific manner.
- $info indicates that the assertion failure carries no specific severity.

•If an assertion fails and no else clause is specified, the tool shall, by default call $error.

```
//With Pass and Fail statement; Fail verbosity info;
assert(expression) $display("expression evaluates to true"); else $display("expression
evaluates to false");

//Only With Pass statement;
assert(expression) $display("expression evaluates to true");

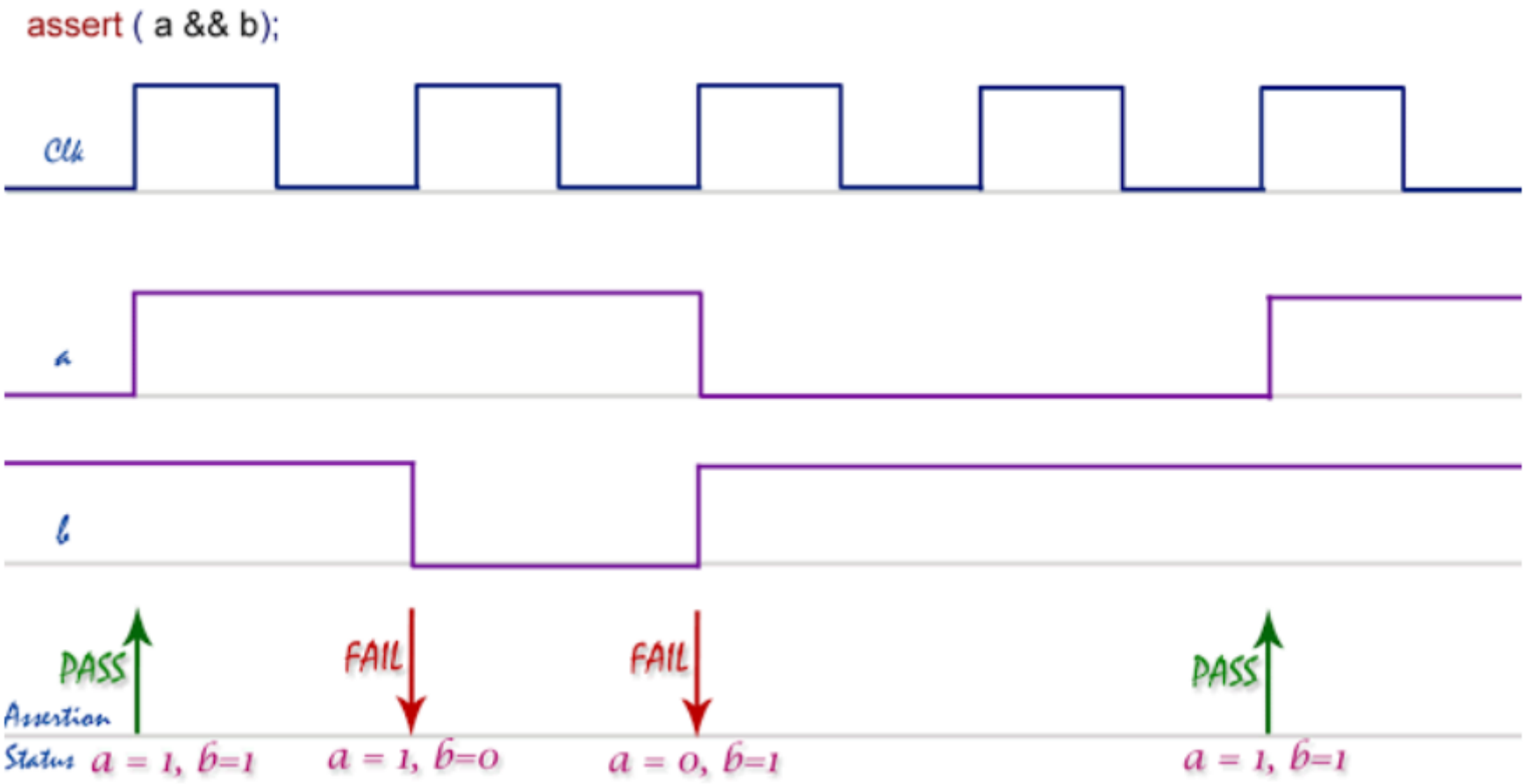
//With Pass and Fail statement; Fail verbosity fatal;
assert(expression) $display("expression evaluates to true"); else $fatal("expression
evaluates to false");

//Only With Fail statement; Multiple statements in Faile condition and Fail verbosity fatal;
assert(expression)
      else begin
        …….
        …….
        $fatal("expression evaluates to false");
      end

//Only With Fail statement; Fail verbosity warning;
assert(expression) else $warning("expression evaluates to false");

//With Label and Fail statement; Fail verbosity warning;
label: assert(expression) else $warning("expression evaluates to false");
```

```
always @(posedge clk) assert (a && b);
```



```
module asertion_ex;
  bit clk,a,b;

  //clock generation
  always #5 clk = ~clk;

  //generating 'a'
  initial begin
    a=1;
    b=1;
    #15 b=0;
    #10 b=1;
        a=0;
    #20 a=1;
    #10;
    $finish;
  end

  //Immediate assertion
  always @(posedge clk) assert (a && b);

endmodule
```

```
ncsim: *E,ASRTST (./testbench.sv,23): (time 15 NS) Assertion asertion_ex.__assert_1 has failed
ncsim: *E,ASRTST (./testbench.sv,23): (time 25 NS) Assertion asertion_ex.__assert_1 has failed
ncsim: *E,ASRTST (./testbench.sv,23): (time 35 NS) Assertion asertion_ex.__assert_1 has failed
Simulation complete via $finish(1) at time 55 NS + 0
```

Concurrent Assertions:

Concurrent assertions check the sequence of events spread over multiple clock cycles.

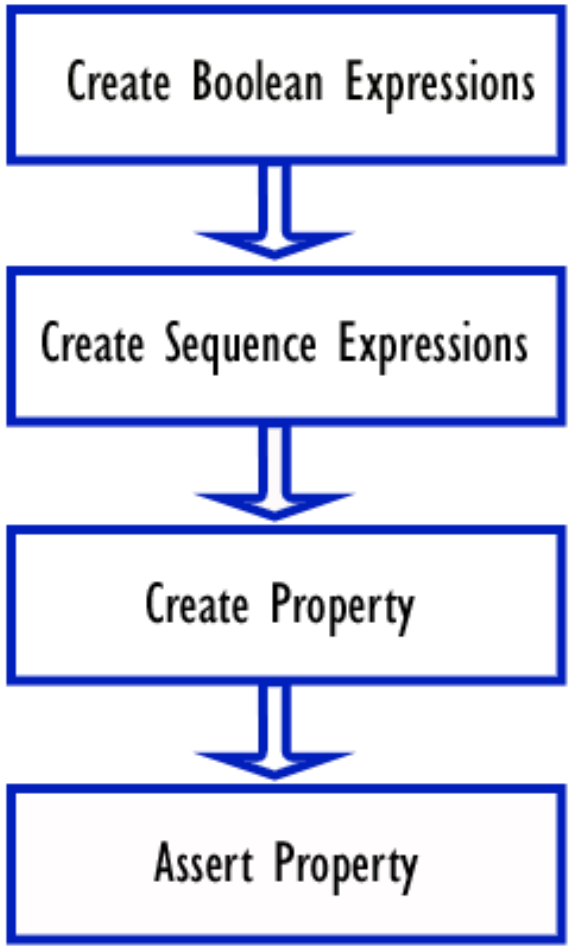
- The concurrent assertion is evaluated only at the occurrence of a clock tick
- The test expression is evaluated at clock edges based on the sampled values of the variables involved
- It can be placed in a procedural block, a module, an interface or a program definition

```
c_assert: assert property(@(posedge clk) not(a && b));
```

The Keyword differentiates the immediate assertion from the concurrent assertion is "property."

Building blocks of SVA

Below diagram shows the steps involved in the creation of an SVA checker,



```
if (A == B) ... // Simply checks if A equals B
assert (A == B); // Asserts that A equals B; if not, an error is generated
```

"The Read and Write signals should never be asserted together."

```
assert property ( !(Read && Write) );
```

"A Request should be followed by an Acknowledge occurring no more than two clocks after the Request is asserted."

```
assert property ( @(posedge Clock) Req |-> ##[1:2] Ack);
```

Boolean expressions

The functionality is represented by the combination of multiple logical events. These events could be simple Boolean expressions.

Sequence

Boolean expression events that evaluate over a period of time involving single/multiple clock cycles. SVA provides a keyword to represent these events called "sequence."

Syntax

```
sequence name_of_sequence;
  ......
endsequence
```

Property

- A number of sequences can be combined logically or sequentially to create more complex sequences.
- SVA provides a keyword to represent these complex sequential behaviors called "property".

Syntax

```
property name_of_property;
  test expression or
  complex sequence expressions
endproperty
```

Assert

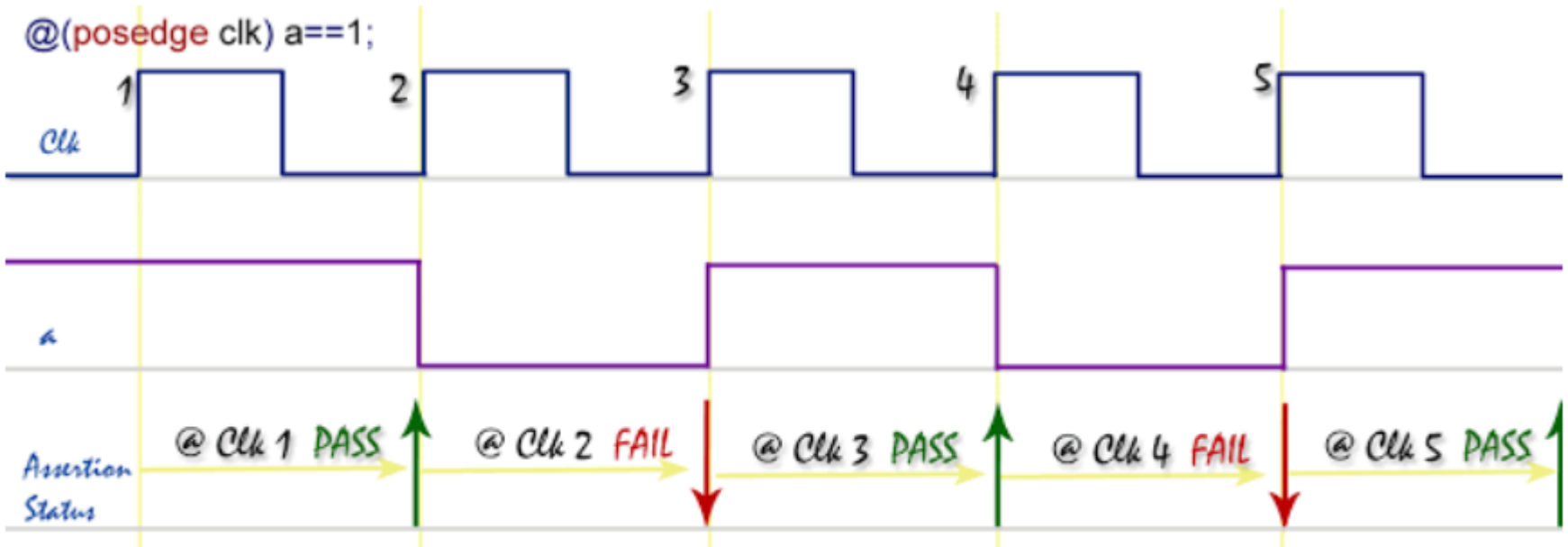
- The property is the one that is verified during a simulation.
- It has to be asserted to take effect during a simulation. SVA provides a keyword called "assert" to check the property.

Syntax

```
assertion_ name: assert_property( property_name );
```

SVA Sequence example

```
sequence seq_1;
  @(posedge clk) a==1;
endsequence
```



sequence seq_1 checks that the signal "a" is high on every positive edge of the clock. If the signal "a" is not high on any positive clock edge, the assertion will fail.

Non-overlapped implication

The non-overlapped implication is denoted by the symbol |=>
If there is a match on the antecedent, then the consequent expression is evaluated in the next clock cycle.

```
property p;
  @(posedge clk) a |=> b;
endproperty
a: assert property(p);
```

if signal "a" is high on a given positive clock edge, then signal "b" should be high on the next clock edge.

The implication with a fixed delay on the consequent

```
property p;
  @(posedge clk) a |-> ##2 b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" should be high after 2 clock cycles.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |=> b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
Assertion 'a_1' FAILED at time: 25ns (3 clk), testbench.sv(25), scope: asertion_ex, start-time: 15ns (2 clk)
Assertion 'a_1' FAILED at time: 45ns (5 clk), testbench.sv(25), scope: asertion_ex, start-time: 35ns (4 clk)
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##2 b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
Assertion 'a_1' FAILED at time: 35ns (4 clk), testbench.sv(25), scope: asertion_ex, start-time: 15ns (2 c
```

The implication with a sequence as an antecedent

```
sequence seq_1;
 (a && b) ##1 c;
endsequence

sequence seq_2;
  ##2 !d;
endsequence

property p;
  @(posedge clk) seq_1 |-> seq_2;
endpeoperty
a: assert property(p);
```

Property checks that, if the sequence seq_1 is true on a given positive edge of the clock, then start checking the seq_2 (“d” should be low, 2 clock cycles after seq_1 is true).

```
module asertion_ex;
  bit clk,a,b,c,d;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  sequence seq_1;
    (a && b) ##1 c;
  endsequence

  sequence seq_2;
    ##2 !d;
  endsequence
```

```
  property p;
    @(posedge clk) seq_1 |-> seq_2;
  endproperty
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

Timing windows in SVA Checkers

```
property p;
  @(posedge clk) a |-> ##[1:4] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then within 1 to 4 clock cycles, the signal "b" should be high.

Overlapping timing window

```
property p;
  @(posedge clk) a |-> ##[0:4] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" should be high in the same clock cycle or within 4 clock cycles.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[1:4] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[0:4] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

Indefinite timing window

- The upper limit of the timing window specified in the right-hand side can be defined with a "$" sign which implies that there is no upper bound for timing.
- This is called the "eventuality" operator.
- The checker will keep checking for a match until the end of the simulation.

```
property p;
  @(posedge clk) a |-> ##[1:$] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" will be high eventually starting from the next clock cycle.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[1:$] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

SystemVerilog Repetition operators

```
property p;
  @(posedge clk) a |-> ##1 b ##1 b ##1 b;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 consecutive clock cycles.

The Consecutive repetition operator is used to specify that a signal or a sequence will match continuously for the number of clocks specified.

Syntax

```
signal [*n] or sequence [*n]

"n" is the number of repetitions.
```

with repetition operator above sequence can be re-written as,

```
property p;
  @(posedge clk) a |-> ##1 b[*3];
endproperty
a: assert property(p);
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##1 b[*3];
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

`a ##1 b [*3] ##1 c`	// Equiv. to **a ##1 b ##1 b ##1 b ##1 c**
`(a ##2 b) [*2]`	// Equiv. to **(a ##2 b ##1 a ##2 b)**

`(a ##2 b)[*1:3]`	// Equiv. to **(a ##2 b)**
	// or **(a ##2 b ##1 a ##2 b)**
	// or **(a ##2 b ##1 a ##2 b ##1 a ##2 b)**

go to repetition

The go-to repetition operator is used to specify that a signal will match the number of times specified not necessarily on continuous clock cycles.

```
signal [->n]
```

```
property p;
  @(posedge clk) a |-> ##1 b[->3] ##1 c;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 clock cycles followed by “c” should be high after ”b” is high for the third time.

```
module asertion_ex;
  bit clk,a,b,c;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##1 b[->3] ##1 c;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

SVA Methods

$rose

```
$rose(boolean expression or signal name)
```

returns true if the least significant bit of the expression changed to 1. Otherwise, it returns false.

```
sequence seq_rose;
  @(posedge clk) $rose(a);
endsequence
```

Sequence seq_rose checks that the signal "a" transitions to a value of 1 on every positive edge of the clock. If the transition does not occur, the assertion will fail.

$fell

```
$fell(boolean expression or signal name)
```

Sequence seq_fell checks that the signal "a" transitions to a value of 0 on every positive edge of the clock. If the transition does not occur, the assertion will fail.

```
sequence seq_fell;
  @(posedge clk) $fell(a);
endsequence
```

Sequence seq_fell checks that the signal "a" transitions to a value of 0 on every positive edge of the clock. If the transition does not occur, the assertion will fai

$stable

```
$stable(boolean expression or signal name)
```

returns true if the value of the expression did not change. Otherwise, it returns false.

```
sequence seq_stable;
  @(posedge clk) $stable(a);
endsequence
```

Sequence seq_stable checks that the signal "a" is stable on every positive edge of the clock.
If there is any transition occurs, the assertion will fail.

$past

```
$past(signal_name, number of clock cycles)
```

provides the value of the signal from the previous clock cycle.

```
property p;
  @(posedge clk) b |-> ($past(a,2) == 1);
endproperty
a: assert property(p);
```

Property checks that, in the given positive clock edge, if the “b” is high, then 2 cycles before that, a was high.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) b |-> ($past(a,2) == 1);
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

$past construct with clock gating

The $past construct can be used with a gating signal. on a given clock edge, the gating signal has to be true even before checking for the consequent condition.

```
$past (signal_name, number of clock cycles, gating signal)
```

```
Property p;
  @(posedge clk) b |-> ($past(a,2,c) == 1);
endproperty
a: assert property(p);
```

Property checks that, in the given positive clock edge, if the “b” is high, then 2 cycles before that, a was high only if the gating signal "c' is valid on any given positive edge of the clock.

Built-in system functions

- $onehot(expression)
  - checks that only one bit of the expression can be high on any given clock edge.
- $onehot0(expression)
  - checks only one bit of the expression can be high or none of the bits can be high on any given clock edge.
- $isunknown(expression)
  - checks if any bit of the expression is X or Z.
- $countones(expression)
  - counts the number of bits that are high in a vector.

```
a_1: assert property( @(posedge clk) $onehot(state) );
a_2: assert property( @(posedge clk) $onehot0(state) );
a_3: assert property( @(posedge clk) $isunknown(bus) ) ;
a_4: assert property( @(posedge clk) $countones(bus)> 1 );
```

Assert statement a_1 checks that the bit vector "state" is one-hot.
Assert statement a_2 checks that the bit vector "state" is zero one-hot.
Assert statement a_3 checks if any bit of the vector "bus" is X or Z.
Assert statement a_4 checks that the number of ones in the vector "bus" is greater than one.

disable iff

In certain design conditions, we don't want to proceed with the check if some condition is true. this can be achieved by using disable iff.

```
property p;
  @(posedge clk)
  disable iff (reset) a |-> ##1 b[->3] ##1 c;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 clock cycles followed by “c” should be high after ”b” is high for the third time. During this entire sequence, if reset is detected high at any point, the checker will stop.

ended

while concatenating the sequences, the ending point of the sequence can be used as a synchronization point. This is expressed by attaching the keyword "ended" to a sequence name.

```
sequence seq_1;
 (a && b) ##1 c;
endsequence

sequence seq_2;
  d ##[4:6] e;
endsequence

property p;
  @(posedge clk) seq_1.ended |-> ##2 seq_2.ended;
endpeoperty
a: assert property(p);
```

property checks that, sequence seq_1 and SEQ_2 match with a delay of 2 clock cycles in between them. the end point of the sequences does the synchronization.

```
assert property(@(posedge clk)
   disable iff (!reset)
   a |=> b ##1 c);
```

when a gets the value 1 that b takes the value 1 one cycle later and that c takes the value 1 one additional cycle later or that reset got the value 0:

```
always_ff @(posedge clk)
   case (state)
    s0 :
      state <= s1;
      assert property
         ($past(state) = s4);
```

Here, the concurrent assertion is placed inside a case statement. The check of the assertion is performed every rising edge of the clock and is started only if the branch `s0` of the case statement is executed.

A simple UP/DOWN COUNTER design is presented. Counter assertions deployed directly at the source can greatly reduce the time to debug since these assertions will point to the exact cause of a Counter error without the need for extensive back-tracing debug when design fails.

A simple UP/DOWN COUNTER design is presented as the DUT.

*) The counter has 8 bit data input and 8 bit data output
*) When ld_cnt_ is asserted (active Low), data_in is loaded and output to data_out

*) When count_enb (active High) is enabled (high) and
    *) updn_cnt is high, data_out = data_out+1;
    *) updn_cnt is low, data_out = data_out-1;

*) When count_enb is LOW, data_out = data_out;

```
module counter (
  input clk, rst_, ld_cnt_, updn_cnt, count_enb,
  input [7:0] data_in,
  output logic [7:0] data_out
  );

always @ (posedge clk or negedge rst_)
begin
  if (!rst_)
    begin
    `ifndef check1
       data_out <= 0;
    `endif
     end              //LOAD DATA
  else                if (!ld_cnt_)
  begin                 data_out <= data_in;

                      //HOLD DATA
                      `ifndef check2
                       else if (!count_enb)
                        data_out <= data_out;
                      `endif
```

```
    //COUNT DATA
    `ifdef check3
    else
      case (updn_cnt)
        1'b1: data_out <= data_out - 1;
        1'b0: data_out <= data_out + 1;
      endcase
    `else
    else
      case (updn_cnt)
        1'b1: data_out <= data_out + 1;
        1'b0: data_out <= data_out - 1;
      endcase
    `endif

  end
end
endmodule
```

Code assertions to check for the following conditions in the 'counter' design.

CHECK # 1. Check that when 'rst_' is asserted (==0) that data_out == 8'b0

CHECK # 2. Check that if ld_cnt_ is deasserted (==1) and count_enb is not enabled (==0) that data_out HOLDS it's previous value.

Disable this property if rst is low.

CHECK # 3. Check that if ld_cnt_ is deasserted (==1) and count_enb is enabled (==1) that if updn_cnt==1 the count goes UP and if updn_cnt==0 the count goes DOWN.

Disable this property if rst is low.

```
//---------------------------------
//      CHECK # 1. Check that when 'rst_' is asserted (==0) that data_out == 8'b0
//---------------------------------
`ifdef check1
property counter_reset;
  @(clk) disable iff (rst_) !rst_ |=> (data_out == 8'b0);
endproperty

counter_reset_check: assert property(counter_reset)
  else $display($stime,,,"\t\tCOUNTER  RESET CHECK FAIL:: rst_=%b data_out=%0d \n",
                                                    rst_,data_out);
`endif
```

```
//-----------------------------------
//       CHECK # 2. Check that if ld_cnt_ is deasserted (==1) and count_enb is not enabled
//                  (==0) that data_out HOLDS it's previous value.
//                  Disable this property 'iff (!rst)'
//-----------------------------------
`ifdef check2
property counter_hold;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & !count_enb) |=> data_out ===
$past(data_out);
endproperty

counter_hold_check: assert property(counter_hold)
  else $display($stime,,,"\t\tCOUNTER  HOLD CHECK FAIL:: counter HOLD \n");
`endif
```

```
// CHECK # 3. Check that if ld_cnt_ is deasserted (==1) and count_enb is
// enabled (==1) that if updn_cnt==1 the count goes UP and if updn_cnt==0 the
// count goes DOWN.
//---------------------------------

`ifdef check3
property counter_count;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & count_enb) |->
    if (updn_cnt) ##1 (data_out-8'h01) == $past(data_out)
    else          ##1 (data_out+8'h01) == $past(data_out);
endproperty

counter_count_check: assert property(counter_count)
  else $display($stime,,,"\t\tCOUNTER COUNT CHECK FAIL:: UPDOWN COUNT using
$past \n");
```

```
//-----------------------------------
// Alternate way of writing assertion for CHECK # 3
// Check for count using local variable
//-----------------------------------
/*
`ifdef check3
property counter_count_local;
logic[7:0] local_data;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & count_enb, local_data = data_out)
|->
    if (updn_cnt) ##1 (data_out == (local_data+8'h01))
    else          ##1 (data_out == (local_data-8'h01));
endproperty

counter_count_check: assert property(counter_count)
  else $display($stime,,,"\t\tCOUNTER COUNT CHECK FAIL:: UPDOWN COUNT using
$past \n");

`endif
*/
```